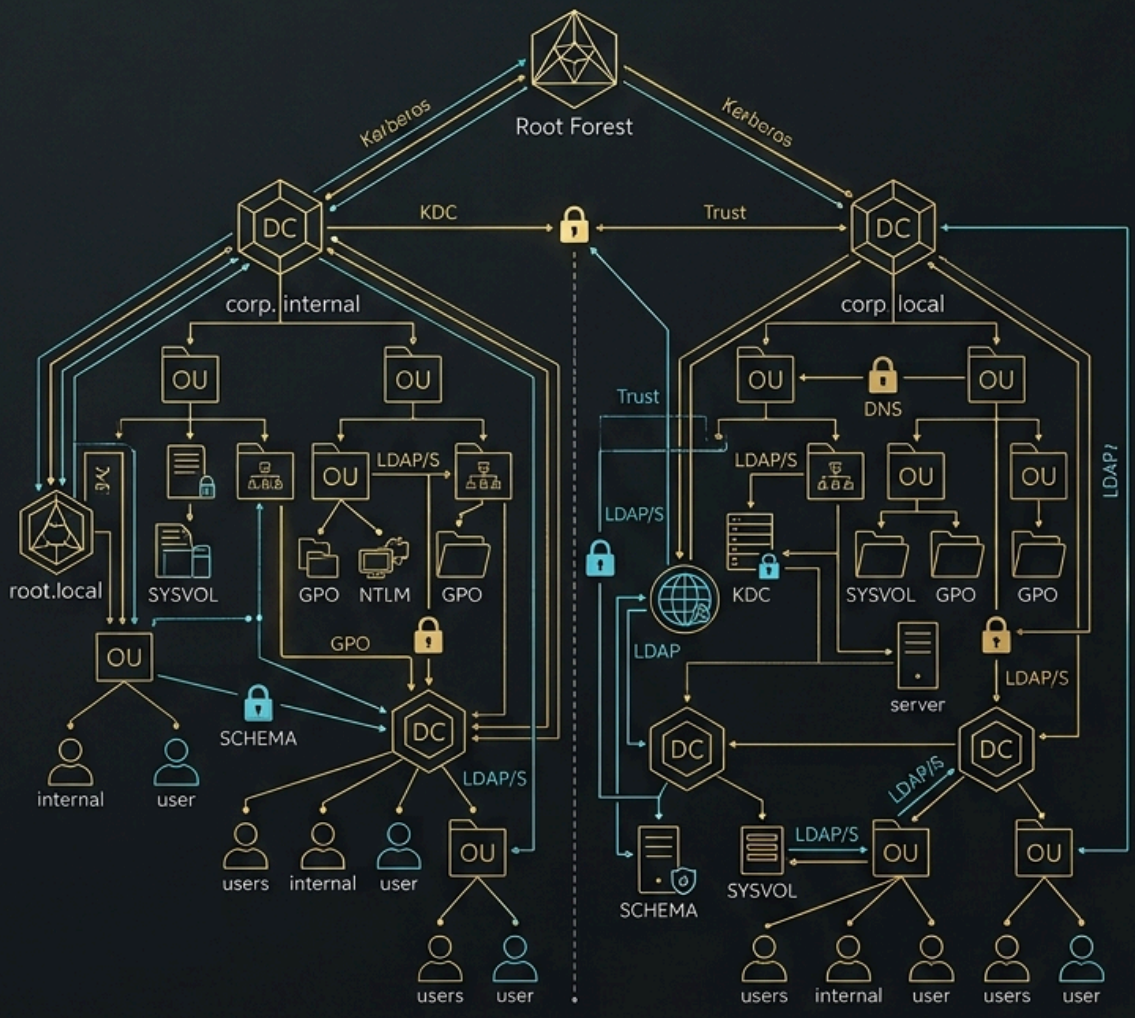
ACTIVE DIRECTORY
PROTOCOL INTERNALS

A Deep Dive into the Inner Workings of
AD Security, Schemas, and Authentication



By AHMED SOPHY ELZEKY (0xZiki)

INDEPENDENT OPEN-SOURCE EDITION

ACTIVE DIRECTORY PROTOCOL INTERNALS

Active Directory Protocol Internals — Volume 1: Foundations
First Edition: October 2026
Author: Ahmed Sophy Elzeky (0xZiki)
Published by The Active Directory Arsenal Open-Source Project

Copyright © 2026 Ahmed Sophy Elzeky.
This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0).
To view a copy of this license, visit: https://creativecommons.org/licenses/by-nc-sa/4.0/

Typeset natively using Typst on Arch Linux.

Conventions Used in This Book

The following typographic conventions are used throughout this architectural reference:

- **Serif body text:** Primary architectural narrative and protocol theory.
- **Monospace font (`Fira Code`):** Wire-level protocols, LDAP attributes, registry paths, and CLI commands.
- **Framed Schematics:** Unbreakable ASCII packet flows and multi-master replication architectures.
- **Security Context Callouts:** Strategic friction points bridging design decisions to offensive tradecraft.

Table of Contents

Active Directory Architecture — Architectural Structure 8
1. The Genesis & Architectural Purpose of AD DS 8
2. Logical Plane vs. Physical Plane 10
3. Syntax Breakdown & Concrete Object Anatomy 12
4. Database Internals: The ESE Jet Blue Engine & Encryption Hierarchy 14
5. Multi-Master Replication & Conflict Resolution 15
6. Flexible Single Master Operation (FSMO) Roles 17
7. Defensive Telemetry, Detection Engineering & Hardening 17
8. Authoritative Specifications, RFCs & Cross-Vault Links 19

Kerberos Authentication — The Primary Authentication Protocol 20
1. Architectural Foundations & The Core Dilemma 20
2. Scenario Walkthrough: From Workstation to File Share 23
3. Deep-Dive: Anatomy of Tickets, Keys & The PAC 29
4. Cryptographic Engines, Key Derivation & Ticket Storage 33
5. Delegation Architecture & Cross-Tier Identity Transfer 35
6. Architectural Friction Points & Vulnerability Foundations 38
7. Active Directory Schema, Attributes & Directory Objects 42
8. Defensive Telemetry, Detection Engineering & Hardening 44
9. Authoritative Specifications, RFCs & Cross-Vault Links 46

NTLM Authentication — The Fallback Protocol 48
1. Architectural Foundations & The Core Dilemma 48
2. The NTLM Wire Protocol & Domain Passthrough 51
3. Cryptographic Deep-Dive: NTLM Mathematics 55
4. Demystifying NTLM Relay, MIC, and Channel Binding 57
5. Architectural Friction Points & Vulnerability Foundations 59
6. Defensive Telemetry, Detection Engineering & Hardening 60
7. Authoritative Specifications, RFCs & Cross-Vault Links 62

LDAP Protocol — Directory Access Backbone & Relay Attack Surface 64
1. Architectural Foundations & Purpose 64
2. Protocol Internals & The Wire Exchange 66
3. The Four LDAP Bind Models Dissected 67
4. Search Filter Syntax, Operators & Matching Rules 69
5. Directory Limits, Pagination & Extended Controls 70
6. Demystifying LDAP Signing & Channel Binding Tokens (CBT) 71
7. Architectural Vulnerability Primitives (Zero Tool Dumping) 72
8. Defensive Telemetry, Detection Engineering & Hardening 73
9. Authoritative Specifications, RFCs & Cross-Vault Links 74

Security Descriptors & ACLs — Access Control Architecture in Active Directory 76
1. Architectural Foundations & Purpose 76
2. Walkthrough: The SRM Access Check Algorithm 78
3. Dissecting the Binary `nTSecurityDescriptor` 80
4. Deconstructing SDDL (Security Descriptor Definition Language) 82

5. Demystifying Privilege: Active Directory as a Directed Graph 84
6. The Defense of Privileged Objects: AdminSDHolder & SDProp 85
7. Defensive Telemetry, Detection Engineering & Hardening 87
8. Authoritative Specifications, RFCs & Cross-Vault Links 88

Group Policy Objects (GPO) — The Central Policy Engine in AD 89
1. Architectural Foundations & Purpose 89
2. Scenario Walkthrough: Deploying Enterprise Hardening 91
3. The Split Architecture & Version Synchronization 92
4. The Processing Hierarchy: The LSDOU Precedence Engine 94
5. Client-Side Execution: The Pull-Based Architecture 95
6. Architectural Vulnerability Primitives (Zero Tool Dumping) 97
7. Defensive Telemetry, Detection Engineering & Hardening 98
8. Authoritative Specifications, RFCs & Cross-Vault Links 100

Privileged Groups & Access Tokens — Runtime Kernel Authorization 101
1. Architectural Foundations: Static Objects vs. Runtime Tokens 101
2. Walkthrough: From Logon Proof to Process Execution 103
3. Token Anatomy: Primary vs. Impersonation Tokens 104
4. Mandatory Integrity Control (MIC) & UAC Linked Tokens 105
5. Dangerous Privileges: The Win32 Kernel Superpowers 106
6. Built-in Groups & Scopes in Active Directory 108
7. Defensive Telemetry, Detection Engineering & Hardening 109
8. Authoritative Specifications, RFCs & Cross-Vault Links 111

AD-Integrated DNS Architecture — The Identity Resolution Backbone 112
1. Architectural Foundations: Why Active Directory Cannot Function Without DNS 112
2. Walkthrough: The DC Locator Process in Action 114
3. The Physical Reality: How DNS Records Exist in LDAP 116
4. Dynamic Updates: Non-Secure vs. Secure (GSS-TSIG) 117
5. Zone Scavenging and the Tombstoning Lifecycle 118
6. Architectural Vulnerability Primitives (Zero Tool Dumping) 119
7. Defensive Telemetry, Detection Engineering & Hardening 120
8. Authoritative Specifications, RFCs & Cross-Vault Links 122

Certificate Services Architecture (AD CS) — Foundations 123
1. Architectural Foundations: Why Kerberos Needed an Asymmetric Companion 123
2. Walkthrough: Workstation Certificate Enrollment 125
3. The Active Directory PKI Containers in the Configuration Partition 126
4. Certificate Templates: Structure, EKUs & SAN Flags 127
5. Enrollment Protocols Demystified 128
6. Architectural Implications: The Genesis of AD CS Vulnerabilities 129
7. Defensive Telemetry, Detection Engineering & Hardening 130
8. Authoritative Specifications, RFCs & Cross-Vault Links 132

Trust Architecture — Relationships Between Domains 133
1. Architectural Foundations: Identity Federation Without Database Replication 133
2. The Cryptographic Anchor: The Trust Key & The TDO 136
3. Walkthrough: Cross-Domain Kerberos Referral Flow 137

4. Boundary Enforcement: SID Filtering vs. Intra-Forest Trust Mechanics 139
5. Defensive Telemetry, Detection Engineering & Hardening 141
6. Authoritative Specifications, RFCs & Cross-Vault Links 142

About the Author 143

Active Directory Architecture — Architectural Structure

> **Architectural Mission:** Master the foundational engineering of Active Directory Domain Services (AD DS). Understand the historical leap from single-master flat SAM directories to distributed hierarchical LDAP multi-master architectures, deconstruct the logical versus physical boundaries of forests and domains, analyze the binary database structures of the Extensible Storage Engine (`ntds.dit`), trace the three-layer credential encryption hierarchy, and examine the deterministic mathematics of multi-master replication and conflict resolution.

1. The Genesis & Architectural Purpose of AD DS

1.1 The Problem: Why NT4 Domains Collapsed Under Enterprise Scale

Prior to the launch of Windows 2000, Microsoft enterprise networks relied on the **Windows NT 4.0 Domain Model**. That architecture was designed around the local-area network (LAN) realities of the early 1990s. As businesses expanded globally, the NT4 architecture suffered from three catastrophic engineering bottlenecks:

1. **The Single-Master Write Bottleneck (The PDC Availability Ceiling):** In an NT4 domain, exactly one server operated as the **Primary Domain Controller (PDC)**. All other servers operated as **Backup Domain Controllers (BDCs)** holding read-only replicas of the directory.
   - *The Failure:* **Every directory write operation had to occur on the PDC.** If an employee attempted to change their password, if an administrator created an account, or if a user was locked out, the request had to be forwarded directly to the single PDC. If the PDC suffered a network partition, hardware failure, or reboot, the entire enterprise domain became read-only. Identity modification ground to an absolute halt.
2. **Flat Namespace with Severe SAM Size Limits:** NT4 stored identities inside the registry-backed **Security Account Manager (SAM)** database. The SAM was completely flat: it lacked containers, departments, and hierarchical nesting. Furthermore, because the SAM hive had to reside entirely within non-paged kernel memory, the database had a hard ceiling of approximately 40,000 total objects (users, groups, and machine accounts

combined). Large enterprises were forced to deploy dozens of disjointed, unmanageable domains simply to bypass kernel memory limits.

3. **Non-Transitive, Isolated Trust Topologies:** Trust relationships between NT4 domains were manual, one-way, and non-transitive. If Domain A trusted Domain B, and Domain B trusted Domain C, Domain A did *not* trust Domain C. Connecting N domains in an enterprise required managing $N \times (N-1)$ manual trust links—an administrative impossibility that collapsed under its own complexity.

1.2 The Active Directory Revolution: Hierarchical Multi-Master Architecture

To permanently eliminate these bottlenecks, Microsoft engineered **Active Directory Domain Services (AD DS)** around three core architectural pillars:

- **Hierarchical Directory Tree (X.500 / LDAP):** AD DS replaces flat registries with a hierarchical Directory Information Tree (DIT) accessed via standard Lightweight Directory Access Protocol (LDAP) queries. Resources are organized into trees of Organizational Units (OUs), allowing granular delegation of administrative authority without multiplying domains.
- **Multi-Master Replication with Deterministic Conflict Resolution:** Every writable Domain Controller (DC) holds an authoritative, writable copy of the domain database. Any client can change their password on *any* accessible DC in their geographic site. Changes propagate asynchronously to peer DCs across the network using the Directory Replication Service Remote Protocol (DRSUAPI / MS-DRSR).
- **The Forest as a Unified Security Space:** AD DS introduced the **Forest** as the root container of administrative trust. All domains within a forest share a single, forest-wide Schema (the blueprint of allowed object classes and attributes) and a common Configuration partition, linked together by automatic, two-way transitive Kerberos trusts.

1.3 The Fundamental Axiom: Forest vs. Domain Boundaries

One of the most consequential security misconceptions in Windows engineering is confusing an *administrative boundary* with a *security boundary*:

- **The Forest is the TRUE Security Boundary:** The forest defines the ultimate boundary of security isolation in Active Directory. All domains within a forest share a common Schema, a common Configuration partition, and an implicit mesh of two-way transitive Kerberos trust paths.

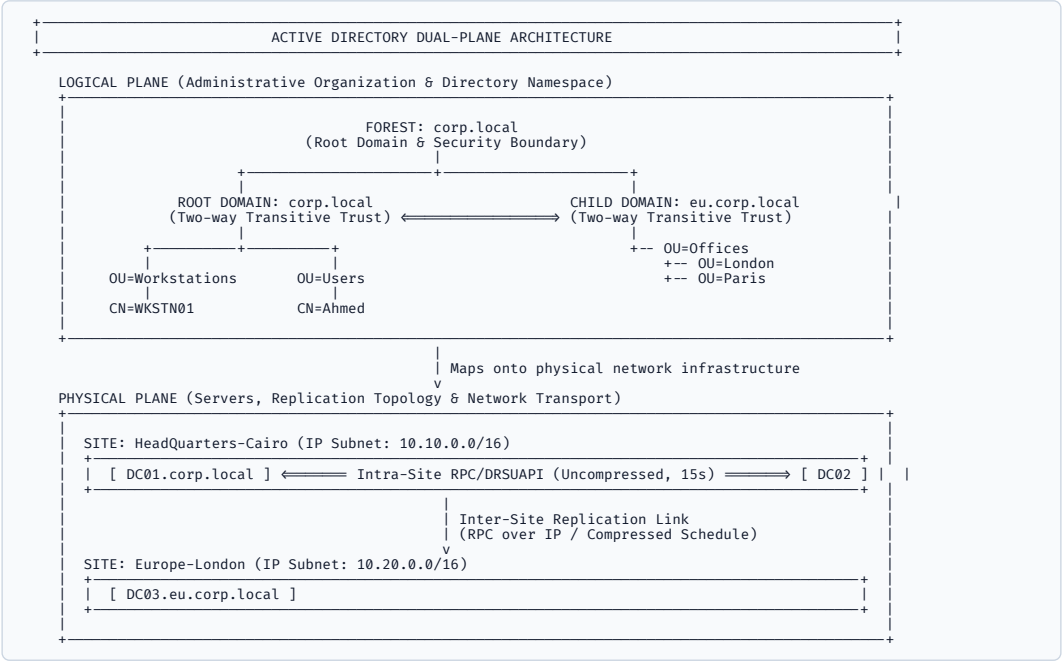
> **Architectural Law:** A compromise of *any* domain within an Active Directory forest can be leveraged to achieve complete, forest-wide compromise of all other domains. Domain boundaries do not prevent cross-domain compromise within the same forest.

- **The Domain is Merely an Administrative and Replication Boundary:** A domain defines the replication scope of the Domain Naming Context (`ntds.dit`), the scope of domain-local security groups, and default administrative policy (such as password complexity). It serves to partition replication traffic across WAN links and segment administrative day-to-day operations—it does **not** provide adversarial isolation against a compromised peer domain.

2. Logical Plane vs. Physical Plane

Active Directory strictly decouples how an organization is **logically managed** from how its servers are **physically networked**.

2.1 Diagram 1: Logical Plane vs. Physical Plane



2.2 The Logical Architecture

1. **Forest:** The top-level container representing the entire enterprise directory instance. It is identified by the name of the very first domain installed within it (the Forest Root Domain, e.g., `corp.local`).
2. **Domain Tree:** A hierarchy of one or more domains sharing a contiguous DNS namespace (e.g., `eu.corp.local` is a child domain of `corp.local`).
3. **Domain:** A logical partition containing security accounts (users, computers, groups) that share a common security database, domain SID, and directory policies.
4. **Organizational Unit (OU):** A container object used to organize resources *within* a domain. OUs are the fundamental building block for delegating

administrative permissions and linking Group Policy Objects (GPOs). OUs do not possess SIDs; they are purely organizational.

2.3 The Physical Architecture

1. **Domain Controller (DC):** A physical or virtual server running Windows Server that hosts Active Directory Domain Services, processes authentications (Kerberos/NTLM), and replicates directory changes.
2. **Global Catalog (GC):** A designated Domain Controller that maintains a complete replica of its own domain's objects, plus a **Partial Attribute Set (PAS)** of every object in every other domain across the entire forest. The Global Catalog enables forest-wide user searches and resolves universal group memberships during Kerberos ticket issuance without querying remote child domains.
3. **Sites:** One or more TCP/IP subnets connected by high-speed, reliable LAN links (e.g., an office building or datacenter). Sites control authentication routing: a workstation at `10.10.1.50` uses DNS Site SRV records to automatically discover and authenticate against a DC located in its local site rather than traversing a slow WAN link.
4. **Site Links & The KCC:** Physical connections between sites. The **Knowledge Consistency Checker (KCC)** is a built-in process running on every DC that automatically computes the most efficient replication topology (creating connection objects) to synchronize data across sites while minimizing network overhead.

2.4 The Four Directory Partitions (Naming Contexts)

Every Domain Controller's database is logically divided into distinct **Naming Contexts (NCs)**:

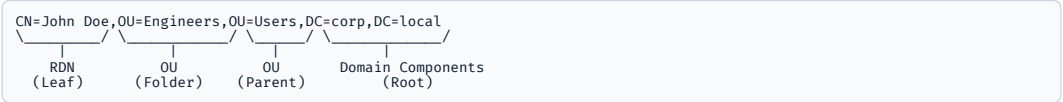
Naming Context	Replication Scope	Purpose & Contents
Schema NC	Forest-wide	Defines the fundamental rules of the directory: every allowed Object Class (e.g., `user`, `computer`) and Attribute (e.g., `sAMAccountName`, `userPassword`). Identical across every DC in the entire forest.
Configuration NC	Forest-wide	Stores physical topology and service settings: Sites, Subnets, Site Links, Directory Replication connectors, and enterprise services (such as AD CS Certificate Templates).
Domain NC	Domain-wide	The actual payload database for a specific domain: all Users, Computers, Groups, OUs, and GPOs. Replicated exclusively among DCs within that specific domain.
Application NC	Variable / Custom	Used by directory-integrated applications. Most notably includes **ForestDNSZones** (forest-wide DNS records) and **DomainDNSZones** (domain-

Naming Context	Replication Scope	Purpose & Contents
		specific DNS records) for Active Directory-Integrated DNS (ADIDNS).

3. Syntax Breakdown & Concrete Object Anatomy

3.1 Deconstructing LDAP Distinguished Names (DN)

Every object in Active Directory is located via its **Distinguished Name (DN)**—a standardized X.500 string detailing the path from the leaf object up to the root of the directory tree.



- **Reading Direction:** Evaluated from **left to right** to traverse from the leaf object down to the root, but represents the tree hierarchy from **right to left**:
  - `DC=corp,DC=local`: The Domain Components establishing the domain partition root `corp.local`.
  - `OU=Users`: The primary top-level Organizational Unit directly beneath the domain root.
  - `OU=Engineers`: A child Organizational Unit nested within `OU=Users`.
  - `CN=John Doe`: The **Relative Distinguished Name (RDN)** representing the specific leaf user object.
- **Prefix Designators:**
  - `CN` (Common Name): Used for users, computers, groups, and standard schema containers.
  - `OU` (Organizational Unit): Used exclusively for directory organizing containers.
  - `DC` (Domain Component): Used for DNS domain parts (`corp`, `local`).

3.2 Concrete Object Walkthrough: Human User (`jdoe`)

Consider a typical human user account in Active Directory:

- **Distinguished Name:** `CN=John Doe,OU=Engineers,OU=Users,DC=corp,DC=local`
- **`sAMAccountName`:** `jdoe` (The legacy pre-Windows 2000 logon name, maximum 20 characters).
- **`userPrincipalName` (UPN):** `jdoe@corp.local` (Modern internet-style identity format).

The Mathematical Anatomy of a SID:

Every security principal possesses a unique **Security Identifier (SID)**:

$$\text{SID} = \mathbf{S\text{-}1\text{-}5\text{-}21\text{-}3941234567\text{-}1234567890\text{-}987654321\text{-}1105}$$

```
S - 1 - 5 - 21 - 3941234567 - 1234567890 - 987654321 - 1105
|   |   |   |    \_____________________________________/    \__/
|   |   |   |                      |                         |
|   |   |   |          96-bit Domain Identifier              |
|   |   |   |         (Identical for all domain accounts)    |
|   |   |   |                                                |
|   |   |   +-- Sub-Authority Prefix (21 = Non-unique domain)
|   |   |
|   |   +-- Identifier Authority (5 = SECURITY_NT_AUTHORITY)
|   |
|   +-- Revision Level (Always 1)
|
+-- String Format Prefix
                                                          |
                                                          v
                                                Relative Identifier (RID)
                                                Unique integer assigned by DC
                                                (1105 = John Doe)
```

- `S-1-5-21`: Standard Windows NT domain account authority.
- `3941234567-1234567890-987654321`: The unique 96-bit domain security ID generated when the domain was created. Every user, computer, and group in `corp.local` shares this exact prefix.
- `1105` (The RID): The **Relative Identifier**. RIDs below `1000` are reserved for built-in accounts (e.g., `500` for Administrator, `501` for Guest, `502` for krbtgt, `512` for Domain Admins). Custom accounts are assigned RIDs sequentially starting at `1000+`.

Calculating `userAccountControl` (UAC) Bitmasks:

The `userAccountControl` attribute is a 32-bit integer that governs the operational state of the account:

```
Bit Flag Constants:
- 0×00000001 (1)        : SCRIPT
- 0×00000002 (2)        : ACCOUNTDISABLE
- 0×00000200 (512)      : NORMAL_ACCOUNT
- 0×00010000 (65536)    : DONT_EXPIRE_PASSWORD
- 0×00400000 (4194304)  : DONT_REQ_PREAUTH
```

- **Standard Enabled Account (`512`):**

$$\text{Value} = 512 \ (0\text{x}00000200) \Rightarrow \text{NORMAL_ACCOUNT}$$

- **Disabled Account (`514`):**

$$\text{Value} = 512 + 2 = 514 \ (0\text{x}00000202) \Rightarrow \text{NORMAL_ACCOUNT} \mid \text{ACCOUNTDISABLE}$$

- **Service Account with Non-Expiring Password (`66048`):**

$$\text{Value} = 512 + 65536 = 66048 \ (0\text{x}00010200) \Rightarrow \text{NORMAL_ACCOUNT} \mid \text{DONT_EXPIRE_PASSWORD}$$

3.3 Concrete Object Walkthrough: Computer Account (`SRV01$`)

A machine joined to Active Directory is simply another security principal with its own password and Kerberos capabilities:

- **Distinguished Name:** `CN=SRV01,OU=Servers,DC=corp,DC=local`
- **`sAMAccountName`:** `SRV01$`
- **The Trailing `$` Convention:** Historically carried over from Windows NT to prevent naming collisions in flat namespaces. The `$` signifies to operating system APIs that this principal is a **Machine Trust Account** rather than a human user.

- **`userAccountControl: 4096` (`0×00001000`):** Bit flag `0×1000` corresponds to `WORKSTATION_TRUST_ACCOUNT`. Computer accounts automatically rotate their own 120-character machine passwords with the Domain Controller every 30 days.

4. Database Internals: The ESE Jet Blue Engine & Encryption Hierarchy

4.1 The Database Architecture: `ntds.dit`

Active Directory does not run on Microsoft SQL Server. It runs on the **Extensible Storage Engine (ESE)**, historically known as **Jet Blue**—a high-performance, transactional B-tree storage engine optimized for massive read throughput.

The entire directory database resides in a single file on every Domain Controller: `%SystemRoot%\NTDS\ntds.dit`

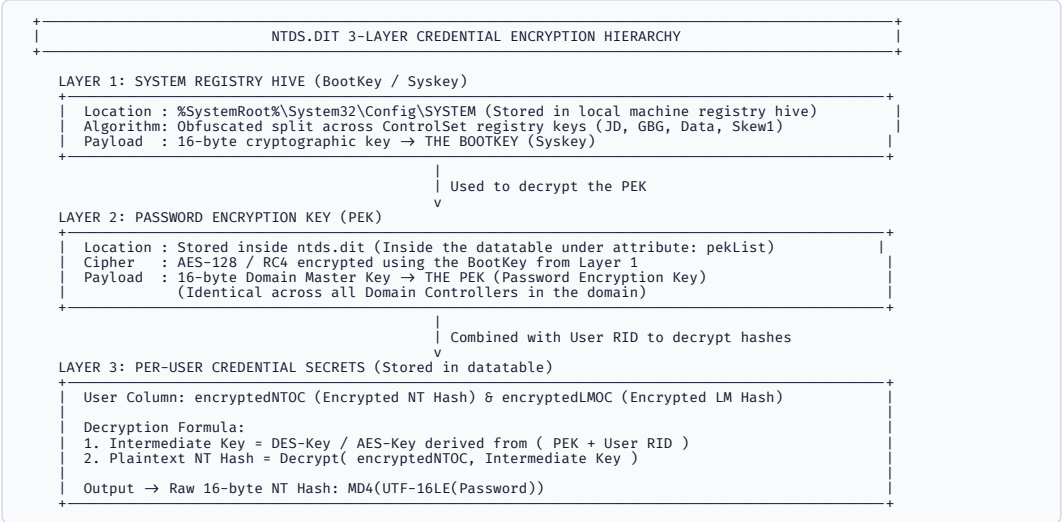
Within `ntds.dit`, data is organized into three primary tables:

1. **`datatable`:** The heart of the directory. Contains every single object, container, and attribute record. Each row represents an object; columns represent attributes (e.g., `sAMAccountName`, `objectSid`, `userPassword`).
2. **`link_table`:** Stores multi-valued relational links and backlinks (e.g., group memberships). Links the `member` attribute of a group to the `memberOf` attribute of a user without duplicating object data.
3. **`secret_table`:** Stores internal Local Security Authority (LSA) secret parameters, domain trust passwords, and cryptographic state variables.

4.2 The 3-Layer Credential Encryption Hierarchy

Password hashes are never stored in plaintext within `ntds.dit`. Active Directory implements a strict three-layer cryptographic envelope to protect credentials stored on disk:

4.3 Diagram 2: NTDS.dit 3-Layer Encryption Architecture



Why Stealing `ntds.dit` Alone Yields Zero Passwords:
If an adversary obtains an offline copy of `ntds.dit` (e.g., via volume shadow copy), the credential tables are mathematically unreadable. The per-user hashes are encrypted with the **Password Encryption Key (PEK)**. The PEK itself is stored inside `ntds.dit` in an encrypted state, sealed with the **BootKey (Syskey)**. The BootKey resides exclusively within the **`SYSTEM` registry hive** of that specific Domain Controller.

To unseal and extract domain credentials, an adversary must possess **both** files simultaneously:

$$\text{ntds.dit} + \text{SYSTEM Hive} \Rightarrow \text{BootKey} \xrightarrow{\text{Decrypts}} \text{PEK} \xrightarrow{\text{Decrypts}} \text{NT Hashes}$$

5. Multi-Master Replication & Conflict Resolution

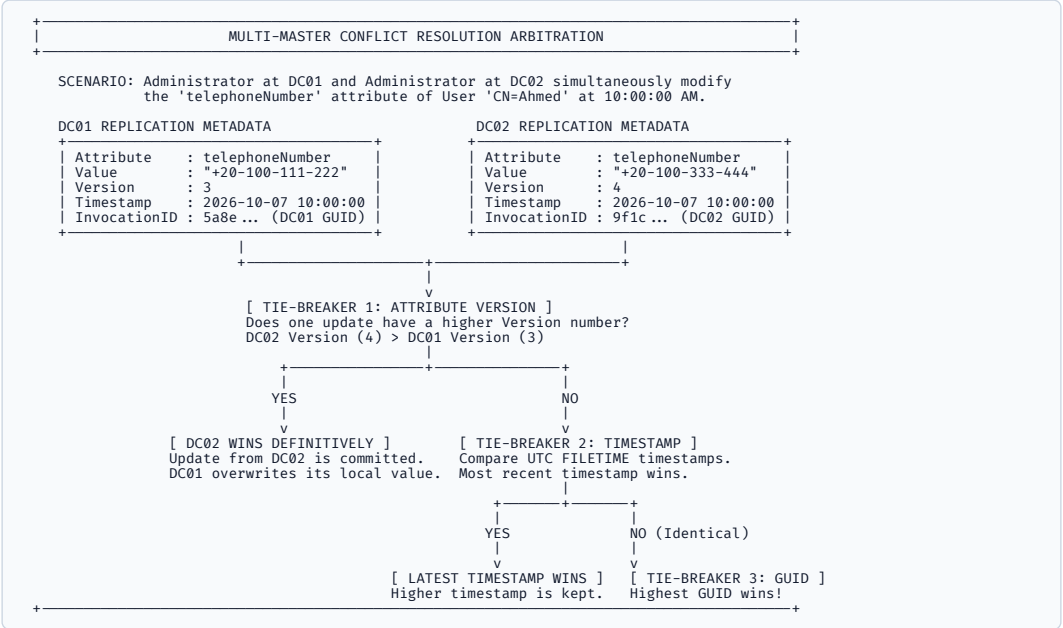
5.1 The Replication Engine: DRSUAPI

Active Directory domains maintain multi-master consistency via the **Directory Replication Service Remote Protocol (DRSUAPI / MS-DRSR)** executing over RPC (TCP port 135 and dynamic RPC ephemeral ports).

Replication is governed by three mathematical coordinates:

1. **USN (Update Sequence Number):** A 64-bit integer maintained locally by every Domain Controller. Every time a DC modifies any attribute, it increments its local USN by 1 and stamps the change with that new USN. **USNs are strictly local to each DC** DC01's USN 5000 has zero relation to DC02's USN 5000.
2. **InvocationID:** A unique 128-bit GUID identifying the specific database engine instance running on a DC. If a DC is restored from backup, it generates a new InvocationID to prevent sequence corruption.
3. **High Watermark Vector & Up-To-Dateness (UTD) Vector:**
   - The *High Watermark Vector* tracks the highest USN that a DC has received from a specific replication partner.
   - The *Up-To-Dateness Vector* tracks the highest originating write USN that a DC has received from *every* DC across the entire domain, preventing replication loops.

5.2 Diagram 3: Multi-Master Replication Conflict Resolution



5.3 Conflict Resolution Rules: How AD Prevents Database Divergence

When two Domain Controllers modify the same object attribute concurrently, Active Directory applies a deterministic three-tier tie-breaking algorithm:

1. **Version Number:** Every time an attribute is modified at its origin, its attribute version increments by 1. The write with the **higher version number always wins**.
2. **Timestamp:** If both versions are identical, the DC compares the originating UTC timestamps (`FILETIME`). The update with the **most recent timestamp wins**.
3. **InvocationID:** If both the version and the timestamp are identical to the microsecond, the DC executes a raw binary comparison of the originating DCs' `InvocationID` GUIDs. The update associated with the **higher binary GUID wins**.

Resolving Container Naming Conflicts (CNF Mangling):
If two administrators create an object with the identical name in the same OU on different DCs simultaneously, Active Directory cannot allow duplicate sibling RDNs.

- The object that loses the tie-breaker is automatically renamed (**mangled**):

$$\text{CN=Sarah} \Rightarrow \text{CN=Sarah}\backslash\mathbf{0ACNF}{:}\langle\text{ObjectGUID}\rangle$$

- This prevents replication crashes and preserves both objects for administrative review.

6. Flexible Single Master Operation (FSMO) Roles

While 99% of Active Directory operations are multi-master, certain operations cannot tolerate replication race conditions. To prevent logical inconsistencies, Microsoft designates five operations as **Flexible Single Master Operation (FSMO)** roles:

FSMO Role Name	Scope	Operational Function & Failure Impact
Schema Master	Forest-wide	Holds the exclusive right to modify the directory schema. Only one DC in the entire forest holds this role. If offline, schema extensions (e.g., Exchange setup) fail; normal authentications are unaffected.
Domain Naming Master	Forest-wide	Authorizes the addition or removal of domains and application partitions across the forest. If offline, new domains cannot be provisioned.
PDC Emulator	Domain-wide	**The most critical role in the domain.** • Acts as the primary time server (NTP) for the domain. • Receives immediate replication of password changes. • Arbitrates account lockouts and bad password attempts. • Serves as the default target for Group Policy editing. • Executes the **SDProp** thread every 60 minutes.
RID Master	Domain-wide	Allocates pools of 500 Relative Identifiers (RIDs) to each DC. When a DC creates users or computers, it draws from its local pool. If the RID Master is offline and a DC exhausts its pool, that DC cannot create new objects.
Infrastructure Master	Domain-wide	Translates cross-domain object references (Phantom Objects) when users in one domain are added to groups in another. *(Note: Must not be placed on a Global Catalog server in multi-domain forests unless all DCs are GCs).*

7. Defensive Telemetry, Detection Engineering & Hardening

7.1 Security Event Telemetry

Monitoring architectural operations within Active Directory relies on auditing core Directory Service events:

Event ID	Log Name	Operational Meaning	Strategic Significance
5136	Security	A directory service object was modified.	**Primary Telemetry:** Captures changes to object attributes, including the modified attribute name, the old/new values, and the caller identity.
5137	Security	A directory service object was created.	Tracks newly provisioned users, computers, or OUs.
4932	Security	An Active Directory replication partner was synchronized.	Monitors routine and scheduled DRSUAPI replication cycles.
4933	Security	Active Directory replication sync ended.	Telemetry for replication health.
4662	Security	An operation was performed on an object.	**DCSync Detection:** Generated when replication rights (`DS-Replication-Get-Changes-All`) are exercised against the domain root object by a non-DC account.

7.2 Detection Heuristics & Hunting Logic

1. Unauthorized Directory Replication (DCSync) Telemetry (KQL):
Flags replication operations requested by endpoints or accounts that are **not** legitimate Domain Controllers:

```
SecurityEvent
| where EventID == 4662
| where ObjectServer == "DS"
| where ObjectType =~ "{19195a5b-6da0-11d0-afd3-00c04fd930c9}" // Domain-DNS object class
| where AccessMask has "0×100" // Control Access
| where Properties has "1131f6aa-9c07-11d1-f79f-00c04fc2dcd2" // DS-Replication-Get-Changes
    or Properties has "1131f6ad-9c07-11d1-f79f-00c04fc2dcd2" // DS-Replication-Get-Changes-All
| where SubjectUserName !endswith "$" // Exclude legitimate machine accounts of DCs
| project TimeGenerated, SubjectUserName, SubjectDomainName, Computer, Properties
```

7.3 Enterprise Hardening Blueprint

Protecting the core architecture of Active Directory requires enforcing Tier-0 isolation principles:

1. **Enforce the Tier-0 Boundary Model:**
   - Treat Domain Controllers, the Forest Root Domain, and all domain identity stores as **Tier-0 Assets**.
   - Prohibit Tier-0 administrators from logging into lower-tier systems (Tier-1 servers, Tier-2 workstations) to prevent credential theft.
2. **Protect the Database and Registry Backups:**
   - Ensure that system state backups containing `ntds.dit` and the `SYSTEM` hive are encrypted with enterprise-grade key management and restricted to authorized backup operators.
3. **Isolate FSMO Role Holders:**

    - Place the PDC Emulator on high-availability, heavily monitored infrastructure with strict network access controls.
4. **Deploy Virtualization Safeguards:**
    - When running virtualized Domain Controllers, ensure that administrators do not use VM-level snapshots. Restoring a DC via snapshot rolls back its local database without updating peers, triggering **USN Rollback** and permanent replication failure.

8. Authoritative Specifications, RFCs & Cross-Vault Links

8.1 Authoritative Specifications

- **[MS-ADTS]:** *Active Directory Technical Specification* — The comprehensive technical specification for Active Directory architecture, naming contexts, schema, and directory operations.
- **[MS-DRSR]:** *Directory Replication Service (DRS) Remote Protocol* — Formal documentation of the replication protocol, USN synchronization, and UTD vectors.
- **[MS-SAMR]:** *Security Account Manager (SAM) Remote Protocol* — Documents account management APIs and the structure of SIDs and RIDs.
- **RFC 4511 / RFC 4512:** *Lightweight Directory Access Protocol (LDAP: The Protocol & Directory Information Models)*.

Kerberos Authentication — The Primary Authentication Protocol

> **Architectural Mission:** Master the internal mechanics of Kerberos Version 5 (RFC 4120) and Microsoft's enterprise extensions (MS-KILE, MS-PAC, MS-SFU). Understand how symmetric cryptography, ephemeral session keys, and privilege containers interact so that every downstream attack—from offline ticket cracking to cross-forest delegation abuse—becomes an obvious, mathematically inevitable consequence of protocol design.

1. Architectural Foundations & The Core Dilemma

1.1 The Fundamental Dilemma: Why Direct Authentication Fails

Consider a typical morning in an enterprise environment running Active Directory (AD): An employee named **Ahmed** sits down at his assigned workstation `WKSTN01.corp.local` and logs in with his corporate credentials. Thirty minutes later, Ahmed needs to open an engineering specification spreadsheet stored on a network file server at `\\FS01.corp.local\engineering\specifications.xlsx`.

To access that file, the file server `FS01` must answer two fundamental questions:

1. **Authentication:** Who is attempting to connect? (Is this genuinely Ahmed?)
2. **Authorization:** What is this caller permitted to do? (Does Ahmed have read access to the engineering folder?)

In a naive network architecture, Ahmed's workstation would simply transmit his username and password (or password hash) directly over the wire to `FS01`. In an enterprise with thousands of servers and tens of thousands of users, this direct credential transmission model immediately collapses under three fatal architectural vulnerabilities:

1. **Credential Exposure to Untrusted Service Endpoints (The Rogue Server Problem):** If `WKSTN01` transmits Ahmed's password or reusable cryptographic hash directly to `FS01`, the integrity of Ahmed's credentials is only as secure as `FS01`'s least trustworthy administrator. If `FS01` is compromised by an adversary, or if the service administrator is untrustworthy, the rogue server can harvest Ahmed's plaintext password or hash and reuse it across every other system in the enterprise—including Domain Controllers (DCs) and human resources systems.

2. **Plaintext Sniffing and Session Hijacking:** Even if encrypted at the transport layer, transmitting reusable credentials across arbitrary network segments exposes raw authenticators to wire-sniffing, man-in-the-middle interception, and replay attacks.
3. **Secret Proliferation and $N \times M$ Cryptographic Overhead:** If every workstation needed to authenticate directly with every service server without sending credentials over the network, every user would need to share a unique pre-shared secret with every single application server in the forest. In a domain of 10,000 users and 500 servers, this would require managing $5,000,000$ distinct shared cryptographic keys across workstations and endpoints—an administrative impossibility.

1.2 The Kerberos Solution: Cryptographic Envelopes and the Trusted Third Party

To solve this dilemma, Active Directory relies on **Kerberos Version 5**, an open standard protocol originally engineered at MIT (Project Athena) under **RFC 4120**, heavily enhanced by Microsoft through proprietary extensions (**MS-KILE**, **MS-PAC**, **MS-SFU**).

The design philosophy of Kerberos is built upon two immutable pillars:

1. **The Shared Secret Never Leaves the Owner:** Neither the user's password nor their long-term cryptographic key is ever transmitted across the network wire. Long-term keys are used solely as local encryption and decryption keys to seal and unseal temporary cryptographic envelopes.
2. **The Trusted Third Party (TTP):** All entities in the domain—human users, workstation computer accounts, service accounts, and domain controllers—mutually trust a single centralized authority: the **Key Distribution Center (KDC)**. The KDC holds a master database of long-term secrets for every principal in the domain.

When Ahmed wants to access `FS01`, he does not present his password to `FS01`. Instead:

- Ahmed proves his identity once to the KDC.
- The KDC hands Ahmed an encrypted, time-stamped credential called a **Ticket Granting Ticket (TGT)**.
- When Ahmed wants to talk to `FS01`, he presents his TGT back to the KDC and asks for an encrypted **Service Ticket** specifically addressed to `FS01`.
- The KDC issues a Service Ticket encrypted with `FS01`'s private long-term key, which Ahmed cannot open or tamper with.
- Ahmed presents this Service Ticket to `FS01`. Because only `FS01` and the KDC know `FS01`'s long-term key, `FS01` can decrypt the ticket, verify the KDC's digital signature, read Ahmed's identity and group memberships, and grant access—all without `FS01` ever knowing Ahmed's password.

1.3 Terminology & Core Primitives (Zero-Acronym Onboarding)

Before tracing the wire packets, every component and acronym must be crystal clear:

- **Principal:** Any uniquely identifiable entity in the Kerberos realm that can request authentication or host a service. Principals take two forms:
  - *User Principals:* Represent human users or machine accounts (e.g., `ahmed@CORP.LOCAL` or `WKSTN01$@CORP.LOCAL`).
  - *Service Principals:* Represent network services running on hosts (e.g., `cifs/FS01.corp.local`).
- **Realm:** The administrative domain of authentication authority, represented in uppercase (e.g., `CORP.LOCAL`). In Active Directory, the Kerberos Realm maps directly to the Active Directory Domain Name.
- **KDC (Key Distribution Center):** The centralized authentication server running on every Active Directory Domain Controller (listening on UDP/TCP port 88). The KDC is logically split into two sub-services:
  - **AS (Authentication Service):** Issues initial Ticket Granting Tickets (TGTs) to verify principal identities.
  - **TGS (Ticket Granting Service):** Issues Service Tickets to authenticated clients presenting valid TGTs.
- **TGT (Ticket Granting Ticket):** An encrypted, time-bounded identity token issued by the Authentication Service (AS). It proves that the client has successfully authenticated to the domain. It is encrypted exclusively using the secret key of the domain's root Kerberos service account (`krbtgt`). The client cannot read or modify its TGT.
- **TGS (Ticket Granting Service) Ticket / Service Ticket:** An encrypted ticket issued by the Ticket Granting Service (TGS) allowing a client to authenticate to a specific application or service (such as SMB file sharing, MSSQL, or HTTP). It is encrypted with the long-term secret key of the target service account.
- **SPN (Service Principal Name):** A unique, forest-wide string registered on an Active Directory user or computer account that maps a network service to the specific Active Directory account whose cryptographic keys will be used to encrypt tickets for that service (e.g., `cifs/FS01.corp.local` or `MSSQLSvc/sql01.corp.local:1433`).
- **PAC (Privilege Attribute Certificate):** A proprietary Microsoft extension data structure appended to Kerberos tickets. It contains the user's complete Windows security context—Security Identifiers (SIDs), group memberships, user account control flags, and cryptographic signatures.
- **Session Key:** An ephemeral, short-lived symmetric cryptographic key generated on the fly by the KDC. A session key is shared exclusively between two

parties (either Client ↔ KDC, or Client ↔ Service Server) to encrypt and sign communication for a single session without exposing long-term keys.
- **Authenticator:** A small data structure generated locally by a client containing the client's principal name and a microsecond-accurate timestamp, encrypted with the relevant ephemeral Session Key. It proves that the client genuinely possesses the Session Key without transmitting the key itself.
- **Pre-Authentication (PA-DATA):** A mechanism where the client must prove knowledge of its secret key (typically by encrypting the current timestamp) *before* the KDC will issue an AS-REP response containing a TGT.
- **FAST (Flexible Authentication Secure Tunneling):** An RFC 6113 Kerberos extension (often termed Kerberos Armoring) that encapsulates initial Kerberos exchanges inside an armored, encrypted tunnel derived from the computer account's keys, protecting pre-authentication exchanges against offline cracking.

2. Scenario Walkthrough: From Workstation to File Share

2.1 The Scenario Setup

To anchor every packet, timestamp, and key transformation in reality, follow this exact enterprise scenario:

- **Client User:** `ahmed` (Domain user account `CORP\ahmed`, Long-Term Key: K_{ahmed}).
- **Client Machine:** `WKSTN01.corp.local` (Windows 11 Enterprise).
- **Domain Controller / KDC:** `DC01.corp.local` (Active Directory Domain Services on `CORP.LOCAL`, holding root account `krbtgt` with Long-Term Key: K_{krbtgt}).
- **Target File Server:** `FS01.corp.local` (Windows Server hosting SMB share `\\FS01.corp.local\engineering`, running as computer account `CORP\FS01$` with Long-Term Key: K_{FS01}).
- **Target SPN:** `cifs/FS01.corp.local`.

2.2 Diagram 1: The Enterprise Kerberos Triad & Packet Flow



2.3 Phase 1: The Authentication Service (AS) Exchange

When Ahmed boots `WKSTN01` and enters his domain credentials, the Local Security Authority Subsystem Service (LSASS) on `WKSTN01` initiates the **Authentication Service (AS) Exchange** to log Ahmed into the domain.

Step 1: AS-REQ (Client → KDC)

Ahmed's machine constructs a Kerberos Request (`AS-REQ`, Kerberos message type 10) and sends it over UDP (or TCP) port 88 to `DC01.corp.local`.

- **Plaintext Body (`KDC-REQ-BODY`):**
  - `cname` (Client Name): `ahmed`

    - `realm`: `CORP.LOCAL`
    - `sname` (Service Name): `krbtgt/CORP.LOCAL` (Ahmed is explicitly asking for a ticket to the Ticket Granting Service)
    - `till`: Requested ticket end time (typically current time + 10 hours)
    - `nonce`: A 32-bit cryptographically random integer generated by `WKSTN01` to detect replay attacks.
    - `etype`: The list of encryption types `WKSTN01` supports, ordered by preference (e.g., `AES256-CTS-HMAC-SHA1-96` [etype 18], `AES128-CTS-HMAC-SHA1-96` [etype 17], `RC4-HMAC` [etype 23]).
- **Pre-Authentication Data (`padata`):**
    - Type: `PA-ENC-TIMESTAMP` (padata type 2).
    - Data: The current system timestamp (accurate to microseconds), encrypted using Ahmed's long-term symmetric key (K_{ahmed}).
    - *How K_{ahmed} is derived:* When Ahmed typed his password, `WKSTN01` converted the password into an AES-256 key via PBKDF2 using the domain salt `CORP.LOCALahmed` (or directly into an NTLM hash for legacy RC4).

Step 2: KDC Processing & AS-REP (KDC → Client)
Upon receiving the packet, the KDC performs the following steps:

1. **Principal Lookup:** Queries the Active Directory database (`ntds.dit`) for `ahmed@CORP.LOCAL`. It retrieves Ahmed's stored password hash and encryption keys.
2. **Pre-Authentication Verification:** The KDC takes Ahmed's key (K_{ahmed}) and decrypts the `PA-ENC-TIMESTAMP`.
    - The KDC compares the decrypted timestamp against its own current domain controller clock.
    - If the clock skew between client and KDC is greater than 5 minutes (the default Kerberos tolerance window), the request is rejected with `KRB_AP_ERR_SKEW`.
    - If the timestamp decrypts cleanly and falls within the acceptable skew, the KDC is cryptographically certain that the caller knows Ahmed's password.
3. **Session Key Generation:** The KDC generates a cryptographically random, ephemeral 256-bit symmetric key: the **Logon Session Key** ($K_{\text{session_AS}}$).
4. **PAC Assembly:** The KDC looks up Ahmed's user SID (`S-1-5-21-` ... `-1105`), his primary group RID (`513` - Domain Users), and every global and universal security group Ahmed belongs to. It builds the Privilege Attribute Certificate (PAC) structure and signs it.
5. **Envelope Sealing:** The KDC crafts two encrypted envelopes:
    - **The Ticket Granting Ticket (TGT):** Encrypted using the domain's root `krbtgt` account key (K_{krbtgt}). Inside this ticket lies Ahmed's identity, validity timestamps, the PAC, and a copy of $K_{\text{session_AS}}$.

- **The Encrypted Response Part (`enc-part`):** Encrypted using Ahmed's key (K_{ahmed}). Inside this package lies the exact same copy of $K_{\text{session_AS}}$, the ticket expiration timestamps, and the client nonce matching the AS-REQ.

6. **Transmission:** The KDC packages both envelopes into an `AS-REP` packet (message type 11) and transmits it back to `WKSTN01`.

Client Ingestion:
`WKSTN01` receives the `AS-REP`. Because it possesses K_{ahmed}, it unseals the `enc-part`. It verifies that the returned nonce matches what it sent, extracts the **Logon Session Key** ($K_{\text{session_AS}}$), and caches both the unsealed $K_{\text{session_AS}}$ and the opaque, still-encrypted TGT safely inside the memory of `lsass.exe`. Ahmed is now fully logged in.

2.4 Phase 2: The Ticket Granting Service (TGS) Exchange

Now, Ahmed clicks on `\\FS01.corp.local\engineering`. The Windows operating system detects that SMB needs to connect to the file share. The workstation checks its local Kerberos ticket cache in LSASS:

- *Do we have a Service Ticket for* `cifs/FS01.corp.local`*?* **No.**
- *Do we have a valid TGT?* **Yes.**

The workstation now initiates the **Ticket Granting Service (TGS) Exchange**.

Step 3: TGS-REQ (Client → KDC)

`WKSTN01` constructs a `TGS-REQ` packet (message type 12) and dispatches it to `DC01` on port 88.

- **Request Body (`KDC-REQ-BODY`):**
  - `sname`: `cifs/FS01.corp.local` (The Service Principal Name representing CIFS/SMB on `FS01`).
  - `realm`: `CORP.LOCAL`
  - `nonce`: A new random 32-bit integer.
- **Pre-Authentication (`padata`):**
  - Type: `PA-TGS-REQ` (padata type 1).
  - Payload 1: The **TGT** received during Phase 1 (opaque ciphertext encrypted with K_{krbtgt}).
  - Payload 2: An **Authenticator** generated by `WKSTN01`. The Authenticator contains Ahmed's client name and the current timestamp, encrypted using the **Logon Session Key** ($K_{\text{session_AS}}$).

Step 4: KDC Processing & TGS-REP (KDC → Client)

The KDC receives the `TGS-REQ` and executes the following verification:

1. **TGT Decryption:** The KDC decrypts the TGT using its own long-term K_{krbtgt} key. If decryption succeeds, the KDC knows this TGT was minted by an authorized Domain Controller and has not been modified.
2. **Authenticator Validation:** From inside the decrypted TGT, the KDC extracts the Logon Session Key ($K_{\text{session_AS}}$). It uses $K_{\text{session_AS}}$ to decrypt Ahmed's Authenticator.

- It checks that the timestamp inside the Authenticator matches the current time within the 5-minute skew window.
- It verifies that the client name inside the Authenticator matches the client name inside the TGT.
- This proves that whoever sent this packet is the legitimate holder of the TGT and is not replaying an intercepted ticket.

3. **SPN Resolution:** The KDC looks up the requested SPN `cifs/FS01.corp.local` in Active Directory. It identifies that this SPN belongs to the computer account `FS01$` and retrieves `FS01$`'s long-term computer account key (K_{FS01}).
4. **Service Session Key Minting:** The KDC generates a brand-new ephemeral symmetric key: the **Service Session Key** ($K_{\text{session_FS}}$).
5. **PAC Copying & Resigning:** The KDC extracts Ahmed's PAC from the TGT, verifies the KDC signature, checks if any resource-group filtering is required, and re-signs the PAC's Server Signature using K_{FS01}.
6. **Ticket Sealing:** The KDC crafts two new cryptographic envelopes:
   - **The Service Ticket (TGS Ticket):** Encrypted using the target service's long-term key (K_{FS01}). It contains Ahmed's identity, the PAC, validity timestamps, and a copy of the Service Session Key ($K_{\text{session_FS}}$).
   - **The Response Wrapper (`enc-part`):** Encrypted using the client's Logon Session Key ($K_{\text{session_AS}}$). It contains the other copy of the Service Session Key ($K_{\text{session_FS}}$) and the request nonce.
7. **Transmission:** The KDC packages these into a `TGS-REP` packet (message type 13) and sends it to `WKSTN01`.

Client Ingestion:
`WKSTN01` unseals the `enc-part` using its cached $K_{\text{session_AS}}$. It extracts the **Service Session Key** ($K_{\text{session_FS}}$), associates it with the SPN `cifs/FS01.corp.local`, and stores both the Service Ticket (still encrypted with K_{FS01}) and $K_{\text{session_FS}}$ in LSASS.

2.5 Phase 3: The Application (AP) Exchange

`WKSTN01` now possesses everything required to talk to `FS01.corp.local`. Notice that up to this exact moment, **`FS01` has not received a single packet and has no idea Ahmed exists.**

Step 5: AP-REQ (Client → Application Server)
`WKSTN01` opens a TCP connection to `FS01.corp.local` on port 445 (SMB) and initiates the SMB Tree Connect negotiation by attaching a Kerberos `AP-REQ` packet (message type 14).

- **Payload of AP-REQ:**
  1. **The Service Ticket:** The opaque blob received from the TGS-REP, encrypted with K_{FS01}.
  2. **The Authenticator:** A brand-new data structure created by `WKSTN01` containing:
     - Client Name: `ahmed`
     - Timestamp: Current workstation time.

- Subkey (Optional): An optional client-proposed session subkey.
- Sequence Number: For packet sequencing.
- *Encrypted with:* The **Service Session Key** ($K_{\text{session_FS}}$).

3. **AP Options:** Flags specifying whether the client requests mutual authentication (`mutual-required`).

Step 6: Application Server Processing & AP-REP (Server → Client)
`FS01` receives the `AP-REQ` over port 445 and processes it completely offline from the KDC:

1. **Ticket Unsealing:** `FS01` uses its own long-term machine account key (K_{FS01}) to decrypt the incoming Service Ticket.
2. **Session Key Extraction:** From the decrypted ticket, `FS01` extracts the Service Session Key ($K_{\text{session_FS}}$) and Ahmed's identity.
3. **Authenticator Verification:** `FS01` uses $K_{\text{session_FS}}$ to decrypt the incoming Authenticator.
   - It verifies that the timestamp is within 5 minutes.
   - It checks its local replay cache (a memory buffer of recently received authenticators) to ensure this exact authenticator hasn't been replayed within the skew window.
   - It confirms that the name inside the Authenticator matches the name inside the decrypted ticket.
4. **PAC Ingestion & Access Token Creation:** `FS01` extracts the PAC from the ticket, verifies the Server Signature with K_{FS01}, and passes the SIDs directly into the Windows Security Subsystem (LSA/Kernel). The Windows kernel constructs a primary security access token for Ahmed, listing his user SID and all his group SIDs.
5. **Mutual Authentication (`AP-REP`):** If `mutual-required` was requested, `FS01` constructs an `AP-REP` packet (message type 15). It takes the timestamp from Ahmed's Authenticator, encrypts it with $K_{\text{session_FS}}$, and sends it back to `WKSTN01`.
   - When `WKSTN01` decrypts this timestamp with its copy of $K_{\text{session_FS}}$ and sees its own timestamp mirrored back, it has mathematical proof that `FS01` successfully decrypted the Service Ticket. Ahmed is now guaranteed he is talking to the genuine `FS01` and not an impersonator.

File Access Granted:
Both sides now use $K_{\text{session_FS}}$ (or a derived subkey) to digitally sign and optionally encrypt SMB traffic. The Windows I/O manager evaluates the Discretionary Access Control List (DACL) on `\\FS01\engineering\specifications.xlsx` against Ahmed's newly created access token. The token contains the SID for `Engineering-Department-Global`, which has `Read/Execute` permissions. The file handle opens, and Ahmed begins reading the spreadsheet.

3. Deep-Dive: Anatomy of Tickets, Keys & The PAC

3.1 Diagram 2: Anatomical Breakdown of a Ticket & Session Key



3.2 The Envelope Model: Outer Wrapper vs Inner Ciphertext

A Kerberos ticket functions exactly like an armored, tamper-evident security pouch:

- **The Outer Wrapper (Plaintext):** Contains metadata that any network hop, firewall, or endpoint operating system can inspect without breaking encryption. This includes the Kerberos protocol version (`5`), the domain realm (`CORP.LOCAL`), and the target service principal name (`krbtgt/CORP.LOCAL` or `cifs/FS01.corp.local`).

- **The Encrypted Ticket Part (`EncTicketPart`):** The actual payload inside the pouch is encrypted with a symmetric key known *only* to the issuing KDC and the recipient service.
  - In a **TGT**, the `EncTicketPart` is sealed with K_{krbtgt}.
  - In a **Service Ticket**, the `EncTicketPart` is sealed with the target service account's secret key (K_{service}).
- **The Client Ingestion Mechanism:** When the KDC returns an `AS-REP` or `TGS-REP`, the client receives the encrypted ticket blob, which it cannot open. Alongside that blob, the KDC includes a parallel encrypted data block (`enc-part`) encrypted with a key the client *does* possess. By decrypting this parallel block, the client extracts the **Session Key**. Now, both the client and the eventual destination server possess the identical Session Key, while no eavesdropper on the wire has seen it.

3.3 The Heart of Active Directory: Privilege Attribute Certificate (PAC)

To truly understand Active Directory internals, one must understand why MIT Kerberos and Microsoft Kerberos fundamentally diverge.

Why MIT Kerberos Did Not Have a PAC:
When MIT designed Kerberos v5 for UNIX workstations in Project Athena, Kerberos had exactly one operational responsibility: **Authentication** ("Are you truly Ahmed?"). Once an authenticated UNIX workstation verified Ahmed's Kerberos identity, the workstation consulted its own local configuration files (`/etc/passwd`, `/etc/group`, or local LDAP/NIS) to determine what groups Ahmed belonged to and what files he could read. Authorization was completely decoupled from Kerberos tickets.

Why Microsoft Had to Invent the PAC:
In Active Directory, identity is not defined by a simple username string. Windows access control is based entirely on **Security Identifiers (SIDs)** and centralized group tokens. When a user accesses an object in Windows, the operating system kernel compares the SIDs embedded in the caller's access token against the Discretionary Access Control List (DACL) attached to the securable object.

If Microsoft had implemented RFC 4120 Kerberos without modification, an architectural nightmare would have ensued: Every time Ahmed tried to open a file on `FS01`, execute a query on `SQL01`, or browse an internal website on `WEB01`, that target server would have to pause the thread, open a network connection back to a Domain Controller over LDAP, and query:

> *"Hey DC, user ahmed just presented a valid Kerberos ticket. What groups is ahmed a member of? What is his primary group RID? Does he have any SID History from legacy domain migrations?"*

In an enterprise with 50,000 employees, hundreds of servers, and millions of daily file, database, and web requests, the Domain Controllers would instantly collapse under catastrophic **LDAP connection storms**. The network would suffer massive latency overhead on every single user action.

The Microsoft Solution:
Microsoft invented the **Privilege Attribute Certificate (PAC)**, formalized under specification **[MS-PAC]**. Instead of forcing every service server to query the DC on every connection, the KDC queries the Active Directory database *once* during initial ticket issuance. The KDC compiles Ahmed's entire security profile—his user SID, his primary group RID, all domain global and universal security group SIDs, and any historical SIDs—and packages them directly inside the encrypted authorization-data field of the Kerberos ticket.

When `FS01` receives Ahmed's Service Ticket, `FS01` simply decrypts the ticket, reads the pre-compiled PAC, and directly builds a complete Windows Security Access Token in kernel memory. **Zero LDAP calls to the Domain Controller are needed.**

3.4 Cryptographic Signatures & The Dual-Signature Trust Model

If the PAC travels inside a ticket and carries absolute authority regarding what security groups a user belongs to, what stops a malicious user or an attacker from tampering with the PAC to add the Domain Admins SID (`S-1-5-21-...-512`)?

To guarantee integrity, Microsoft implemented a strict **Dual-Signature Model** within the PAC structure:

1. **The Server Signature (`PAC_SERVER_SIGNATURE`):**
   - **Signed With:** The target service account's long-term key (K_{service} for a Service Ticket; K_{krbtgt} for a TGT).
   - **Purpose:** Cryptographically seals the PAC data buffers. When the target service (`FS01`) receives the ticket, it decrypts the ticket using K_{FS01}, locates the Server Signature, and verifies the cryptographic hash against the PAC contents using K_{FS01}. This guarantees to `FS01` that the PAC has not been tampered with since the ticket was generated.
2. **The KDC Signature (`PAC_PRIVSVR_CHECKSUM`):**
   - **Signed With:** The domain's root `krbtgt` key (K_{krbtgt}).
   - **Purpose:** Cryptographically signs the Server Signature itself using the Domain Controller's master key.
   - **Why this is architecturally vital:** Without the KDC Signature, a compromised service server (e.g., an untrusted web application running with its own service account) could decrypt a legitimate ticket sent by Ahmed, modify Ahmed's PAC to insert Enterprise Admins rights, re-sign the PAC using its *own* service key, and forward the forged ticket to another backend server. Because the KDC signature is sealed with K_{krbtgt}—a key known *only* to Domain Controllers—a service cannot tamper

with a ticket or elevate its own privileges without invalidating the KDC signature.

3. **Ticket Checksum & PAC Attributes (Modern Hardening):**
   - Under security updates addressing PAC elevation vulnerabilities (e.g., CVE-2021-42287 and CVE-2022-37967), modern KDCs append additional cryptographic checksums:
     - `PAC_ATTRIBUTES_INFO`: Contains bit flags specifying how the PAC was requested.
     - `PAC_REQUESTOR`: Contains the SID of the account that requested the ticket.
     - `Ticket Checksum`: A cryptographic hash of the entire outer ticket payload signed with K_{krbtgt}, binding the PAC immutably to that specific ticket instance and preventing PAC replay across different tickets.

3.5 Structural Breakdown: PAC Buffers Demystified

The PAC is structured as an array of variable-length buffers preceded by a header (`PACTYPE`). Below is the exact architectural breakdown of the validation buffers:

```
PACTYPE Header:
├── cBuffers  : Total count of validation buffers inside the PAC (typically 5 to 7)
├── Version   : PAC structure version (0)
└── Buffers[] : Array of directory pointers (BufferType, BufferSize, Offset)

Buffer 1: KERB_VALIDATION_INFO (Buffer Type 0×00000001)
├── LogonTime / LogoffTime / KickOffTime : Account lifecycle timestamps
├── PasswordLastSet / PasswordMustChange : Password management timestamps
├── EffectiveName / FullName             : Account display strings
├── LogonScript / ProfilePath / HomeDir  : User profile environmental paths
├── UserId (User RID)                    : The account's relative identifier (e.g., 1105)
├── PrimaryGroupId                       : Primary group RID (e.g., 513)
├── GroupCount & GroupIds[]              : Array of group RIDs and attributes in the local domain
├── UserFlags / UserAccountControl       : Account state flags (e.g., NORMAL_ACCOUNT)
├── UserSessionKey                       : 128-bit session key for SMB signing fallback
├── LogonServer                          : Name of the DC that authenticated the user
├── LogonDomainName & LogonDomainId      : Domain NetBIOS name and Domain Security Identifier (Domain SID)
└── ExtraSids[] (SID History)            : Array of full SIDs representing cross-domain migration history

Buffer 2: PAC_CLIENT_INFO (Buffer Type 0×0000000A)
├── ClientId  : Windows FILETIME timestamp indicating when the ticket was generated by the KDC
└── Name      : UTF-16LE encoded client username string (used to verify ticket ownership)

Buffer 3: PAC_SERVER_SIGNATURE (Buffer Type 0×00000006)
├── SignatureType : Cryptographic algorithm (HMAC-SHA1-96-AES256 [etype 18] or HMAC-MD5 [etype 23])
└── Signature     : Cryptographic digest computed over all PAC data buffers using the Target Service Key

Buffer 4: PAC_PRIVSVR_CHECKSUM (KDC Signature, Buffer Type 0×00000007)
├── SignatureType : Cryptographic algorithm
└── Signature     : Cryptographic digest computed over the Server Signature using the Domain KDC Key (K_krbtgt)

Buffer 5: PAC_UPN_DNS_INFO (Buffer Type 0×0000000C)
├── UpnLength & UpnOffset              : User Principal Name (e.g., ahmed@corp.local)
├── DnsDomainNameLength & Offset       : FQDN of the domain (e.g., corp.local)
└── SamAccountNameLength & Offset      : sAMAccountName string

Buffer 6 & 7: Modern Hardening Buffers (CVE-2021-42287 / CVE-2022-37967)
├── PAC_ATTRIBUTES_INFO (0×00000011)   : Flags indicating whether PAC signatures were enforced
└── PAC_REQUESTOR (0×00000012)         : SID of the ticket requester to prevent identity spoofing
```

Because this entire structure is cryptographically signed and encrypted inside the ticket, `FS01` simply executes `KerbValidatePacSignature` locally. If valid, `FS01` passes `KERB_VALIDATION_INFO` directly to the Windows Security Reference Monitor (SRM) to create Ahmed's local Windows token without contacting Active Directory.

4. Cryptographic Engines, Key Derivation & Ticket Storage

4.1 Supported Encryption Types

Kerberos negotiates cryptographic algorithms via integer identifiers called **Encryption Types (etypes)** defined in RFC standards and Microsoft specifications:

Etype ID	Standard Name	Cryptographic Primitive	Key Size	Modern Security Assessment
18	`AES256-CTS-HMAC-SHA1-96`	AES-256 Cipher Text Stealing + HMAC-SHA1	256 bits	**Gold Standard.** Strongly recommended for all domain accounts and services.
17	`AES128-CTS-HMAC-SHA1-96`	AES-128 Cipher Text Stealing + HMAC-SHA1	128 bits	**Secure.** Standard AES performance tier.
23	`RC4-HMAC` (RFC 4757)	Rivest Cipher 4 + MD4/MD5 HMAC	128 bits	**Architecturally Fragile.** Legacy suite. Vulnerable to fast offline brute-force cracking.
16	`DES3-CBC-SHA1`	Triple DES + SHA1	168 bits	**Deprecated.** Disabled by default in modern Windows Server releases.
1 / 3	`DES-CBC-CRC / MD5`	Single DES (56-bit)	56 bits	**Broken.** Deprecated and blocked by default since Windows Server 2008 R2.

4.2 Key Derivation Mechanics

Kerberos never uses a user's plaintext password directly as an encryption cipher key. It converts the human-readable password into a raw cryptographic key through deterministic derivation algorithms:

1. AES Keys Derivation (RFC 3962):

For modern AES-128 and AES-256 encryption, Active Directory uses **PBKDF2 (Password-Based Key Derivation Function 2)** with HMAC-SHA1:

$$\text{Salt} = \text{UPPERCASE}(\text{Realm}) + \text{CaseSensitive}(\text{sAMAccountName})$$

$$\text{Example Salt} = \text{CORP.LOCALahmed}$$

$$K_{\text{AES256}} = \text{PBKDF2}(\text{HMAC-SHA1}, \text{Password}, \text{Salt}, \text{Iterations} = 4096, \text{KeyLength} = 32 \ \ \text{bytes})$$

Because the salt includes the full realm and username, two users in different domains with the exact same password produce completely distinct AES keys. Furthermore, the 4,096 iterations of PBKDF2 impose substantial computational resistance against offline dictionary attacks.

2. The Legacy RC4 Trap (RFC 4757):

For `RC4-HMAC` (etype 23), the key derivation algorithm is dangerously simple:

$$K_{\text{RC4}} = \text{MD4}(\text{UTF-16LE}(\text{Password})) \equiv \text{NTLM Hash}$$

In the RC4 Kerberos cipher suite, **the Kerberos symmetric key is mathematically identical to the user's NTLM hash**. There is no salt. There are zero derivation iterations. This cryptographic equivalence is one of the most critical architectural friction points in Active Directory:

- If an adversary captures an account's NTLM hash, they automatically hold that account's Kerberos RC4 key without needing the plaintext password.
- If an adversary captures an RC4 Kerberos response, cracking that Kerberos ticket is mathematically identical to cracking a raw NTLM hash offline.

4.3 Ticket Storage & In-Memory Formats

Once Kerberos tickets are minted, endpoints must cache them in memory to avoid requesting new tickets on every single network operation.

1. Windows Operating System Storage:

- **LSASS Process Memory:** On Windows, all active Kerberos tickets are managed by the Kerberos Security Support Provider (`kerberos.dll`) running inside the **Local Security Authority Subsystem Service (`lsass.exe`)**.
- **Logon Sessions:** LSASS segregates tickets by Windows Logon Session (identified by a 64-bit Locally Unique Identifier, or LUID).
  - Ahmed's interactive desktop runs in a specific Logon Session LUID (e.g., `0x3e7` for SYSTEM, `0x14a89b` for Ahmed).
  - Only processes executing within Ahmed's logon session (or processes possessing elevated `SeDebugPrivilege` running as SYSTEM) can access Ahmed's cached TGT and Service Tickets.
- **The `.kirbi` File Format:** When tools extract tickets from Windows memory, they serialize the ASN.1 `KRB-CRED` structure (defined in RFC 4120 Section 5.8) into raw binary or Base64-encoded `.kirbi` files.

2. Linux / macOS Storage:

- **Credential Caches (ccache):** POSIX environments running MIT Kerberos, Heimdal, or SSSD store tickets in Credential Cache files.
  - The active cache is pointed to by the environment variable `KRB5CCNAME`.
  - Typical location: `/tmp/krb5cc_<UID>` or in the Linux kernel keyring (`KEYRING:persistent:<UID>`).
- **Keytab Files (`.keytab`):** Unlike Windows computer accounts which rotate their own machine passwords automatically in the registry, Linux servers hosting Kerberos services store their long-term symmetric keys in **Keytab** files (typically `/etc/krb5.keytab`). A keytab is essentially a file containing principal names paired with their derived DES, RC4, or AES cryptographic keys.

5. Delegation Architecture & Cross-Tier Identity Transfer

5.1 The Architectural Need for Multi-Tier Delegation

In modern enterprise applications, services rarely exist in isolation. Consider a classic three-tier enterprise web application:

1. **Tier 1 (User Desktop):** Ahmed browses to an internal HR portal at `https://hr.corp.local`.
2. **Tier 2 (Front-End Web Server):** `WEB01.corp.local` hosts the web application.
3. **Tier 3 (Back-End Database):** `SQL01.corp.local` stores the employee salary records.

If Ahmed queries his personal payroll record, `WEB01` must query `SQL01`. But how should `WEB01` authenticate to `SQL01`?

- *Option A:* `WEB01` connects to `SQL01` using a static generic service account (`svc_web`).
  - *Failure:* `SQL01` has no idea which human user initiated the query. The database cannot enforce database-level row permissions or maintain an audit log of who viewed what salary record.
- *Option B:* Ahmed allows `WEB01` to impersonate him against `SQL01`.
  - *The Architectural Challenge:* Ahmed only gave `WEB01` a Service Ticket for `http/hr.corp.local`. `WEB01` cannot use that ticket to authenticate to `mssql/SQL01.corp.local` because that ticket is encrypted with `WEB01`'s key, not `SQL01`'s key!

To solve this, Microsoft introduced **Kerberos Delegation**—mechanisms that allow a service to impersonate an authenticated user when contacting downstream backend services.

5.2 Diagram 3: Delegation Architectures Compared



5.3 Unconstrained Delegation: The Complete TGT Handover

Introduced in Windows 2000, **Unconstrained Delegation** was the original delegation model.

- **The Configuration:** An Active Directory administrator marks a computer or service account with the `TRUSTED_FOR_DELEGATION` bitmask in its `userAccountControl` attribute.
- **The Mechanism:** When Ahmed connects to `WEB01` over Kerberos, Ahmed's workstation inspects `WEB01`'s Active Directory object. Seeing that `WEB01` has Unconstrained Delegation enabled, `WKSTN01` automatically extracts Ahmed's

TGT from LSASS, marks it as forwardable, and embeds a **full copy of Ahmed's TGT** directly inside the Authenticator of the `AP-REQ` packet!

- **The Execution:** When `WEB01` decrypts the `AP-REQ`, it extracts Ahmed's full TGT and injects it directly into `WEB01`'s own local LSASS ticket cache.
- **The Architectural Hazard:** `WEB01` now possesses Ahmed's TGT. Because the TGT is unconstrained, `WEB01` can present that TGT to the KDC to request Service Tickets for **any service in the entire Active Directory forest** as Ahmed. If an adversary compromises `WEB01`, any user who subsequently connects to `WEB01` (including a Domain Administrator) has their full TGT dumped from LSASS, resulting in complete forest compromise.

5.4 Constrained Delegation: S4U2Self & S4U2Proxy

To remediate the catastrophic blast radius of Unconstrained Delegation, Microsoft introduced **Constrained Delegation** in Windows Server 2003, backed by two proprietary Kerberos protocol extensions: **Service for User to Self (S4U2Self)** and **Service for User to Proxy (S4U2Proxy)** (specified in **[MS-SFU]**).

Core Concepts:

- **Whitelist Enforcement:** Delegation is locked down to a strictly defined list of SPNs populated in the front-end account's `msDS-AllowedToDelegateTo` attribute (e.g., `mssql/SQL01.corp.local`). The front-end cannot impersonate the user to arbitrary services.
- **No Client TGT Delivery:** The client does not transmit their TGT to the front-end server.
- **Protocol Transition (S4U2Self):** What if Ahmed authenticated to `WEB01` via an HTTP basic form, a client certificate, or NTLM instead of Kerberos? The front-end server doesn't even have a Kerberos ticket from Ahmed!
  - Under S4U2Self, if `WEB01` has the `TRUSTED_TO_AUTH_FOR_DELEGATION` flag set, `WEB01` can send a `TGS-REQ` to the KDC requesting a Service Ticket **to itself** representing Ahmed, without requiring Ahmed to provide a password.
- **Impersonating Downstream (S4U2Proxy):**
  - Once `WEB01` has a Service Ticket for Ahmed (either from a real Kerberos AP-REQ or generated via S4U2Self), `WEB01` sends a `TGS-REQ` containing that ticket to the KDC, asking for a ticket to `mssql/SQL01.corp.local`.
  - The KDC verifies that `mssql/SQL01.corp.local` is listed in `WEB01`'s `msDS-AllowedToDelegateTo` attribute.
  - If verified, the KDC issues a Service Ticket for Ahmed to access `SQL01`.

5.5 Resource-Based Constrained Delegation (RBCD)

While Constrained Delegation solved the security flaws of Unconstrained Delegation, it introduced a significant administrative problem:

- **Centralized Administrative Friction:** To configure Classic Constrained Delegation, the front-end server's `msDS-AllowedToDelegateTo` attribute must be edited. Modifying this attribute requires elevated domain privileges (Domain Admins or delegated administrators of the front-end account). The adminis-

trators of the target backend service (`SQL01`) have zero control over who is allowed to delegate access to their own database.

In Windows Server 2012, Microsoft introduced **Resource-Based Constrained Delegation (RBCD)**:

- **Inverting the Trust Relationship:** Instead of configuring delegation permissions on the *caller* (`WEB01`), permissions are configured on the *resource target* (`SQL01`).
- **The Security Descriptor:** The resource target object in Active Directory possesses an attribute named `msDS-AllowedToActOnBehalfOfOtherIdentity`. This attribute contains a binary security descriptor (an Access Control List) specifying which machine or service accounts are permitted to impersonate users to this resource.
- **The Flow:**
  1. `WEB01` uses S4U2Self to obtain a ticket for Ahmed to itself.
  2. `WEB01` uses S4U2Proxy to ask the KDC for a ticket to `SQL01` as Ahmed.
  3. The KDC inspects `SQL01`'s `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute.
  4. If `WEB01`'s SID is present in that security descriptor, the KDC issues the ticket.
- **Architectural Shift:** Now, local server administrators can control their own delegation boundaries. However, as explored in Section 6, this creates a major architectural attack surface if an adversary obtains write permissions over any computer object's attributes.

6. Architectural Friction Points & Vulnerability Foundations

Security vulnerabilities in Active Directory are rarely software coding "bugs" in the Kerberos binary. Instead, they are the mathematically predictable consequences of **architectural trade-offs** between usability, distributed performance, backward compatibility, and cryptographic design.

6.1 Offline Cryptanalysis of Service Envelopes (The Architectural Origin of Kerberoasting)

- **The Design Decision:** To maintain enterprise performance, the KDC issues Service Tickets encrypted using the long-term secret key of the target service account (K_{service}). The target service decrypts the ticket locally without needing to verify the ticket with the KDC over the network.
- **The Protocol Rule:** Any valid domain user (even the least-privileged user in the forest) has legitimate rights to query the KDC and request a Service Ticket for *any* registered Service Principal Name (SPN) in Active Directory.
- **The Friction Point:** If an enterprise administrator configures a service (such as MSSQL or IIS) to execute under a standard Active Directory User

Account rather than a built-in computer account or a Group Managed Service Account (gMSA), that service's long-term key (K_{service}) is derived from a human-created password.

- **The Architectural Consequence:** An adversary authenticated as Ahmed can query the KDC for a Service Ticket addressed to `MSSQLSvc/sql01.corp.local`. The KDC dutifully returns a `TGS-REP` containing a ciphertext block encrypted with that service account's password key. The adversary does not need to send a single packet to `SQL01`. They extract the ciphertext from the `TGS-REP` packet, carry it to an offline GPU cracking cluster, and guess billions of candidate passwords against the ciphertext. If the service account has a weak or medium-complexity password, the adversary cracks the secret offline completely undetected by endpoint defenses on `SQL01`.

6.2 Pre-Authentication Bypass & Arbitrary AS-REP Harvesting (The Mechanism Behind AS-REP Roasting)

- **The Design Decision:** Pre-Authentication exists specifically to prevent offline guessing attacks against user passwords. If a client had to receive an `AS-REP` before proving knowledge of its key, an attacker could request an `AS-REP` for any username in the domain and receive ciphertext encrypted with that user's password hash.
- **The Backward-Compatibility Exception:** Active Directory includes a User Account Control flag named `DONT_REQ_PREAUTH` (`0×00400000`). This flag was created decades ago for legacy applications and Unix Kerberos implementations that did not support pre-authentication timestamps.
- **The Friction Point:** When `DONT_REQ_PREAUTH` is enabled on any user account, the KDC waives the requirement for `PA-ENC-TIMESTAMP`.
- **The Architectural Consequence:** An unauthenticated or low-privileged attacker on the network sends an `AS-REQ` naming that target account. The KDC immediately generates a standard `AS-REP` response containing a session key and metadata encrypted with the target user's password key (K_{user}). The attacker captures this packet and cracks the user's password offline, without ever knowing the password or generating a failed logon event (Event 4771).

6.3 Stateless KDC Trust & Cryptographic Omnipotence (The Structural Flaw of Golden Tickets)

- **The Design Decision:** The Kerberos protocol is **stateless**. Domain Controllers do not maintain a live database of all active, issued tickets. When a client presents a TGT to the KDC during a `TGS-REQ`, the KDC does not ask: *"Did I issue this ticket five minutes ago?"* Instead, the KDC simply decrypts the TGT using its root service key (K_{krbtgt}) and inspects the inner data structures.
- **The Friction Point:** All trust in the entire Active Directory domain is rooted in knowledge of the `krbtgt` account's long-term secret key.

- **The Architectural Consequence:** If an adversary compromises the `krbtgt` password hash (e.g., via directory synchronization or NTDS extraction), the adversary becomes an omnipotent offline KDC:
  - The adversary can locally manufacture a completely fabricated TGT out of thin air.
  - They can populate the PAC with arbitrary SIDs (such as Domain Admins RID `512` or Enterprise Admins RID `519`).
  - They can forge both the Server Signature and KDC Signature using the stolen K_{krbtgt}.
  - They can set the validity timestamp to 10 years in the future.
  - When this forged ticket is presented to *any* Domain Controller in the forest, the DC successfully decrypts it with K_{krbtgt}, verifies the signatures, trusts the PAC implicitly, and grants administrative access.

6.4 Decentralized Service Validation & Local PAC Acceptance (The Mechanism of Silver Tickets)

- **The Design Decision:** As established in Section 2.5, during the Application (AP) Exchange, the target service server (`FS01`) validates the client's Service Ticket completely offline from the KDC.
- **The Friction Point:** The Service Ticket is encrypted with the service account's secret key (K_{service}), and the PAC inside that ticket is signed with that exact same K_{service}. By default, the service does not contact the KDC to verify if the KDC signature is legitimate.
- **The Architectural Consequence:** If an adversary compromises the password hash of a specific computer account (e.g., `FS01$`) or a service account, the adversary can forge a valid **Service Ticket** directly for that service without ever contacting a Domain Controller:
  - The adversary crafts a Service Ticket, injects administrative SIDs into the PAC, signs the Server Signature with K_{service}, and encrypts the ticket with K_{service}.
  - When presented to `FS01`, the file server unseals the ticket, validates its own signature, and grants full local administrative access.
  - Because the KDC was completely bypassed, **zero Kerberos ticket generation events (Event 4769) are logged on the Domain Controllers.**

6.5 Legitimate Pre-Auth with Modified Authorization Data (The Mechanism of Diamond Tickets)

- **The Detection Problem:** Golden Tickets often generate noticeable detection anomalies: an `AP-REQ` or `TGS-REQ` appears without any preceding `AS-REQ` (Event 4768) occurring on the domain controllers, or the ticket uses encryption types or options inconsistent with the client workstation's OS.

- **The Architectural Workaround:** A “Diamond Ticket” operates by requesting a **genuine, legitimate TGT** from the KDC via standard AS-REQ (generating a valid Event 4768). Then, utilizing a compromised `krbtgt` key, the adversary unseals the legitimate TGT, modifies the PAC inside to append privileged SIDs, recalculates the PAC signatures with K_{krbtgt}, and re-encrypts the ticket.
- **The Architectural Consequence:** The resulting TGT possesses legitimate KDC-issued timestamps, correct realm metadata, and a real preceding AS-REQ event in the SIEM, while still conferring unauthorized administrative privileges.

6.6 Ephemeral Ticket Extraction & Identity Replay (Pass-the-Ticket & Overpass-the-Hash)

- **Credential Caching in OS Subsystems:** Because Kerberos requires continuous authentication across background processes, LSASS maintains active tickets in plain memory.
- **The Friction Point:** If an adversary gains local administrative execution on a workstation where a high-privileged user is logged in, the adversary can dump the raw ASN.1 ticket blobs (`.kirbi` or `ccache`) directly from memory.
- **Pass-the-Ticket:** The adversary imports the stolen TGT into their own logon session. Because the TGT is cryptographically valid and signed, the KDC honors it regardless of which physical network interface transmitted the packet.
- **Overpass-the-Hash (Pass-the-Key):** If an adversary only has an NTLM hash, they exploit the cryptographic property that the `RC4-HMAC` Kerberos key equals the NTLM hash (Section 4.2). The adversary presents the NTLM hash directly to the Kerberos subsystem to complete an `AS-REQ` pre-authentication exchange, receiving a fully valid Kerberos TGT and successfully transitioning from legacy NTLM authentication into the modern Kerberos ecosystem.

6.7 Public Key Cryptography Extensions (PKINIT & Shadow Credentials)

- **The Mechanism (RFC 4556): PKINIT (Public Key Cryptography for Initial Authentication)** allows clients to authenticate during the AS Exchange using an X.509 digital certificate instead of a symmetric password. The client signs the pre-authentication data using its private key, and the KDC validates the certificate against Active Directory Certificate Services (AD CS).
- **The Architectural Attribute (`msDS-KeyCredentialLink`):** Windows Server 2016 introduced “Shadow Credentials” to support Windows Hello for Business without full PKI rollouts. An account’s public key can be written directly to its `msDS-KeyCredentialLink` attribute in Active Directory.
- **The Vulnerability Foundation:** If an adversary gains write access (`GenericWrite` or `WriteProperty`) over an Active Directory object (user or com-

puter), the adversary does not need to reset the victim's password. They simply generate an arbitrary X.509 keypair, write the public key into the victim's `msDS-KeyCredentialLink` attribute, and execute an `AS-REQ` using PKINIT to obtain a full TGT for the victim account.

7. Active Directory Schema, Attributes & Directory Objects

7.1 Crucial Directory Attributes & Bitmasks

The following directory attributes in the Active Directory schema directly control Kerberos protocol behavior:

Attribute Name	Syntax / Format	Operational Impact on Kerberos
`servicePrincipalName`	Multi-valued String	Links a network service identity to the directory account whose long-term keys will encrypt its tickets.
`userAccountControl`	32-bit Integer Bitmask	Contains operational flags controlling authentication and delegation: • `DONT_REQ_PREAUTH` (`0×00400000`): Disables pre-auth. • `TRUSTED_FOR_DELEGATION` (`0×00080000`): Enables Unconstrained Delegation. • `TRUSTED_TO_AUTH_FOR_DELEGATION` (`0×01000000`): Enables Protocol Transition (S4U2Self). • `ACCOUNTDISABLE` (`0×00000002`): Prevents ticket issuance.
`msDS-AllowedToDelegateTo`	Multi-valued String	Whitelist of SPNs that this account is permitted to impersonate users to under Classic Constrained Delegation.
`msDS-AllowedToActOnBehalfOfOtherIdentity`	Binary Security Descriptor	Access Control List on a resource object specifying which service principals are allowed to delegate to it via RBCD.
`msDS-KeyCredentialLink`	Multi-valued DN-Binary	Contains public key specifications for PKINIT authentication (Windows Hello / Shadow Credentials).
`msDS-SupportedEncryptionTypes`	32-bit Integer Bitmask	Defines supported ciphers: • `0×1`: DES-CBC-CRC • `0×2`: DES-CBC-MD5 • `0×4`: RC4-HMAC • `0×8`: AES128-CTS-HMAC-SHA1-96 • `0×10`: AES256-CTS-HMAC-SHA1-96
`supplementalCredentials`	Encrypted Octet String	Stores primary password-derived Kerberos keys (AES salts and hashes) in `ntds.dit`.

7.2 Service Principal Name (SPN) Architecture & Forest Constraints

An SPN is the unique identifier by which a Kerberos client references a service instance.

Standard SPN Syntax:

$$\text{SPN} = \langle \text{service class} \rangle / \langle \text{host} \rangle : \langle \text{port} \rangle / \langle \text{service name} \rangle$$

- **Service Class:** The architectural category of the service (e.g., `HOST`, `cifs`, `MSSQLSvc`, `HTTP`, `LDAP`, `WSMan`).
- **Host:** The Fully Qualified Domain Name (FQDN) or NetBIOS name of the server hosting the service (e.g., `fs01.corp.local`).
- **Port:** The listening port (mandatory for multi-instance services like SQL instances: `:1433`).

Critical Forest Constraints:

1. **Uniqueness Requirement:** Within an Active Directory forest, an SPN **must be globally unique**. Two distinct accounts cannot register the same SPN. If an administrator accidentally registers duplicate SPNs, the KDC cannot determine which account key to use, and Kerberos ticket issuance fails with error `KDC_ERR_BADOPTION` or `KRB_ERR_DUPLICATE_TICKETS`.
2. **Account Association Impact:**
   - **Computer Account SPNs:** When an SPN is registered on a machine account (e.g., `HOST/FS01`), the Service Ticket is encrypted with the computer's 120-character machine password. Machine accounts rotate passwords automatically every 30 days and are practically impossible to crack offline.
   - **User Account SPNs:** When an SPN is registered on a human user account (e.g., `svc_sql`), the ticket is encrypted with the user's password hash, exposing that account directly to offline dictionary cracking.

7.3 The `krbtgt` Domain Account & Dual-Rotation Lifecycle

The `krbtgt` user object (RID `502`) is the root cryptographic authority for the entire Active Directory domain:

- **Properties:** Built-in account, disabled for interactive logon (`ACCOUNTDISABLE`), password managed internally by the Domain Controllers.
- **Key Roles:** Its long-term secret key (K_{krbtgt}) encrypts every TGT issued in the domain and signs the KDC Privilege Attribute Certificate (PAC) signature.
- **The Two-Password Architecture:** Active Directory maintains both the *current* password and the *previous* password for `krbtgt`.
  - When the `krbtgt` password is reset, the old password moves to the "previous" key slot.
  - The KDC continues to honor existing TGTs that were minted using the previous key until their maximum ticket lifetime expires (typically 10 hours).

- **The Mandatory Dual-Reset Procedure:** To completely purge forged Golden Tickets after a security incident, the `krbtgt` account password **must be reset twice**, separated by a time interval longer than the maximum ticket lifetime (at least 10 hours, recommended 24 hours):
  - **First Reset:** Replaces the active key. Old forged tickets continue to function via the "previous" key slot.
  - **Wait Period:** Allows legitimate enterprise users to naturally renew tickets.
  - **Second Reset:** Overwrites the "previous" slot with the key from the first reset, completely invalidating every historical ticket across the entire forest.

8. Defensive Telemetry, Detection Engineering & Hardening

8.1 Windows Security Event Telemetry

Kerberos operations generate rich, high-fidelity security events on Active Directory Domain Controllers:

Event ID	Log Name	Operational Meaning	Strategic Telemetry Significance
4768	Security	A Kerberos authentication ticket (TGT) was requested (`AS-REQ`).	Monitors initial logons. Inspect for accounts with Pre-Authentication disabled (`Pre-Auth Type: 0`) and unusual encryption types (`0x17` vs `0x12`).
4769	Security	A Kerberos service ticket was requested (`TGS-REQ`).	Primary telemetry for service ticket generation. High volumes of requests for user-associated SPNs or RC4 encryption (`TicketOptions: 0x40810000`, `Encryption: 0x17`) indicate reconnaissance or ticket extraction.
4770	Security	A Kerberos service ticket was renewed.	Monitors ticket extension patterns.
4771	Security	Kerberos pre-authentication failed.	**High-Value.** Indicates bad password attempts (`Failure Code: 0x18`) or clock skew (`0x25`). Useful for detecting password spraying.
4624	Security	An account was successfully logged on.	Look for Logon Type 3 (Network) accompanied by Kerberos authentication packages.
4741	Security	A computer account was created.	Telemetry marker for Resource-Based Constrained Delegation (RBCD) exploitation staging.

Event ID	Log Name	Operational Meaning	Strategic Telemetry Significance
5136	Security	A directory service object was modified.	Vital for detecting changes to `msDS-AllowedToDelegateTo`, `msDS-AllowedToActOnBehalfOfOtherIdentity`, or `msDS-KeyCredentialLink`.

8.2 Detection Heuristics & Hunting Logic

1. Kerberoasting Detection (Kusto Query Language - KQL):

Detects anomalous volume of Service Ticket requests requesting legacy RC4 encryption for user accounts:

```
SecurityEvent
| where EventID == 4769
| where ServiceName !endswith "$" and ServiceName ≠ "krbtgt"
| where TicketEncryptionType == "0×17" // RC4-HMAC
| where Status == "0×0" // Success
| summarize RequestCount = count(), SPNList = make_set(ServiceName) by TargetUserName, ClientAddress, bin(TimeGenerated, 5m)
| where RequestCount ≥ 5
| project TimeGenerated, TargetUserName, ClientAddress, RequestCount, SPNList
```

2. Golden Ticket Anomaly Heuristic:

Golden Tickets frequently lack preceding AS-REQ telemetry or exhibit impossible ticket lifetimes:

- **Heuristic A (TGS without AS):** Flag any `Event 4769` where no corresponding `Event 4768` (TGT request) was logged for that user/IP within the preceding 10 hours.
- **Heuristic B (Domain Controller TGS):** Flag any Service Ticket request (`4769`) where the requesting client IP is an unmanaged external workstation but the account claims membership in Domain Admins (`RID 512`).

8.3 Enterprise Hardening Blueprint & Remediation Roadmap

Securing Kerberos requires eliminating weak legacy cryptography and restricting delegation privileges:

1. **Deprecate RC4 Domain-Wide:**
   - Configure Group Policy: `Computer Configuration → Windows Settings -> Security Settings → Local Policies → Security Options → Network security: Configure encryption types allowed for Kerberos`.
   - Restrict allowed encryption types strictly to `AES128_HMAC_SHA1`, `AES256_HMAC_SHA1`, and `Future encryption types`.
2. **Eliminate User-Account SPNs:**
   - Transition all database, web, and custom service accounts from standard Active Directory user accounts to **Group Managed Service Accounts (gMSAs)**. gMSAs use complex, automatically rotated 128-byte passwords that render offline cracking mathematically infeasible.
3. **Audit and Eradicate Unconstrained Delegation:**

- Query all domain objects where `userAccountControl` contains the `TRUSTED_FOR_DELEGATION` flag.
- Refactor these systems to use **Resource-Based Constrained Delegation (RBCD)**, ensuring that sensitive credentials are never stored in intermediate LSASS caches.

4. **Deploy the "Protected Users" Security Group:**
   - Add all Tier-0 administrative accounts (Enterprise Admins, Domain Admins, sensitive service principals) to the **Protected Users** security group (introduced in Windows Server 2012 R2).
   - *Protective Controls Enforced:*
     - Credentials are never cached in LSASS.
     - NTLM authentication is completely disabled for the account.
     - Kerberos pre-authentication cannot be downgraded to RC4.
     - Delegation is strictly disabled (TGTs cannot be forwarded).
     - Ticket lifetime is capped at 4 hours.
5. **Enable FAST (Flexible Authentication Secure Tunneling / Kerberos Armoring):**
   - Enable Kerberos Armoring on all Domain Controllers via GPO (`KDC support for claims, compound authentication and Kerberos armoring`).
   - FAST encrypts the initial `AS-REQ` exchange inside a tunnel sealed with the computer account's key, protecting pre-authentication timestamps from interception and cracking even on untrusted networks.
6. **Enforce PAC Signature Validation (KB5008380 / KB5020805):**
   - Verify that all Domain Controllers have PAC signature validation and ticket checksum enforcement set to full Enforcement Mode, preventing PAC modification and identity spoofing across the directory.

9. Authoritative Specifications, RFCs & Cross-Vault Links

9.1 Authoritative Specifications

- **RFC 4120:** *The Kerberos Network Authentication Service (V5)* — The foundational specification of the protocol architecture, message types, and exchange sequences.
- **RFC 3961 / RFC 3962:** *Encryption and Checksum Specifications for Kerberos 5 & AES Encryption* — Defines key derivation functions, PBKDF2 parameters, and AES cipher modes.
- **RFC 4556:** *Public Key Cryptography for Initial Authentication in Kerberos (PKINIT)* — Protocol specification for certificate-based pre-authentication.
- **RFC 6113:** *A Generalized Framework for Kerberos Pre-Authentication (FAST)* — Specification for Kerberos armoring tunnels.

- **[MS-KILE]:** *Kerberos Protocol Extensions* — Microsoft documentation detailing Windows-specific Kerberos behaviors, logon session management, and cipher negotiations.
- **[MS-PAC]:** *Privilege Attribute Certificate Data Structure* — Official specification defining the structure, buffer formats, and cryptographic signature algorithms of the PAC.
- **[MS-SFU]:** *Kerberos Protocol Extensions: Service for User and Constrained Delegation Protocol* — Formal documentation of S4U2Self and S4U2Proxy extensions.

NTLM Authentication — The Fallback Protocol

> **Architectural Mission:** Master the internal mechanics of the NT LAN Manager (NTLM) authentication protocol suite and Microsoft's Challenge-Response mechanism. Understand why IP-based connections trigger automatic Kerberos fallback, how Type 1, Type 2, and Type 3 messages operate on the wire, why Netlogon secure channels are mandatory for domain member verification, and how cryptographic binding deficiencies expose NTLM to relay attacks despite modern integrity checks.

1. Architectural Foundations & The Core Dilemma

1.1 The Inevitability of NTLM: Why Kerberos Fails Over Raw IP

In modern Active Directory (AD) networks, Kerberos Version 5 is the primary, cryptographically superior authentication protocol. However, Kerberos possesses a strict architectural prerequisite: **it is fundamentally reliant on name resolution and Service Principal Names (SPNs)**.

Consider an everyday enterprise scenario: An engineer named **Ahmed** sits at workstation `WKSTN01.corp.local`. Ahmed needs to open a deployment log located on a member server in the server subnet. Instead of typing the server's Fully Qualified Domain Name (FQDN) `\\srv01.corp.local\share\deployment.log`, Ahmed opens Windows Explorer and types the direct IP address: `\\192.168.1.50\share\deployment.log`

The following sequence immediately plays out within the Windows Security Support Provider Interface (SSPI):

1. The Windows operating system recognizes an outgoing Server Message Block (SMB) file-sharing connection.
2. The SMB client invokes the Negotiate Security Support Provider (`negoexts.dll` / `spnego.dll`) to determine which authentication protocol to use.
3. The Kerberos Security Support Provider (`kerberos.dll`) attempts to construct a Service Principal Name for the target. To request a Service Ticket from the Key Distribution Center (KDC), Kerberos requires an SPN formatted as `service/host` (e.g., `cifs/srv01.corp.local`).
4. **The Friction Point:** Active Directory does not support SPNs bound to raw IP addresses (e.g., `cifs/192.168.1.50` is not a valid, registrable SPN in the forest database). Furthermore, the client cannot safely infer which

Active Directory computer account owns `192.168.1.50` without a trusted reverse DNS lookup, which Kerberos intentionally refuses to trust for ticket requests to prevent spoofing.

5. **The Fallback:** Because Kerberos cannot identify or locate a target SPN, the Negotiate package immediately falls back to the **NTLM Security Support Provider (NTLMSSP)**.

This architectural reality explains why NTLM remains active in virtually every enterprise:

- **Raw IP Address Connections:** Management tools, scripts, and administrative shortcuts that reference IP addresses instead of FQDNs immediately bypass Kerberos.
- **Non-Domain-Joined Endpoints & Workgroups:** Systems that cannot reach a Domain Controller (DC) or do not belong to the Kerberos realm cannot request Ticket Granting Tickets (TGTs).
- **Legacy Applications:** Older custom line-of-business software lacking SPNEGO (Simple and Protected GSS-API Negotiation Mechanism) support.
- **Local Authentication:** Logging into a workstation using a local computer account (e.g., `WKSTN01\localadmin`) rather than a domain account always executes locally via NTLM.

1.2 The Challenge-Response Paradigm

Unlike Kerberos, which relies on a centralized Trusted Third Party (the KDC) issuing encrypted tickets, NTLM is a **Challenge-Response** protocol.

The core philosophy of Challenge-Response is straightforward:

- The client and server (or the client and the Domain Controller) share a secret: the user's password hash.
- The client wants to prove to the server that it knows the password without transmitting the plaintext password or the raw password hash across the network wire.
- The server generates an unpredictable, one-time random number called a **Challenge (Server Nonce)** and hands it to the client.
- The client mathematically combines this Challenge with a cryptographic key derived from its password, producing a **Response**.
- The server (or the Domain Controller acting on its behalf) computes the exact same mathematical function. If the server's calculation matches the client's Response, the client has proved knowledge of the secret, and access is granted.

1.3 Terminology & Core Primitives (Zero-Acronym Onboarding)

To understand the wire interactions, every acronym and component must be defined from first principles:

- **NTLM (NT LAN Manager):** Microsoft's proprietary challenge-response authentication protocol family, comprising legacy LM (LAN Manager), NTLMv1, and modern NTLMv2.
- **NTLMSSP (NTLM Security Support Provider):** The dynamic link library (`msv1_0.dll` / `secur32.dll`) implementing the NTLM protocol underneath the Windows SSPI architecture.
- **SSPI (Security Support Provider Interface):** The Windows kernel and user-mode abstraction layer that allows applications (like SMB, HTTP, or RPC) to request authentication without knowing the internal mathematics of Kerberos or NTLM.
- **SPNEGO (Simple and Protected GSS-API Negotiation Mechanism):** An RFC 4178 pseudo-mechanism used by client and server to negotiate whether to speak Kerberos or NTLM.
- **Nonce (Number Used Once):** A cryptographically random value generated for a single authentication session to prevent replay attacks.
- **LM Hash (LAN Manager Hash):** A legacy, cryptographically broken password hash built using DES (Data Encryption Standard) and split 7-byte password chunks. Deprecated and disabled by default since Windows Vista.
- **NT Hash (NTLM Hash):** The standard password hash in Active Directory, computed as $\text{MD4}(\text{UTF-16LE}(\text{Password}))$. It is 16 bytes long and stored in `ntds.dit` on Domain Controllers and in the SAM (Security Account Manager) on local systems.
- **Net-NTLM Hash (NTLM Challenge-Response):** The temporary cryptographic response transmitted across the network during an NTLM authentication exchange. It is *not* the stored NT hash; it is the mathematical product of the NT hash, the server nonce, a client nonce, and a timestamp.
- **AV Pairs (Attribute-Value Pairs):** Structured metadata fields enclosed inside the NTLM Type 2 Challenge and Type 3 Authenticate messages detailing target server names, timestamps, domain boundaries, and security policies.
- **MIC (Message Integrity Check):** A cryptographic HMAC-MD5 checksum incorporated into modern NTLMv2 Type 3 messages to guarantee that none of the three NTLM messages were modified in transit by a man-in-the-middle.
- **EPA (Extended Protection for Authentication):** A security mechanism that cryptographically binds the NTLM authentication exchange to an outer TLS (Transport Layer Security) session using Channel Binding Tokens (CBT).
- **CBT (Channel Binding Token):** A SHA-256 hash of the target server's TLS certificate, embedded inside the NTLM exchange to prevent cross-protocol relay into encrypted services like HTTPS or LDAPS.
- **Netlogon Secure Channel:** An encrypted RPC (Remote Procedure Call) pipe maintained continuously between every domain member server and a

Domain Controller, used to pass NTLM challenges and responses to the DC for validation.

2. The NTLM Wire Protocol & Domain Passthrough

2.1 The Enterprise Scenario Setup

To trace the wire protocol, follow Ahmed's connection to the IP-based file share:

- **Client Workstation:** `WKSTN01.corp.local` (IP: `192.168.1.100`, User: `CORP\ahmed`).
- **Application Server:** `SRV01.corp.local` (IP: `192.168.1.50`, running SMB service).
- **Domain Controller:** `DC01.corp.local` (IP: `192.168.1.10`, holding the `CORP.LOCAL` domain directory `ntds.dit`).

Because Ahmed connects to `\\192.168.1.50`, Kerberos fails, and `WKSTN01` initiates an NTLM three-message exchange over TCP port 445 (SMB).

2.2 Diagram: The NTLMv2 3-Message Exchange & Netlogon Passthrough



2.3 Message Breakdown Step-by-Step

Step 1: Type 1 Message (`NEGOTIATE_MESSAGE`)

Ahmed’s workstation initiates the handshake by sending an NTLM Type 1 message embedded within the initial SMB2 Session Setup request.

- **Header Signature:** Fixed 8-byte ASCII string: `NTLMSSP\0` (`0×4E 0×54 0×4C 0×4D 0×53 0×53 0×50 0×00`).
- **Message Type:** `0×00000001` (Type 1 Negotiate).
- **Negotiate Flags (`NegotiateFlags`):** A 32-bit bitmask declaring the client's capabilities. Vital flags include:
  - `NTLMSSP_NEGOTIATE_UNICODE` (`0×00000001`): Indicates the client supports UTF-16LE character encoding.
  - `NTLMSSP_REQUEST_TARGET` (`0×00000004`): Requests that the server return its domain/server name in the response.
  - `NTLMSSP_NEGOTIATE_SIGN` (`0×00000010`): Requests message signing support.
  - `NTLMSSP_NEGOTIATE_SEAL` (`0×00000020`): Requests payload encryption (sealing) support.
  - `NTLMSSP_NEGOTIATE_EXTENDED_SESSIONSECURITY` (`0×00080000`): Informs the server that the client demands NTLMv2 session security.
  - `NTLMSSP_NEGOTIATE_128` (`0×20000000`): Demands 128-bit session encryption keys.
- **Client OS Version:** Major, minor, and build version of the client Windows operating system.

Step 2: Type 2 Message (`CHALLENGE_MESSAGE`)
The application server `SRV01` responds with an NTLM Type 2 message containing the challenge.

- **Header Signature:** `NTLMSSP\0`.
- **Message Type:** `0×00000002` (Type 2 Challenge).
- **Server Challenge Nonce:** A cryptographically random **8-byte array** generated on the fly by `SRV01`'s local security authority (e.g., `0×3A 0×7F 0×11 0×9B 0×CC 0×44 0×82 0×1E`). This is the raw challenge that the client must encrypt.
- **Target Name:** The NetBIOS name of the domain or server (e.g., `CORP`).
- **Target Information (`TargetInfo`):** A sequence of structured **Attribute-Value (AV) Pairs** formatted as `[AvId (2 bytes), AvLen (2 bytes), Value (AvLen bytes)]`. Key AV Pairs include:
  - `MsvAvNbDomainName` (`0×0002`): Domain NetBIOS name (`CORP`).
  - `MsvAvNbComputerName` (`0×0001`): Server NetBIOS name (`SRV01`).
  - `MsvAvDnsDomainName` (`0×0004`): Domain FQDN (`corp.local`).
  - `MsvAvDnsComputerName` (`0×0003`): Server FQDN (`srv01.corp.local`).
  - `MsvAvTimestamp` (`0×0007`): Current server time in 64-bit Windows FILETIME format.
  - `MsvAvFlags` (`0×0006`): Flags indicating policy requirements. A value of `0×00000002` (`MIC_PRESENT`) mandates that the client include a Message Integrity Check in Type 3.
  - `MsvAvChannelBindings` (`0×000A`): Placeholder for Channel Binding Tokens (CBT) if TLS is involved.
  - `MsvAvEOL` (`0×0000`): End of list delimiter.

Step 3: Type 3 Message (`AUTHENTICATE_MESSAGE`)
Upon receiving the 8-byte Server Challenge and TargetInfo, `WKSTN01` computes the cryptographic response and packages it into the Type 3 message.

- **Header Signature:** `NTLMSSP\0`.
- **Message Type:** `0×00000003` (Type 3 Authenticate).
- **User and Domain Identification:** Plaintext UTF-16 strings specifying `UserName` (`ahmed`), `DomainName` (`CORP`), and calling `Workstation` (`WKSTN01`).
- **The NTLMv2 Response (`NtChallengeResponse`):** Contains the cryptographic proof of password possession (detailed mathematically in Section 3). It consists of:
  - `NTProofStr` (16 bytes): HMAC-MD5 computed over the Server Challenge and the Client Blob.
  - `Blob` (Variable length): A structured byte array containing a client-generated 8-byte random nonce, the timestamp, and a copy of the server's TargetInfo AV Pairs.
- **Message Integrity Check (MIC):** A 16-byte HMAC-MD5 digest calculated across all three messages (Type 1, Type 2, and Type 3) using the derived session key.
- **Encrypted Session Key:** The user session key encrypted with the master key exchange key, allowing `SRV01` to derive session signing and sealing keys.

2.4 The Critical Role of Netlogon Passthrough Authentication

At Step 3, `SRV01` holds Ahmed's NTLMv2 Response. However, an architectural dilemma emerges: **`SRV01` is merely a member server. It does not store domain user passwords, nor does it possess Ahmed's NT hash.**

`SRV01` cannot verify Ahmed's `NTProofStr` locally. To validate the response, `SRV01` must execute **Passthrough Authentication**:

1. **The Netlogon Secure Channel:** During domain join, every Windows member computer establishes a machine account relationship with the domain. Member servers maintain an authenticated, encrypted Remote Procedure Call (RPC) pipe over the **Netlogon Remote Protocol (MS-NRPC)** connected directly to a Domain Controller.
2. **NetrLogonSamLogonEx Execution:** `SRV01` packages Ahmed's username, domain, the original 8-byte Server Challenge, and Ahmed's full NTLMv2 Response into an RPC function call named `NetrLogonSamLogonEx` (or `NetrLogonSamLogon`) and sends it over the Netlogon pipe to `DC01`.
3. **Domain Controller Verification:**
   - `DC01` queries its Active Directory database (`ntds.dit`) for `CORP\ahmed`.
   - It retrieves Ahmed's stored NT password hash.
   - Using this hash and the data supplied by `SRV01`, `DC01` computes the expected NTLMv2 response.

- If the computed response matches the response Ahmed sent to `SRV01`, `DC01` confirms that Ahmed knows the password.

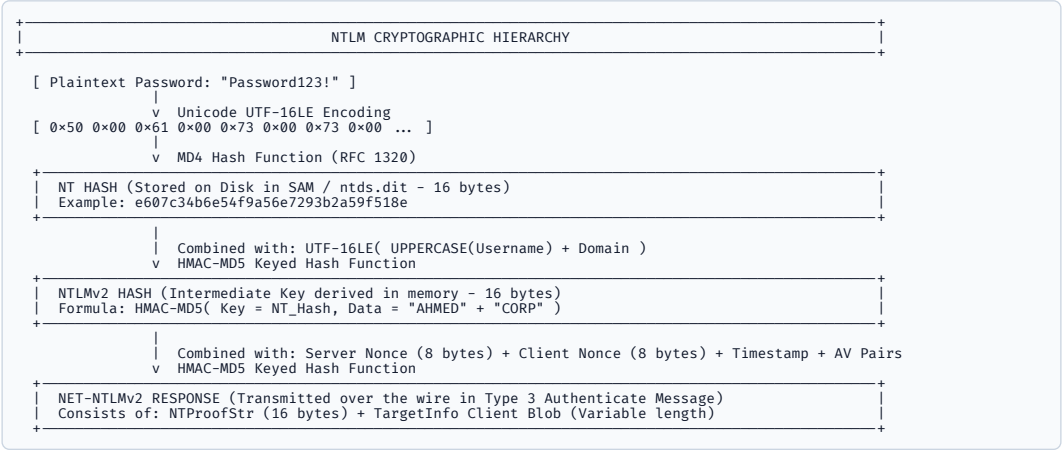
4. **Token Generation & Return:** `DC01` compiles Ahmed's group memberships, user SIDs, and primary group information (the exact same `KERB_VALIDATION_INFO` data found in a Kerberos PAC) and transmits it back to `SRV01` over the Netlogon channel alongside status code `STATUS_SUCCESS` (`0×00000000`).
5. **Local Access Token Creation:** `SRV01` receives the validation info from the DC, instantiates a local Windows Security Access Token for Ahmed, verifies the MIC, evaluates the NTFS file share permissions, and grants access to `deployment.log`.

3. Cryptographic Deep-Dive: NTLM Mathematics

3.1 The Three Generations of Hashes

NTLM authentication involves two completely distinct cryptographic concepts that are frequently confused:

1. **The Stored Hash (Permanent Secret):** The password derivative stored at rest on disk in SAM or `ntds.dit`.
2. **The Net-NTLM Challenge-Response (Ephemeral Proof):** The ephemeral value computed dynamically on the wire during authentication.



3.2 The NTLMv2 Mathematical Derivation

The exact mathematical formulas for computing NTLMv2 authentication on modern Windows systems are defined in specification **[MS-NLMP] §3.3.2**:

Step 1: Compute the NT Hash

$$\text{NT_Hash} = \text{MD4}(\text{UTF-16LE}(\text{Password}))$$

Properties: 16 bytes (128 bits). Static, un-salted, never changes unless the user changes their password.

Step 2: Compute the NTLMv2 Hash

$$\text{NTLMv2_Hash} = \text{HMAC-MD5}(\text{Key} = \text{NT_Hash},$$
$$\text{Data} = \text{UTF-16LE}(\text{UPPERCASE}(\text{Username}) + \text{Domain}))$$

Properties: 16 bytes (128 bits). Binds the password hash to the specific user name and domain realm. Notice that because the username and domain act as a salt, the NTLMv2 hash differs across different domains even if the password is identical.

Step 3: Construct the Client Blob (`temp`)
The client constructs a variable-length byte array containing session metadata:

$$\text{Blob} = \text{Header} \,||\, \text{Reserved} \,||\, \text{Timestamp} \,||\, \text{ClientNonce}$$
$$||\, \text{Reserved} \,||\, \text{TargetInfo} \,||\, \text{ZeroTerminator}$$

- `Header`: 4 bytes (`0×01010000`).
- `Reserved`: 4 bytes (`0×00000000`).
- `Timestamp`: 8 bytes (Current Windows FILETIME).
- `ClientNonce`: 8 bytes of cryptographically random data generated by the client.
- `TargetInfo`: The complete array of AV Pairs received from the server's Type 2 message.

Step 4: Compute the `NTProofStr`
The cryptographic proof transmitted in the Type 3 message is generated by keying HMAC-MD5 with the NTLMv2 Hash over the concatenated Server Challenge and Client Blob:

$$\text{NTLMv2_Hash} = \text{HMAC-MD5}(\text{Key} = \text{NT_Hash},$$
$$\text{Data} = \text{UTF-16LE}(\text{UPPERCASE}(\text{Username}) + \text{Domain}))$$

Step 5: Construct the Wire Response
The final `NtChallengeResponse` field sent inside the Type 3 message is:

$$\text{NtChallengeResponse} = \text{NTProofStr} \quad | \quad \text{Blob}$$

3.3 The Net-NTLM Hash Structure (Offline Cracking Format)

When security tooling captures an NTLM Type 3 message from network traffic, it extracts these values and formats them into a standardized hash format recognized by password analysis tools (Hashcat mode `5600`):

$$\text{Username} :: \text{Domain} : \text{ServerChallenge} : \text{NTProofStr} : \text{Blob}$$

Why Net-NTLMv2 is Resistant to Rainbow Tables: Unlike legacy NTLMv1, which encrypted a fixed 8-byte challenge with raw 7-byte DES keys derived directly from the NT hash, NTLMv2 incorporates the 8-byte **Server Challenge**, an 8-byte **Client Challenge**, and a 64-bit microsecond **Timestamp** inside the hashed data. Because the inputs change dynamically for every single authentication attempt, precomputed rainbow tables are mathematically impossible. To recover the plain-text password, an adversary must execute an iterative dictionary attack computing Steps 1 through 4 for every candidate password.

4. Demystifying NTLM Relay, MIC, and Channel Binding

4.1 The Fundamental Flaw: Decoupling of Authentication and Channel

Why is NTLMv2 vulnerable to relay attacks if its mathematics are modern and resilient against rainbow tables?

The Root Architectural Vulnerability: In standard NTLMv2, the authentication exchange is **cryptographically uncoupled from the network connection over which it travels**. When Ahmed proves knowledge of his password to `192.168.1.50`, the resulting mathematical proof (`NTProofStr`) proves only that:

> *"The user possessing Ahmed's password hash verified Server Challenge X using Client Nonce Y at Time Z."*

The proof does **not** cryptographically prove *which application server* Ahmed intended to talk to, nor does it bind the authentication to the underlying TCP transport layer.

The Man-in-the-Middle Relay Walkthrough:

1. An attacker positions themselves between Ahmed and a target service (or coerces Ahmed's machine to connect to an attacker-controlled endpoint).
2. Ahmed sends an NTLM Type 1 Negotiate message to the attacker.
3. The attacker forwards that Type 1 message to a high-value victim server (e.g., a Domain Controller or database).
4. The victim server generates a legitimate 8-byte Server Challenge and returns a Type 2 message to the attacker.
5. The attacker forwards this exact Type 2 message back to Ahmed.
6. Ahmed, believing he is speaking to a legitimate server, computes a mathematically valid NTLMv2 response using the challenge provided by the victim server, and returns a Type 3 message to the attacker.
7. The attacker forwards this Type 3 Authenticate message to the victim server.
8. The victim server checks the response over its Netlogon secure channel with the DC. The DC validates the challenge and response, confirms the match, and **the attacker is granted an authenticated administrative session as Ahmed on the victim server.**

Ahmed never intended to authenticate to the victim server, yet the victim server accepted his credentials because the mathematical proof was perfectly valid for that challenge.

4.2 The Defense Mechanisms: MIC vs EPA/CBT

To counter NTLM relay attacks, Microsoft engineered two successive layers of defense:

1. Message Integrity Check (MIC)

Introduced under security update **MS08-068** and formalized in **[MS-NLMP]**:

- **The Concept:** A 16-byte HMAC-MD5 signature computed over the concatenated byte streams of all three messages:

$$\text{MIC} = \text{HMAC-MD5}(\text{Key} = \text{ExportedSessionKey},\ \ \text{Data} = \text{Type1} \ \mid\ \text{Type2} \ \mid\ \text{Type3})$$

- **The Intended Protection:** If an attacker intercepts the messages and attempts to tamper with the flags (e.g., removing signing requirements or altering server names), the MIC verification fails on the receiving server because the attacker does not possess the `ExportedSessionKey` required to recalculate the MIC.
- **The Friction Point & Historical Bypasses:** For years, NTLM relay attacks bypassed MIC verification through architectural loopholes:
  - *MIC Dropping (CVE-2019-1040):* If an attacker simply stripped the MIC field from the Type 3 message and flipped the `MIC_PRESENT` flag in the AV Pairs from `0×02` to `0×00`, unpatched servers accepted the downgraded message without verifying the signature.
  - *Same-Protocol Limitations:* Even with a valid MIC, if an attacker relays an authentication exchange between two servers without modifying any fields within the NTLM messages (a pure transparent byte-stream relay), the MIC remains cryptographically valid. MIC prevents message tampering; it does not solve cross-target relaying.

2. Extended Protection for Authentication (EPA) & Channel Binding Tokens (CBT)

To permanently eliminate relaying into encrypted services (such as HTTPS or LDAPS), Microsoft introduced **Extended Protection for Authentication (EPA)**:

- **The Concept:** Cryptographically binding the outer TLS tunnel to the inner NTLM authentication exchange.
- **The Mechanism (Channel Binding Tokens):** When a client establishes an encrypted TLS session with a secure server (e.g., `https://adfs.corp.local` or `ldaps://dc01.corp.local`):
  1. The client retrieves the server's public X.509 certificate presented during the TLS handshake.
  2. The client computes a SHA-256 hash of this server certificate: this hash is the **Channel Binding Token (CBT)**.
  3. When constructing the NTLM Type 3 message, the client places this CBT hash inside the `MsvAvChannelBindings` (`0×000A`) AV Pair within the `TargetInfo` blob.
  4. The client computes `NTProofStr` over this blob.
- **Why EPA Stops NTLM Relay:** If an adversary attempts to intercept Ahmed's authentication and relay it to an LDAPS service on a Domain Controller:

- The attacker has established a separate TLS session with the DC using the DC's certificate.
- However, Ahmed computed his CBT based on the certificate of the service *he* thought he was accessing.
- When the DC unseals the Type 3 message, it compares the CBT hash inside Ahmed's `MsvAvChannelBindings` against the SHA-256 hash of its own TLS certificate.
- The hashes mismatch. The DC immediately rejects the authentication with `SEC_E_BAD_BINDINGS` (`0×8009033f`), completely neutralizing the relay attack.

5. Architectural Friction Points & Vulnerability Foundations

5.1 Pass-the-Hash: The Permanent Key Equivalence

- **The Design Decision:** In NTLM, the user's password is never transmitted across the network. The client uses the 16-byte **NT Hash** as the master cryptographic root to compute challenge responses.
- **The Friction Point:** Because the NT hash is the *only* input required to generate NTLMv1, NTLMv2, and session keys, **the NT hash is mathematically equivalent to the plaintext password** for all NTLM authentication purposes.
- **The Architectural Consequence:** An adversary who dumps the local SAM database of a workstation or extracts `ntds.dit` does not need to crack the password to plaintext. By loading the 16-byte NT hash into memory (via LSASS injection or custom protocol libraries), the adversary can authenticate across the network to any service accepting NTLM, fully impersonating the user.

5.2 Authentication Coercion: Weaponizing Asymmetric Network Trust

- **The Design Decision:** Windows systems feature numerous administrative RPC interfaces (such as print spooling, filesystem replication, and certificate enrollment) that permit authorized clients to instruct a remote server to interact with other network shares.
- **The Friction Point:** When a remote Windows server is instructed to access a UNC path (e.g., `\\attacker-ip\share`), the server's local machine account (`COMPUTER$`) automatically attempts to authenticate to that UNC path using NTLM over SMB.
- **The Architectural Consequence:** An adversary on the local network invokes a built-in, unauthenticated or low-privileged RPC interface on a Domain Controller (e.g., the Print System Remote Protocol `MS-RPRN` or the Encrypting File System Remote Protocol `MS-EFSR`). The adversary forces the DC

machine account to connect outbound to an attacker listener. The attacker captures the DC's incoming Type 3 authentication and relays it to an unhardened service (such as Active Directory Certificate Services Web Enrollment or LDAP without channel binding), immediately achieving full domain privilege escalation.

5.3 SMB Signing Negotiation & The Insecure Default

- **The Design Decision:** SMB Signing uses the session key negotiated during authentication to digitally sign every SMB packet, guaranteeing that messages cannot be tampered with or relayed mid-connection.
- **The Friction Point:** For decades, Microsoft disabled SMB Signing by default on standard client workstations and member servers to minimize CPU overhead on older hardware. It was enforced strictly on Domain Controllers.
- **The Architectural Consequence:** Because member servers accepted unsigned SMB sessions, any NTLM authentication captured from an administrative user could be relayed directly to any member server in the domain, allowing adversaries to execute remote service creation or command execution without ever cracking a password.

6. Defensive Telemetry, Detection Engineering & Hardening

6.1 Windows Security Event Telemetry

NTLM authentication operations generate specific event records on endpoints and Domain Controllers:

Event ID	Log Name	Operational Meaning	Strategic Telemetry Significance
4624	Security	An account was successfully logged on.	**Type 3 (Network) Logons:** Look at the `Authentication Package` field. If it reads `NTLM`, inspect `LM Package Name` (e.g., `NTLM V2`) and `Key Length`. Identifies which endpoints are still relying on NTLM fallback.
4776	Security	The computer attempted to validate the credentials for an account.	**Logged Exclusively on Domain Controllers:** Generated every time a DC validates an NTLM challenge-response via Netlogon passthrough authentication. Key fields: `PackageName:___V1 _0`, `TargetUserName`, and `Workstation`.
8001	Operational	NTLM Auditing: Incoming NTLM traffic.	Generated under `Microsoft-Windows -NTLM/Operational` when NTLM

Event ID	Log Name	Operational Meaning	Strategic Telemetry Significance
			auditing policies are active. Reveals calling process and source IP.
8002	Operational	NTLM Auditing: NTLM traffic to domain accounts.	Critical for planning domain-wide NTLM deprecation without breaking legacy applications.

6.2 Detection Heuristics & Hunting Logic

1. Coerced NTLM Authentication Anomaly (KQL):

Detects high-value computer accounts (Domain Controllers) performing outbound NTLM network logons to unmanaged IP ranges or unusual endpoints:

```
SecurityEvent
| where EventID == 4624
| where LogonType == 3 // Network Logon
| where AuthenticationPackageName == "NTLM"
| where TargetUserName endswith "$" // Computer accounts
| where TargetUserName in ("DC01$", "DC02$", "RODC01$") // Monitor Tier-0 systems
| summarize count(), DistinctIPs = make_set(IpAddress) by TargetUserName, bin(TimeGenerated, 15m)
| project TimeGenerated, TargetUserName, count_, DistinctIPs
```

2. NTLMv1 Usage Detection (KQL):

Flags dangerous legacy NTLMv1 or LM authentications that can be cracked offline in hours:

```
SecurityEvent
| where EventID == 4624
| where AuthenticationPackageName == "NTLM"
| where LmPackageName in ("NTLM V1", "LM")
| project TimeGenerated, TargetUserName, WorkstationName, IpAddress, LmPackageName
```

6.3 Hardening Blueprint & Migration to Zero-NTLM

Securing Active Directory against NTLM exploitation requires a phased, architectural eradication strategy:

1. **Enforce SMB Signing Universally:**
   - Configure GPO: `Computer Configuration → Windows Settings → Security Settings → Local Policies → Security Options`:
     - `Microsoft network client: Digitally sign communications (always)` → **Enabled**
     - `Microsoft network server: Digitally sign communications (always)` → **Enabled**
   - *Impact:* Completely neutralizes SMB-to-SMB NTLM relay attacks across all domain systems.
2. **Enforce LDAP Signing & Channel Binding (CBT):**
   - On all Domain Controllers, enforce LDAP server signing requirements via GPO:
     - `Domain controller: LDAP server signing requirements` → **Require signing**
   - Enforce Channel Binding on LDAPS via registry key:

        - `HKLM\System\CurrentControlSet\Services\NTDS\Parameters\LdapEnforceChannelBinding = 2` (Enforced).
    - *Impact:* Neutralizes cross-protocol NTLM relay into Active Directory LDAP/LDAPS.
3. **Enforce Extended Protection for Authentication (EPA) on Web Services:**
    - In Internet Information Services (IIS), Active Directory Federation Services (AD FS), and Active Directory Certificate Services (AD CS), enable EPA for all HTTP endpoints. Set `Extended Protection` to **Required**.
    - *Impact:* Prevents relaying NTLM credentials into administrative web enrollment interfaces (e.g., `certsrv`).
4. **Audit and Restrict NTLM via Group Policy:**
    - *Phase 1 (Audit Mode):* Set `Network security: Restrict NTLM: Audit Incoming NTLM Traffic` to **Enable auditing for domain accounts**. Review Event 8002 in SIEM to identify legacy applications requiring remediation.
    - *Phase 2 (Restrict Fallback):* Set `Network security: Restrict NTLM: Outgoing NTLM traffic to remote servers` to **Audit All** → transition to **Deny All**.
    - *Phase 3 (Full Deprecation):* Set `Network security: Restrict NTLM: NTLM authentication in this domain` to **Deny for domain accounts**.
5. **Mitigate Coercion Primitives:**
    - Disable the Print Spooler service (`spoolsv.exe`) on all Domain Controllers:

```
Stop-Service -Name Spooler -Force
Set-Service -Name Spooler -StartupType Disabled
```

- Block unauthenticated or cross-boundary RPC coercion interfaces via RPC Filtering policies.

7. Authoritative Specifications, RFCs & Cross-Vault Links

7.1 Authoritative Specifications

- **[MS-NLMP]:** *NT LAN Manager (NTLM) Authentication Protocol* — Official Microsoft specification documenting Type 1, Type 2, and Type 3 message formats, AV Pairs, and NTLMv2 derivation mathematics.
- **[MS-NRPC]:** *Netlogon Remote Protocol* — Formal specification for domain passthrough authentication, `NetrLogonSamLogonEx`, and secure channel establishment.
- **RFC 4178:** *The Simple and Protected Generic Security Service Application Program Interface (GSS-API) Negotiation Mechanism (SPNEGO)* — Documents protocol negotiation between Kerberos and NTLM.

- **RFC 5929:** *Channel Bindings for TLS* — Specification defining Channel Binding Tokens (CBT) and `tls-server-end-point` certificate hashing.

LDAP Protocol — Directory Access Backbone & Relay Attack Surface

> **Architectural Mission:** Master the internal mechanics of the Lightweight Directory Access Protocol (LDAP) and its role as the primary query and modification engine for Active Directory Domain Services (AD DS). Understand how directory trees are structured, how ASN.1 BER encoding governs the wire protocol, how the four bind models establish session trust, how complex prefix search filters evaluate directory entries, and why cryptographic signing and Channel Binding Tokens (CBT) represent the definitive line of defense against directory relay attacks.

1. Architectural Foundations & Purpose

1.1 The Database Highway of Active Directory

Every resource, relationship, and credential state in an Active Directory forest resides within a centralized database file named `ntds.dit` (NT Directory Services - Directory Information Tree) managed by the Extensible Storage Engine (ESE, commonly known as the JET database engine).

While protocols like Kerberos handle ticket granting and SMB handles file transmission, **LDAP (Lightweight Directory Access Protocol)** is the universal, standardized wire protocol used by clients, servers, and administrative tools to interrogate and mutate the directory database.

Whenever a system:

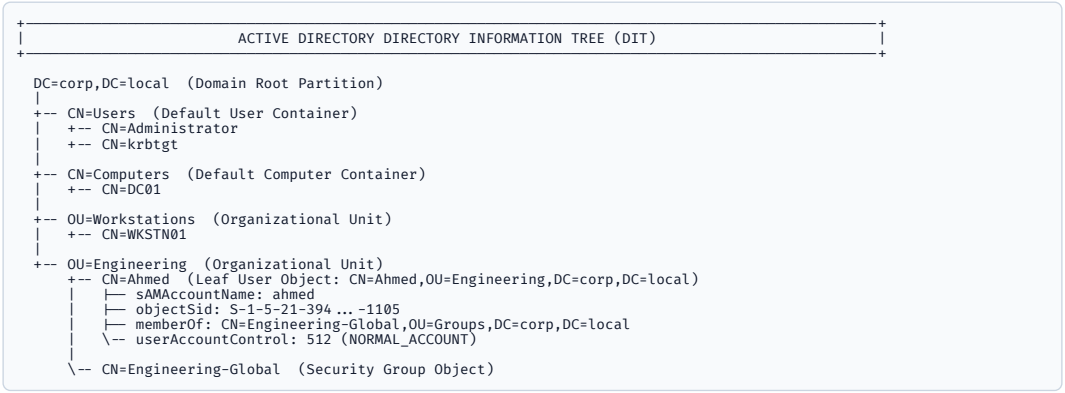
- Looks up a user’s group memberships during logon,
- Resolves an email address to a recipient object,
- Enumerates domain computers for administrative patching,
- Edits a Group Policy Object (GPO), or
- Updates an Access Control List (ACL) on an Organizational Unit (OU),

that operation executes as an **LDAP Request** transmitted over the network to a Domain Controller (DC).

1.2 The Active Directory Directory Information Tree (DIT)

Active Directory organizes information hierarchically according to the X.500 directory model. Every entry in the directory is an **Object** governed by the Active Directory Schema:

- **DN (Distinguished Name):** The unambiguous, unique path identifying an object within the directory hierarchy (e.g., `CN=Ahmed,OU=Engineering,DC=corp,DC=local`).
- **RDN (Relative Distinguished Name):** The leftmost component of a DN that distinguishes an object from its siblings within the same parent container (e.g., `CN=Ahmed`).
- **Object Classes:** Structural templates defining what type of entity the object represents (e.g., `top`, `person`, `organizationalPerson`, `user`, `computer`).
- **Attributes:** Key-value pairs containing specific properties (e.g., `sAMAccountName`, `mail`, `memberOf`, `userAccountControl`, `nTSecurityDescriptor`).



1.3 Terminology & Core Primitives (Zero-Acronym Onboarding)

- **LDAP (Lightweight Directory Access Protocol):** An open, vendor-neutral application protocol (RFC 4511) operating over TCP/UDP port 389 used to query and modify directory services.
- **LDAPS (LDAP over TLS/SSL):** LDAP encapsulated within an encrypted Transport Layer Security (TLS) tunnel, operating over TCP port 636.
- **Global Catalog (GC):** A distributed directory repository running on designated Domain Controllers (listening on TCP port 3268 for cleartext and 3269 for SSL) containing a partial attribute set for every object across all domains in the entire forest.
- **ASN.1 (Abstract Syntax Notation One):** The formal interface definition language used by telecommunication and networking standards to define structured data types independently of computer architecture.
- **BER (Basic Encoding Rules):** The binary encoding standard used by LDAP to serialize ASN.1 messages into Tag-Length-Value (TLV) byte streams over TCP.
- **SASL (Simple Authentication and Security Layer):** A generalized framework (RFC 4422) enabling connection-based protocols to authenticate using pluggable cryptographic mechanisms such as Kerberos (GSS-API) or NTLM.

- **GSS-API (Generic Security Services Application Program Interface):** A standardized security API (RFC 2743) allowing applications to establish an authenticated security context with a remote peer without hardcoding specific authentication algorithms.
- **CBT (Channel Binding Token):** A cryptographic hash of the outer TLS server certificate (defined in RFC 5929) embedded within an inner authentication handshake to prevent relay attacks across different protocol boundaries.
- **OID (Object Identifier):** A globally unique, dot-separated numerical string (e.g., `1.2.840.113556.1.4.803`) assigned by international standards bodies and vendors to uniquely identify directory attribute types, syntaxes, matching rules, and extended controls.

2. Protocol Internals & The Wire Exchange

2.1 The Wire Protocol: ASN.1 BER Structure

Every LDAP communication exchange consists of discrete **LDAPMessage** envelopes serialized using ASN.1 BER encoding. An `LDAPMessage` packet possesses three structural components:

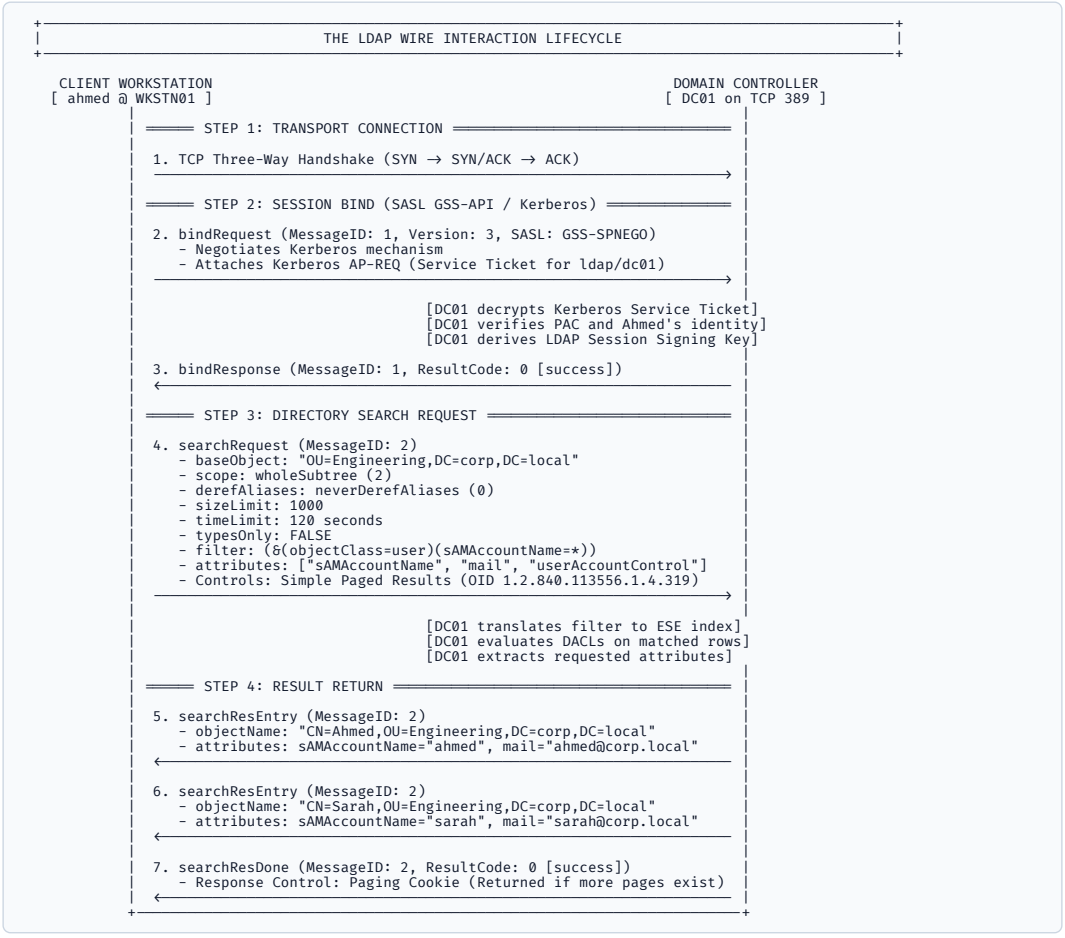
```
LDAPMessage ::= SEQUENCE {
    messageID       MessageID (INTEGER),
    protocolOp      CHOICE {
        bindRequest         BindRequest,
        bindResponse        BindResponse,
        searchRequest       SearchRequest,
        searchResEntry      SearchResultEntry,
        searchResDone       SearchResultDone,
        modifyRequest       ModifyRequest,
        modifyResponse      ModifyResponse,
         ...
    },
    controls        [0] Controls OPTIONAL
}
```

1. **`messageID`:** A 32-bit integer generated by the client to track asynchronous operations. When the server replies, it includes the identical `messageID`, allowing multiple simultaneous queries across a single multiplexed TCP socket.
2. **`protocolOp`:** The specific operation being executed (Bind, Search, Modify, Add, Delete, Extended Operation).
3. **`controls`:** An optional sequence of standardized or vendor-specific flags (represented by OIDs) that alter how the server evaluates the request (such as server-side sorting or paged results).

2.2 Walkthrough: Tracing an LDAP Search on the Wire

Consider our enterprise scenario: Ahmed is logged into `WKSTN01.corp.local`. Ahmed opens a directory administration console or executes a directory query to locate all active domain accounts within the `Engineering` department: `Base DN: OU=Engineering,DC=corp,DC=local Filter: (&(objectClass=user)(sAMAccountName=*))`

The following network interaction occurs between `WKSTN01` (Client) and `DC01` (Domain Controller):



3. The Four LDAP Bind Models Dissected

Before a client can execute a search or modification in Active Directory, it must execute an **LDAP Bind** to establish its identity and security context. Active Directory supports four distinct bind models:

3.1 Anonymous Bind (Unauthenticated Access)

- **The Concept:** The client sends a `bindRequest` with a zero-length DN and a zero-length password, requesting access as `NT AUTHORITY\ANONYMOUS LOGON`.
- **The Active Directory Architectural Reality:**
  - In Windows 2000, Anonymous Bind was enabled by default, allowing anyone on the network to enumerate domain users and password policies without credentials.
  - Beginning with Windows Server 2003, Microsoft disabled anonymous directory queries by default. This control is governed in the Active Directory Configuration partition under:

`CN=Directory Service,CN=Windows NT,CN=Services,CN=Configuration,DC=corp,DC=local` via the **`SeventhDsHeuristic`** string attribute. When this character is `0` or unset, anonymous queries are rejected with error `LDAP_INAPPROPRIATE_AUTH` (`0x30`).

- *Residual Exposure:* Even when anonymous LDAP operations are restricted, unauthenticated clients can still query the root Directory Specific Entry (**RootDSE**) over port 389 to read basic directory metadata, including the forest functional level, naming contexts, supported SASL mechanisms, and the DC's FQDN.

3.2 Simple Bind (Cleartext Credentials)

- **The Concept:** The client sends a `bindRequest` containing the user's plaintext Distinguished Name (`CN=Ahmed,OU=Users,DC=corp,DC=local`) paired with the user's **plaintext password** in cleartext.
- **The Architectural Hazard:** When executed over unencrypted port 389, the user's password travels in raw, unencrypted bytes across the local network segment. Any network tap, span port, or compromised switch can harvest domain credentials in plaintext. Simple Bind should **never** be permitted over port 389; it is acceptable only when encapsulated inside an encrypted LDAPS (TCP 636) TLS tunnel.

3.3 SASL / GSS-API Negotiate Bind (The Enterprise Standard)

- **The Concept:** Standard Windows domain members authenticate to LDAP using **Simple Authentication and Security Layer (SASL)** via the **GSS-API Negotiate** mechanism (SPNEGO).
- **The Protocol Flow:**
  1. The client requests a SASL Bind specifying mechanism `GSS-SPNEGO`.
  2. The client reaches out to the KDC on port 88, requests a Kerberos Service Ticket for the SPN `ldap/DC01.corp.local`, and attaches this ticket to the LDAP bind request.
  3. The Domain Controller decrypts the Service Ticket, validates the client's identity and PAC, and accepts the bind.
  4. **Session Protection:** Both client and DC derive an ephemeral session key from the Kerberos ticket. This key is subsequently used to digitally sign (**LDAP Signing**) or encrypt (**LDAP Sealing**) all subsequent LDAP queries and results, completely protecting the session against tampering and eavesdropping without requiring TLS certificates.

3.4 LDAPS (LDAP over TLS/SSL)

- **The Concept:** An encrypted TLS session negotiated over dedicated TCP port 636 before any LDAP protocol bytes are exchanged.
- **Prerequisites:** To enable LDAPS on a Domain Controller, the DC must possess a valid X.509 machine certificate issued by a trusted enterprise Cer-

tificate Authority (Active Directory Certificate Services - AD CS) containing the `Server Authentication` Extended Key Usage (EKU `1.3.6.1.5.5.7.3.1`).
- **Operational Value:** Provides transport encryption for third-party systems (such as Linux servers, printers, or web applications) that do not support SASL Kerberos binding.

4. Search Filter Syntax, Operators & Matching Rules

4.1 Polish Prefix Notation (RFC 4515)

LDAP search filters do not use standard infix notation (e.g., `A AND B`). Instead, they are constructed using **Polish Prefix Notation**, where the logical operator precedes the operands:

$$\text{Filter} = (\langle\text{operator}\rangle\ (\langle\text{filter}_1\rangle)\ (\langle\text{filter}_2\rangle))$$

Logical Operators:
- **AND (`&`):** All enclosed conditions must evaluate to true.
  - Example: `(&(objectClass=user)(department=Engineering))`
- **OR (`|`):** At least one enclosed condition must evaluate to true.
  - Example: `(|(department=Engineering)(department=Finance))`
- **NOT (`!`):** Inverts the result of the single enclosed condition.
  - Example: `(!(userAccountControl:1.2.840.113556.1.4.803:=2))` (Exclude disabled accounts)

4.2 Matching Rule OIDs (Bitwise Logic & Transitive Search)

Standard LDAP equality checks (`attribute=value`) cannot evaluate bitmask fields or traverse complex recursive directory structures. To solve this, Active Directory implements **Extensible Matching Rules** referenced by specific Object Identifiers (OIDs):

$$\langle\text{attribute}\rangle : \langle\text{matchingRuleOID}\rangle := \langle\text{value}\rangle$$

1. Bitwise AND (`1.2.840.113556.1.4.803` — `LDAP_MATCHING_RULE_BIT_AND`):
Evaluates whether a specific bit is set within an integer bitmask.
- *Primary Use:* Filtering the `userAccountControl` (UAC) attribute.
- *Example (Find accounts with Pre-Authentication disabled):* `(&(objectClass=user)(userAccountControl:1.2.840.113556.1.4.803:=4194304))` *(Because* `DONT_REQ_PREAUTH` *is bit flag* `0x00400000` = `4,194,304` *in decimal).*
- *Example (Find accounts configured with Unconstrained Delegation):* `(&(objectClass=user)(userAccountControl:1.2.840.113556.1.4.803:=524288))` *(Because* `TRUSTED_FOR_DELEGATION` *is bit flag* `0x00080000` = `524,288`*).*

2. Bitwise OR (`1.2.840.113556.1.4.804` — `LDAP_MATCHING_RULE_BIT_OR`):
Evaluates whether *any* of the specified bits are set within an integer bitmask.

3. In-Chain Transitive Matching (`1.2.840.113556.1.4.1941` — `LDAP_MATCHING_RULE_IN_CHAIN`):
Active Directory supports deeply nested group hierarchies: User A is in Group B, Group B is in Group C, and Group C is granted access. A standard equality query for Group C will not return User A.

- *The Solution:* The `LDAP_MATCHING_RULE_IN_CHAIN` rule traverses the entire ancestral tree recursively on the Domain Controller.
- *Example (Find all members of Domain Admins, including nested groups):* `(&(objectClass=user)(memberOf:1.2.840.113556.1.4.1941:=CN=Domain Admins,CN=Users,DC=corp,DC=local))`

5. Directory Limits, Pagination & Extended Controls

5.1 The `MaxPageSize` Bottleneck

Domain Controllers must serve thousands of concurrent clients without exhausting physical memory or bandwidth. To maintain stability, Microsoft enforces hard administrative limits within Active Directory via LDAP query policies (managed under `CN=Default Query Policy,CN=Query-Policies,CN=Directory Service,CN=Windows NT,CN=Services,CN=Configuration,DC=corp,DC=local`).

- **`MaxPageSize` Constraint:** By default, a Domain Controller will return a maximum of **1,000 objects** for any single `searchRequest`.
- **The Problem:** If an administrator or service queries a domain containing 20,000 users with a simple filter like `(objectClass=user)`, the DC returns the first 1,000 records followed by a `searchResDone` packet containing warning code `LDAP_SIZE_LIMIT_EXCEEDED` (`0x04`). The remaining 19,000 objects are discarded!

5.2 Simple Paged Results Control (RFC 2696)

To retrieve large datasets reliably, clients must utilize the **Simple Paged Results Control** (OID `1.2.840.113556.1.4.319`).

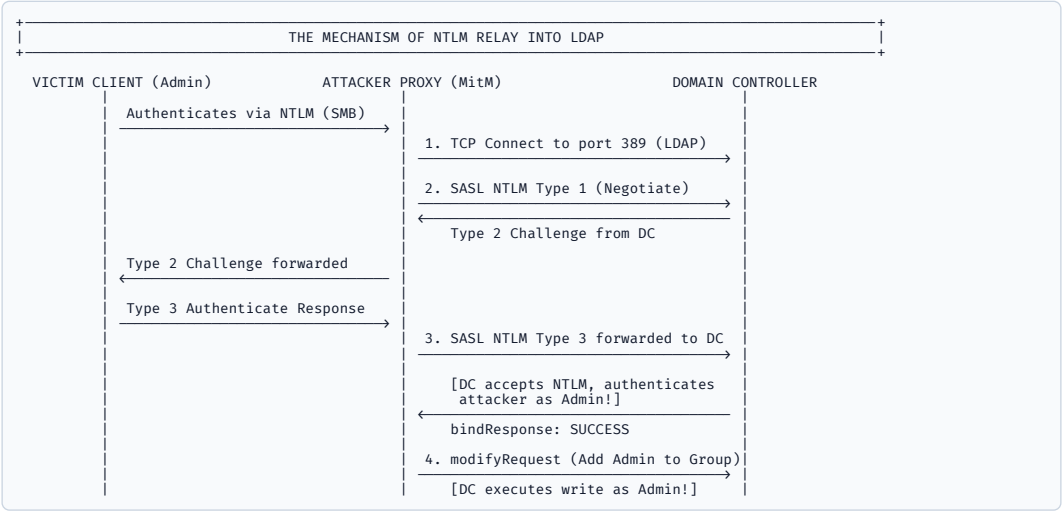
- **The Mechanism:**
  1. The client attaches the paging control to its `searchRequest`, specifying a requested page size (e.g., `size: 500`) and an empty cookie.
  2. The DC returns the first 500 objects, followed by a `searchResDone` message containing an opaque binary **Paging Cookie** generated by the server.
  3. The client issues a new `searchRequest` containing the exact same query, attaching the Paging Cookie received from the previous page.
  4. The DC uses the cookie to resume directory retrieval at the exact row index where the previous page terminated.
  5. The cycle repeats until the DC returns a final page with an empty cookie, signaling completion.

6. Demystifying LDAP Signing & Channel Binding Tokens (CBT)

Security vulnerabilities in directory services frequently stem from how network endpoints protect—or fail to protect—the integrity of their LDAP communication channels.

6.1 The Threat: Unsigned LDAP & Cross-Protocol Relay

Consider an enterprise network where Domain Controllers accept unsigned clear-text LDAP connections over TCP port 389:



1. **The Relay Vector:** An adversary intercepts an incoming NTLM authentication from a privileged workstation (or triggers it via authentication coercion).
2. **The LDAP Injection:** The attacker forwards this authentication to the Domain Controller on TCP port 389 as a SASL NTLM Bind.
3. **The Unsigned Write:** Because LDAP on port 389 does not enforce digital signatures by default, the attacker is granted an authenticated directory session. The attacker immediately transmits an LDAP `modifyRequest` to elevate permissions—such as modifying `nTSecurityDescriptor` or adding an account to `Domain Admins`.

6.2 The Defenses: LDAP Signing vs Channel Binding Tokens

1. LDAP Signing (Integrity on Port 389):

- **Mechanism:** During SASL negotiation (Kerberos or NTLM), the client and server negotiate session security. Every subsequent LDAP request and response is signed with a cryptographic HMAC using the session key.
- **Why It Stops Relays:** When an attacker relays an NTLM authentication, the attacker does not possess the `ExportedSessionKey` held in the victim's memory. If LDAP Signing is enforced on the Domain Controller, any subsequent

`modifyRequest` sent by the attacker without a valid cryptographic signature is immediately rejected with `LDAP_STRONG_AUTH_REQUIRED` (`0×08`).

2. Channel Binding Tokens (CBT on Port 636 / LDAPS):

- **The Problem with LDAPS Relaying:** If an attacker attempts to relay NTLM into secure LDAPS (port 636), the attacker must establish their own TLS session with the DC.
- **The CBT Defense:** With Channel Binding enforced, the client computes a SHA-256 hash of the server's TLS certificate (`tls-server-end-point`) and embeds it within the NTLM Type 3 message. If the authentication was relayed from a different connection, the DC detects that the certificate hash inside the authentication payload does not match its own TLS certificate, and aborts the connection.

7. Architectural Vulnerability Primitives (Zero Tool Dumping)

Security vulnerabilities in Active Directory LDAP arise directly from deliberate architectural conveniences embedded in the directory service design:

7.1 Broad Default Read Permissions (`Authenticated Users`)

- **Design Philosophy:** Active Directory was engineered to allow employees to look up colleagues, resolve email addresses, find file shares, and read Group Policy Objects effortlessly.
- **The Friction Point:** By default, the special identity `NT AUTHORITY\Authenticated Users` possesses `READ_PROP` permissions over almost all attributes of all objects within the Domain Naming Context.
- **The Consequence:** Any compromised user account—even an unprivileged contractor or compromised workstation machine account—can query LDAP to dump the entire directory structure: all users, computers, service accounts, SPNs, group memberships, and security descriptors.

7.2 Directory Modification as the Universal Escalation Highway

- **The Friction Point:** All permissions and structural attributes in Active Directory are updated through LDAP `modifyRequest` packets.
- **Critical Target Attributes:**
  - `nTSecurityDescriptor`: Controls the Access Control List of an object. Gaining write access allows an adversary to grant themselves `GenericAll`.
  - `msDS-AllowedToActOnBehalfOfOtherIdentity`: The binary attribute governing Resource-Based Constrained Delegation (RBCD). Modifying this attribute on a computer object allows an adversary to impersonate any user to that computer.

- `msDS-KeyCredentialLink`: Stores public key certificates for PKINIT / Shadow Credentials. Writing to this attribute allows passwordless authentication as the target.
- `servicePrincipalName`: Controls Kerberos ticket routing. Adding an SPN to a user account exposes it to offline ticket extraction.

8. Defensive Telemetry, Detection Engineering & Hardening

8.1 Directory Service Event Telemetry

Domain Controllers record LDAP operations and security events within the **Directory Service** and **Security** event logs:

Event ID	Log Name	Operational Meaning	Strategic Significance
2886	Directory Service	LDAP signing warnings.	Logged every 24 hours when the DC is configured to allow unsigned LDAP binds.
2887	Directory Service	Summary of unsigned/cleartext LDAP binds.	Aggregates the total count of insecure binds over a 24-hour window.
2889	Directory Service	Unsigned LDAP client connection details.	**High-Value Auditing:** Identifies the exact client IP address, username, and bind type of every unsigned LDAP connection. Vital for identifying systems requiring remediation before enforcement.
3074 / 3075	Directory Service	LDAP Channel Binding audit events.	Identifies clients connecting without proper Channel Binding Tokens.
5136	Security	A directory service object was modified.	**Primary Attack Telemetry:** Records exact attribute modifications in Active Directory, displaying the object DN, modified attribute name, and the old/new values.

8.2 Detection Heuristics & Hunting Logic

1. Dangerous Attribute Modifications (KQL):

Detects modifications to high-risk Active Directory attributes typically targeted during privilege escalation:

```
SecurityEvent
| where EventID == 5136
| where AttributeLDAPDisplayName in (
    "msDS-AllowedToActOnBehalfOfOtherIdentity",
    "msDS-KeyCredentialLink",
    "nTSecurityDescriptor",
    "servicePrincipalName",
    "adminCount"
)
| project TimeGenerated, SubjectUserName, ObjectDN, AttributeLDAPDisplayName, AttributeValue, OperationType
```

2. Auditing Insecure LDAP Clients:
Monitor Directory Service Event `2889` to generate an inventory of all applications transmitting unsigned LDAP requests across the enterprise:

```
Log: Directory Service
Event ID: 2889
Filter: EventData contains "Client IP" and "Binding Type: 0" (Cleartext) or "Binding Type: 1" (Unsigned SASL)
```

8.3 Enterprise Hardening Blueprint

Eradicating LDAP attack surface requires enforcing cryptographic signing and channel binding across all Domain Controllers:

1. **Enforce LDAP Server Signing:**
   - Configure GPO: `Computer Configuration → Windows Settings → Security Settings → Local Policies → Security Options`:
     - `Domain controller: LDAP server signing requirements` → **Require signing**
   - *Impact:* Neutralizes SASL NTLM relay into LDAP port 389 across the entire domain.
2. **Enforce LDAPS Channel Binding (CBT):**
   - On all Domain Controllers, enforce Channel Binding Tokens by deploying the registry value:
     - Path: `HKLM\SYSTEM\CurrentControlSet\Services\NTDS\Parameters`
     - Value Name: `LdapEnforceChannelBinding`
     - Value Type: `DWORD`
     - Value Data: `2` (Enforce Channel Binding)
   - *Impact:* Completely prevents cross-protocol NTLM relaying into LDAPS port 636.
3. **Restrict Anonymous LDAP Access:**
   - Verify that the `SeventhDsHeuristic` attribute on `CN=Directory Service,CN=Windows NT,CN=Services,CN=Configuration,DC=corp,DC=local` is not set to `2` (which forces anonymous access support).
4. **Deploy Enterprise PKI for Universal LDAPS:**
   - Ensure every Domain Controller automatically enrolls a valid Server Authentication certificate from an enterprise Certificate Authority to provide secure, encrypted TLS connectivity for non-Windows clients.

9. Authoritative Specifications, RFCs & Cross-Vault Links

9.1 Authoritative Specifications

- **RFC 4511:** *Lightweight Directory Access Protocol (LDAP): The Protocol* — The foundational specification defining message envelopes, operations, and ASN.1 encoding.

- **RFC 4515:** *Lightweight Directory Access Protocol (LDAP): String Representation of Search Filters* — Formal specification for filter syntax and prefix notation.
- **RFC 2696:** *LDAP Control Extension for Simple Paged Results* — Specification for pagination and server-side cookie management.
- **RFC 4422:** *Simple Authentication and Security Layer (SASL)* — The general framework for pluggable connection-based authentication.
- **[MS-ADTS]:** *Active Directory Technical Specification* — Microsoft specification detailing Active Directory directory partitions, schema rules, and matching rule OIDs.

Security Descriptors & ACLs — Access Control Architecture in Active Directory

> **Architectural Mission:** Master the internal mechanics of the Windows Access Control Model as implemented within Active Directory Domain Services (AD DS). Understand how the Security Reference Monitor (SRM) processes access tokens against security descriptors, deconstruct the binary `nTSecurityDescriptor` and its component Access Control Entries (ACEs), master the string syntax of the Security Descriptor Definition Language (SDDL), and demystify why enterprise privilege in Active Directory is not a binary status flag, but a directed mathematical graph of discretionary permissions.

1. Architectural Foundations & Purpose

1.1 Authentication vs Authorization: The Core Separation

In modern operating systems and directory services, security is divided into two distinct cryptographic and kernel operations:

1. **Authentication:** Establishes and verifies identity ("Who are you?"). In Active Directory, this is accomplished via Kerberos tickets or NTLM challenge-response handshakes. When Ahmed logs in, authentication produces an authenticated identity represented by a unique **Security Identifier (SID)**.
2. **Authorization:** Governs what that authenticated identity is permitted to execute ("What are you allowed to do?").

In Windows and Active Directory, **there is no such architectural concept as an "administrative user" in the abstract**. An account is not inherently privileged because of its name or account type. Instead, privilege is purely the outcome of a continuous, mathematical evaluation performed by the operating system kernel: **evaluating a caller's Access Token against the Security Descriptor attached to a requested resource**.

1.2 The Active Directory Access Control Model

The access control model across all Windows NT derivatives rests upon four interconnected components:

1. **The Securable Object:** Any resource that can have security enforced upon it (such as a user account in `ntds.dit`, an Organizational Unit, an NTFS file,

a registry key, or a service process). Every Active Directory object stores its authorization policy within a binary attribute named `nTSecurityDescriptor`.

2. **The Access Token (The Subject):** Generated during logon by the Local Security Authority Subsystem Service (LSASS). The Access Token accompanies every process and thread executed by the user. It contains the user's primary SID, the SIDs of all security groups the user belongs to (direct and nested), and user-held system privileges (e.g., `SeDebugPrivilege`).
3. **The Security Descriptor (The Object):** A structured binary data container attached to the securable object defining who owns it, who can access it, and what actions generate security audit logs.
4. **The SRM (Security Reference Monitor):** The core security engine within the Windows NT executive kernel (`ntoskrnl.exe`), mirrored in user-mode by Active Directory Domain Services, responsible for executing the formal **Access Check Algorithm**.

1.3 Terminology & Core Primitives (Zero-Acronym Onboarding)

- **SID (Security Identifier):** A variable-length cryptographic byte string (e.g., `S-1-5-21-394 ... -1105`) that uniquely identifies a security principal (user, group, computer, or domain) across the forest.
- **RID (Relative Identifier):** The final numeric sub-authority component of a SID (e.g., `1105` for Ahmed, `500` for the built-in Administrator, `512` for Domain Admins) uniquely distinguishing an object within a specific domain.
- **SRM (Security Reference Monitor):** The Windows kernel component that evaluates access requests by comparing caller access tokens against object security descriptors.
- **ACL (Access Control List):** A container list composed of zero or more individual Access Control Entries.
- **DACL (Discretionary Access Control List):** The specific ACL that grants or denies access rights to users and groups. It is "discretionary" because the owner of the object has the discretion to assign whatever permissions they choose.
- **SACL (System Access Control List):** The specific ACL that instructs the operating system on which access attempts (successful or failed) should generate security audit events in the Windows Event Log.
- **ACE (Access Control Entry):** A single discrete rule inside an ACL specifying: *"Principal X is Granted/Denied Permission Y on Object Z."*
- **SDDL (Security Descriptor Definition Language):** A standardized, human-readable text string format used by Windows to serialize and represent binary security descriptors.
- **AdminSDHolder:** A special template object located in the Active Directory System container (`CN=AdminSDHolder,CN=System,DC=corp,DC=local`) that protects the DACLs of all high-value administrative accounts and groups.

- **SDProp (Security Descriptor Propagator):** A background maintenance thread executing on the Primary Domain Controller (PDC) Emulator that periodically overwrites the DACLs of protected administrative accounts with the clean template from `AdminSDHolder`.
- **GUID (Globally Unique Identifier):** A 128-bit integer used in Active Directory schema to uniquely identify object classes, specific attributes (Property Sets), and extended access rights.

2. Walkthrough: The SRM Access Check Algorithm

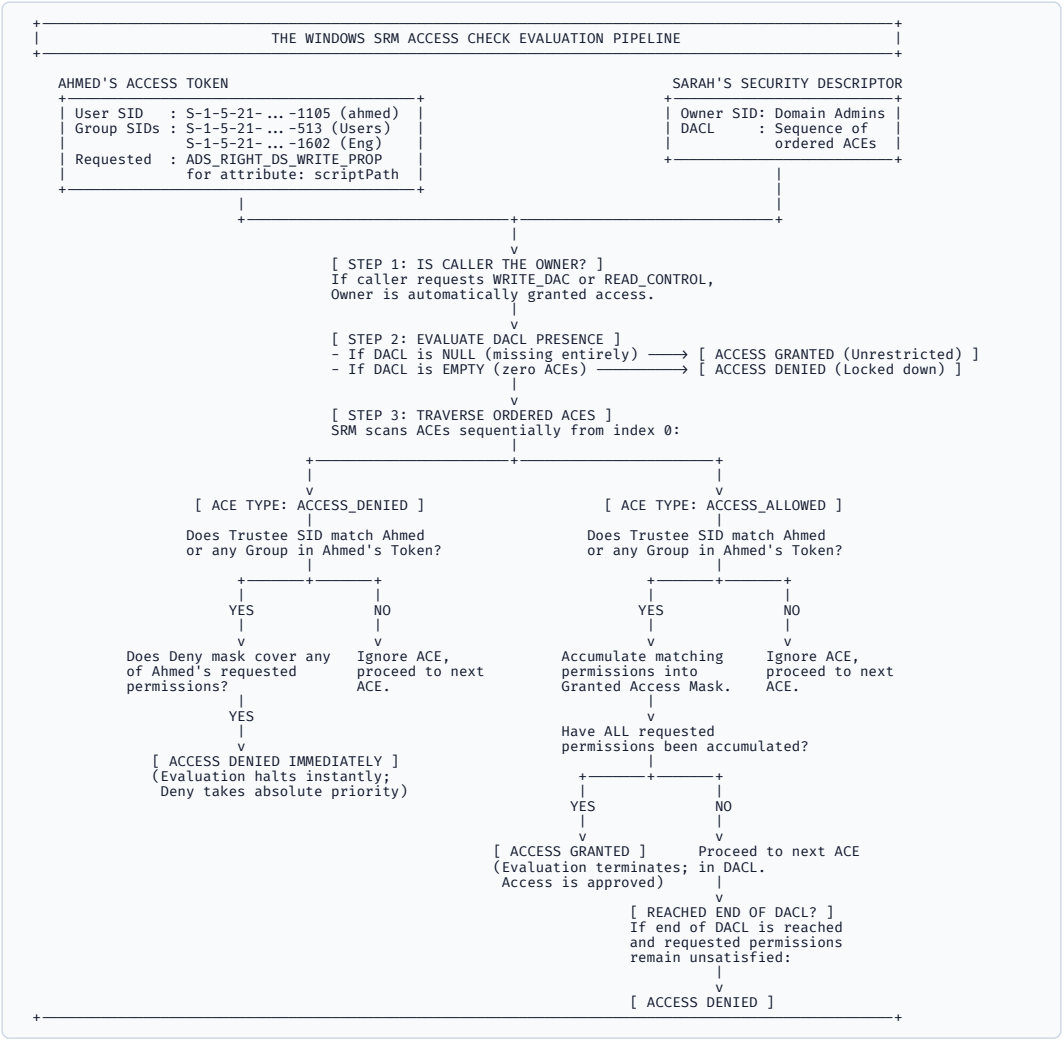
2.1 The Scenario Setup

To observe how the Windows Security Reference Monitor (SRM) enforces authorization, consider this concrete enterprise scenario:

- **Requesting User:** `ahmed` (SID: `S-1-5-21-394 ... -1105`, member of `Engineering-Users` group SID `S-1-5-21-394 ... -1602`).
- **Target Object:** A user object representing a marketing department head: `CN=Sarah,OU=Marketing,DC=corp,DC=local` (SID: `S-1-5-21-394 ... -1250`).
- **The Desired Operation:** Ahmed executes an administrative tool or script that attempts to modify Sarah's account properties by updating her `scriptPath` attribute (or forcing a password reset).

The tool transmits an LDAP `modifyRequest` targeting `Sarah`'s Distinguished Name to `DC01`. Before the Domain Controller writes a single byte to `ntds.dit`, the security subsystem initiates the **SRM Access Check Algorithm**.

2.2 Diagram: The Access Check Decision Pipeline



2.3 Step-by-Step Traversal Rules

The SRM access check algorithm is strictly deterministic and follows canonical processing rules:

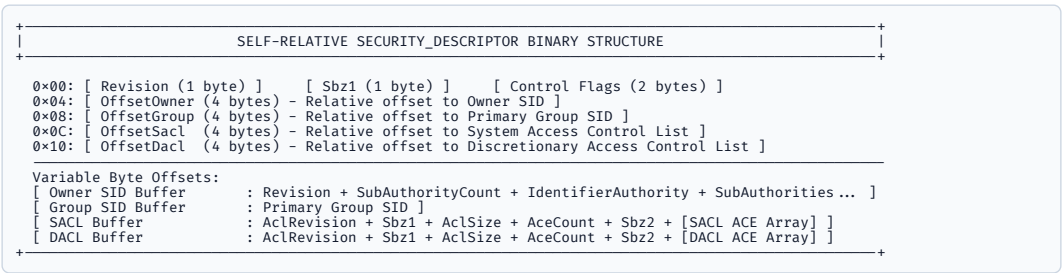
1. **Owner Rights Processing:** If Ahmed requests `WRITE_DAC` (the permission to modify the object's DACL) or `READ_CONTROL` (the permission to read the security descriptor), the SRM compares Ahmed's SID against the **Owner SID** field of Sarah's security descriptor. The object Owner has an immutable, mathematical guarantee: **an owner can always read and modify their own object's DACL**, even if a DACL explicitly denies them access!
2. **Canonical DACL Ordering:** To ensure predictable security semantics, Windows enforces a specific order when writing ACEs to a DACL:
   - **Explicit Deny ACEs** appear first.
   - **Explicit Allow ACEs** appear second.

- **Inherited Deny ACEs** (passed down from parent OUs) appear third.
- **Inherited Allow ACEs** appear fourth.

3. **Sequential Traversal:** The SRM steps through the DACL from top to bottom:
   - When an **Access-Denied ACE** matches Ahmed or any of his group memberships, the SRM checks if the denied permissions intersect with the requested permissions. If they intersect, the algorithm **aborts immediately and returns** `STATUS_ACCESS_DENIED`.
   - When an **Access-Allowed ACE** matches Ahmed or his groups, the granted rights are merged into the caller's running `GrantedAccess` bitmask.
   - If the `GrantedAccess` bitmask completely covers all rights Ahmed requested, the evaluation halts immediately, returning `STATUS_SUCCESS`.
4. **Default Deny Termination:** If the SRM reaches the final ACE in the DACL and Ahmed's requested rights have not been completely satisfied, access is denied. **Absence of an explicit Allow ACE equals a Deny.**

3. Dissecting the Binary `nTSecurityDescriptor`

When stored in `ntds.dit` or transmitted over LDAP, security descriptors are serialized in **Self-Relative Format** (specified in **[MS-DTYP] §2.4.6**). In a self-relative descriptor, all data structures (SIDs and ACLs) reside within a single contiguous memory buffer, referenced by relative byte offsets from the start of the structure.



3.1 Field-by-Field Breakdown

- `Revision` **(1 byte):** The version of the security descriptor structure. For all modern Windows systems, this is `0x01`.
- `Sbz1` **(1 byte):** "Should be zero" padding byte for memory alignment.
- `Control` **(2 bytes —** `SECURITY_DESCRIPTOR_CONTROL`**):** A bitmask defining the behavioral characteristics of the descriptor. Crucial flags include:
  - `SE_DACL_PRESENT` (`0x0004`): The security descriptor contains a valid DACL. (If unset, the object has a NULL DACL, granting unrestricted access to everyone).
  - `SE_DACL_PROTECTED` (`0x1000`): **Blocks ACL Inheritance.** Prevents permissions defined on parent containers (like OUs) from flowing down into this object.

    - `SE_DACL_AUTO_INHERITED` (`0x0400`): Indicates that the DACL was built by automatically propagating inherited ACEs from parent containers.
    - `SE_SELF_RELATIVE` (`0x8000`): Confirms that the descriptor is stored in self-relative format rather than using absolute memory pointers.
- **`OffsetOwner` (4 bytes):** Byte offset from the start of the buffer to the Owner SID.
- **`OffsetGroup` (4 bytes):** Byte offset to the primary group SID (retained for POSIX compliance).
- **`OffsetSacl` (4 bytes):** Byte offset to the SACL. If zero, no SACL exists.
- **`OffsetDacl` (4 bytes):** Byte offset to the DACL.

3.2 Anatomy of an Access Control Entry (ACE)

An ACL is simply an array of individual ACE structures preceded by an 8-byte header (`ACL_HEADER`). Every ACE begins with a standardized 4-byte header:

```
ACE Header:
├── AceType  (1 byte)  : Determines whether this ACE Allows, Denies, Audits, or targets an Object.
├── AceFlags (1 byte)  : Controls inheritance and audit behavior.
└── AceSize  (2 bytes) : Total byte length of this ACE structure.
```

ACE Types (`AceType`):

1. **`ACCESS_ALLOWED_ACE_TYPE` (`0x00`):** Standard allow rule granting specific permissions to a trustee SID.
2. **`ACCESS_DENIED_ACE_TYPE` (`0x01`):** Standard deny rule blocking specific permissions for a trustee SID.
3. **`ACCESS_ALLOWED_OBJECT_ACE_TYPE` (`0x05`): The foundation of Active Directory granular control.** Unlike file systems where permissions apply to an entire file, Active Directory objects possess individual attributes and extended rights. An Object ACE contains two optional 128-bit GUIDs:
    - `ObjectType GUID`: Specifies the exact schema attribute (e.g., `userPassword`), attribute set, or extended right (e.g., `User-Force-Change-Password`) governed by this ACE.
    - `InheritedObjectType GUID`: Specifies the class of child objects (e.g., only child `user` objects) that inherit this ACE when attached to an OU container.
4. **`ACCESS_DENIED_OBJECT_ACE_TYPE` (`0x06`):** Object-specific deny rule.

Inheritance Flags (`AceFlags`):

- `CONTAINER_INHERIT_ACE` (`0x02`): Containers (such as OUs) propagate this ACE to child containers.
- `OBJECT_INHERIT_ACE` (`0x01`): Containers propagate this ACE to child leaf objects (such as users or computers).
- `INHERIT_ONLY_ACE` (`0x08`): The ACE does not apply to the current object; it applies exclusively to child objects that inherit it.
- `INHERITED_ACE` (`0x10`): Indicates that this ACE was inherited from a parent container rather than being explicitly defined on this object.

4. Deconstructing SDDL (Security Descriptor Definition Language)

Because binary security descriptors are unwieldy to view or edit, Microsoft created **SDDL (Security Descriptor Definition Language)**—a compact, formal string representation formalized in **[MS-DTYP] §2.5.1**.

4.1 Master SDDL Syntax Layout

An SDDL string encodes the four components of a security descriptor in a standardized prefix format:

$$\text{SDDL} = \mathbf{O}{:}\langle\text{OwnerSID}\rangle \quad \mathbf{G}{:}\langle\text{GroupSID}\rangle \quad \mathbf{D}{:}\langle\text{DACL_Flags}\rangle\langle\text{ACE_List}\rangle \quad \mathbf{S}{:}\langle\text{SACL_Flags}\rangle\langle\text{ACE_List}\rangle$$

Individual ACE String Format:
Every ACE inside the DACL (`D:`) or SACL (`S:`) is enclosed in parentheses and contains exactly **six semicolon-delimited fields**:

$$(\text{AceType} \;;\; \text{AceFlags} \;;\; \text{Rights} \;;\; \text{ObjectType} \;;\; \text{InheritedObjectType} \;;\; \text{TrusteeSID})$$

```
Field 1: AceType             → 'A' (Allow), 'D' (Deny), 'OA' (Object Allow), 'OD' (Object Deny)
Field 2: AceFlags            → 'CI' (Container Inherit), 'OI' (Object Inherit), 'ID' (Inherited)
Field 3: Rights (AccessMask) → Two-character abbreviations or hex mask ('GA', 'RP', 'WP', etc.)
Field 4: ObjectType          → Schema GUID of specific attribute or extended right (or empty)
Field 5: InheritedObjectType → Schema GUID of child object class allowed to inherit (or empty)
Field 6: TrusteeSID          → SID string or well-known trustee alias ('DA', 'WD', 'AU', etc.)
```

4.2 Dissecting Enterprise SDDL Examples

Example 1: Standard Property Read/Write

`D:(A;;RPWP;;;WD)`

- `D:` → Discretionary Access Control List (DACL).
- `A` → `AceType`: Access Allowed.
- `;;` → `AceFlags`: None (explicitly applied to this object, not inherited).
- `RPWP` → `Rights`: Read Property (`RP`, `0×00000010`) and Write Property (`WP`, `0×00000020`).
- `;;` → `ObjectType` and `InheritedObjectType`: Empty (applies broadly to all properties on the object).
- `WD` → `TrusteeSID`: Well-known alias for **Everyone** (`World` — `S-1-1-0`).
- *Plain English:* **"Allow Everyone to read and write all properties on this object."**

Example 2: Administrative Full Control

`D:P(A;;GA;;;DA)`

- `D:P` → DACL with `P` flag (`SE_DACL_PROTECTED` — **Inheritance is blocked** parent container ACEs will not flow down).
- `A` → Access Allowed.
- `GA` → `Rights`: **Generic All** (`0×10000000` — Full Unconstrained Control).
- `DA` → `TrusteeSID`: Well-known alias for **Domain Admins** (`S-1-5-21-` ... `-512`).
- *Plain English:* **"Block inheritance, and grant Domain Admins complete unconstrained control."**

Example 3: Granular Password Reset (Object ACE)

`D:(OA;;CR;00299570-246d-11d0-a7ec-00c04fb677f1;;PS)`

- `OA` → `AceType`: **Object Access Allowed** (`ACCESS_ALLOWED_OBJECT_ACE_TYPE`).
- `CR` → `Rights`: Control Access Right (`0×00000100` — Extended Right).
- `00299570-246d-11d0-a7ec-00c04fb677f1` → `ObjectType`: The Active Directory schema GUID for **`User-Force-Change-Password`**.
- `PS` → `TrusteeSID`: Well-known alias for **Principal Self** (`S-1-5-10`).
- *Plain English:* **"Grant the user itself the extended right to change its own password."**

4.3 Reference Table: Common SDDL Aliases & Rights

SDDL Code	Meaning / Constant	Numerical Value	Category
GA	GENERIC_ALL	0×10000000	Full Unconstrained Administrative Control
GR	GENERIC_READ	0×20000000	Read all properties and permissions
GW	GENERIC_WRITE	0×40000000	Write all properties and attributes
RC	READ_CONTROL	0×00020000	Read the Security Descriptor (DACL/Owner)
WD	WRITE_DAC	0×00040000	**Modify the DACL** (Can grant oneself GenericAll)
WO	WRITE_OWNER	0×00080000	**Seize Ownership** of the object
RP	ADS_RIGHT_DS_READ_PROP	0×00000010	Read a specific directory property
WP	ADS_RIGHT_DS_WRITE_PROP	0×00000020	Write a specific directory property
CR	ADS_RIGHT_DS_CONTROL_ACCESS	0×00000100	Execute an Extended Right (e.g., DCSync, Reset Password)
CC	ADS_RIGHT_DS_CREATE_CHILD	0×00000001	Create child objects within a container
DC	ADS_RIGHT_DS_DELETE_CHILD	0×00000002	Delete child objects within a container

Well-Known Trustee Aliases:

- **DA:** Domain Administrators (`S-1-5-21-...-512`)
- **EA:** Enterprise Administrators (`S-1-5-21-...-519`)
- **BA:** Built-in Administrators (`S-1-5-32-544`)
- **AU:** Authenticated Users (`S-1-5-11`)
- **WD:** Everyone (`World` — `S-1-1-0`)
- **PS:** Principal Self (`S-1-5-10`)
- **SY:** Local System (`NT AUTHORITY\SYSTEM` — `S-1-5-18`)

5. Demystifying Privilege: Active Directory as a Directed Graph

Why do modern Active Directory security evaluations focus on graph theory rather than looking for accounts marked as “Administrators”?

5.1 The Mathematical Model: Privilege is a Graph

In Active Directory, identity and access form a **Directed Graph** $G = (V, E)$:

- **Vertices (V):** Principals and objects (Users, Groups, Computers, OUs, GPOs, Domains).
- **Edges (E):** Access Control Entries (ACEs), group memberships, and architectural relationships directed from a Subject to an Object:

$$\text{Subject} \xrightarrow{\text{Edge Type}} \text{Object}$$

Privilege escalation in Active Directory is simply the mathematical discovery of a **Transitive Path** from a low-privileged vertex to a Tier-0 administrative vertex:

```
[ ahmed ]
   |
   ├── (MemberOf)
   v
[ Engineering-Lead-Group ]
   |
   ├── (WriteDacl)
   v
[ IT-Service-Account ]
   |
   ├── (GenericAll)
   v
[ Domain Admins Group ]
   |
   ├── (MemberOf)
   v
[ Full Forest Compromise ]
```

Ahmed is not in Domain Admins. Ahmed’s group is not in Domain Admins. Yet Ahmed mathematically controls Domain Admins because **he can traverse a chain of discretionary ACE permissions**.

5.2 The High-Impact ACE Primitives Explained

1. `GenericAll` **(Full Control):**
   - **Mechanism:** Bestows all standard, generic, and object rights simultaneously.
   - **Impact:** The trustee can change the target’s password, modify its group memberships, rewrite its SPNs, edit its attributes, or seize ownership.
2. `WriteDacl` **(**`WRITE_DAC`**):**
   - **Mechanism:** The right to modify the Discretionary Access Control List of the object.
   - **The Escalation Path:** An attacker with `WriteDacl` on an object does not need any other rights. They simply craft a new ACE granting themselves `GenericAll`, append it to the DACL, and write it back via LDAP. `WriteDacl` **is mathematically equivalent to** `GenericAll`**.**
3. `WriteOwner` **(**`WRITE_OWNER`**):**
   - **Mechanism:** The right to seize ownership of the object.

- **The Escalation Path:** As established in Section 2.3, the Windows SRM guarantees that an object Owner always retains the right to read and modify the DACL (`WRITE_DAC`). An attacker with `WriteOwner` changes the owner to their own account, uses owner rights to grant themselves `GenericAll`, and takes full control. **`WriteOwner` is mathematically equivalent to `GenericAll`.**

4. **`WriteProperty` over Sensitive Attributes:**
   - **`msDS-AllowedToActOnBehalfOfOtherIdentity`:** Writing this binary security descriptor on a computer object configures Resource-Based Constrained Delegation (RBCD), allowing the attacker to impersonate any user to that machine.
   - **`msDS-KeyCredentialLink`:** Writing to this attribute injects a public key certificate, enabling passwordless authentication via PKINIT (Shadow Credentials).
   - **`servicePrincipalName` (SPN):** Writing an SPN onto a user account exposes that account to offline Kerberoasting.
5. **Extended Rights (`CR` — `ControlAccess`):**
   - **`User-Force-Change-Password` (`00299570-246d-11d0-a7ec-00c04fb677f1`):** Allows an attacker to reset a user's password directly via LDAP or RPC without knowing the user's current password.
   - **`DS-Replication-Get-Changes` & `DS-Replication-Get-Changes-All` (DCSync Rights):** When granted on the Domain Root object (`DC=corp,DC=local`), these two extended rights allow a principal to mimic a Domain Controller using the Directory Replication Service Remote Protocol (MS-DRSR), dumping all password hashes across the forest.

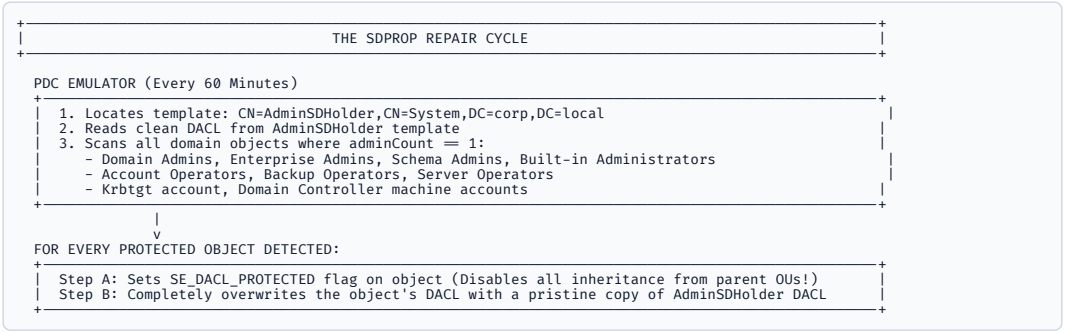
6. The Defense of Privileged Objects: AdminSDHolder & SDProp

Because Active Directory inheritance and delegation can accumulate complex, unintentional ACLs over time, Microsoft engineered an automated immune system to protect Tier-0 accounts: **AdminSDHolder** and the **SDProp (Security Descriptor Propagator)** process.

6.1 The Problem: ACL Drift on Administrative Accounts

Consider an enterprise where an administrator accidentally delegates permissions on an Organizational Unit (`OU=Admins,DC=corp,DC=local`) to a junior helpdesk group. Because ACEs inherit downward by default, the helpdesk group would suddenly inherit `GenericAll` or `ResetPassword` over the Domain Administrator accounts residing inside that OU!

6.2 The Protection Mechanism: AdminSDHolder & SDProp



1. **The Protected Objects (`adminCount = 1`):** Active Directory maintains an internal list of sensitive administrative groups and accounts (Domain Admins, Enterprise Admins, Schema Admins, Administrators, Account Operators, Backup Operators, and the `krbtgt` account). When an account is added to any of these groups, the directory sets its **`adminCount`** attribute to `1`.
2. **The SDProp Background Thread:** Every 60 minutes, a background thread named **SDProp** wakes up on the Domain Controller holding the Primary Domain Controller (PDC) Emulator FSMO role.
3. **Inheritance Severing:** SDProp inspects every object with `adminCount = 1`. It forcibly applies the `SE_DACL_PROTECTED` flag to the object's security descriptor, instantly severing inheritance from its parent OU. Any accidental delegation granted at the OU layer is stripped away.
4. **Template Enforcement:** SDProp copies the pristine DACL from `CN=AdminSDHolder,CN=System,DC=corp,DC=local` and overwrites the object's DACL, restoring strict administrative access.

6.3 The Architectural Implications for Persistence

- **The Ineffective Temporary Backdoor:** If an adversary compromises a Domain Admin and adds a rogue ACE directly to the Domain Admin's DACL, the backdoor will function for at most 60 minutes. As soon as SDProp runs, the rogue ACE is eradicated.
- **The Ultimate Stealth Persistence (`AdminSDHolder` Poisoning):** Conversely, if an adversary with temporary domain administrative access adds an ACE granting their hidden account `GenericAll` directly to the **`CN=AdminSDHolder` template object**, SDProp works in the attacker's favor! Every 60 minutes, SDProp automatically pushes the attacker's backdoor ACE onto every Domain Admin, Enterprise Admin, and protected group across the entire Active Directory forest.

7. Defensive Telemetry, Detection Engineering & Hardening

7.1 Security Event Telemetry

Modifications to Active Directory security descriptors generate high-fidelity security audit events:

Event ID	Log Name	Operational Meaning	Strategic Significance
5136	Security	A directory service object was modified.	**High-Value:** Generated when `nTSecurityDescriptor` is edited over LDAP. Shows the Object DN, the modified attribute (`nTSecurityDescriptor`), and the raw SDDL change.
4670	Security	Permissions on an object were changed.	Emitted when local or directory object DACLs are modified, displaying the old and new SDDL strings.
4738	Security	A user account was modified.	Look for changes to `adminCount` or password reset events initiated by non-administrative callers.
4728 / 4732	Security	A member was added to a security group.	Monitors additions to privileged administrative groups.

7.2 Detection Heuristics & Hunting Logic

1. Dangerous DACL Modifications on AdminSDHolder (KQL):

Detects any modification targeting the `AdminSDHolder` template container—a definitive marker of directory persistence staging:

```
SecurityEvent
| where EventID == 5136
| where ObjectDN =~ "CN=AdminSDHolder,CN=System,DC=corp,DC=local"
| where AttributeLDAPDisplayName == "nTSecurityDescriptor"
| project TimeGenerated, SubjectUserName, SubjectDomainName, ObjectDN, AttributeValue, OperationType
```

2. Detecting DCSync Rights Grants on the Domain Root (KQL):

Monitors when `DS-Replication-Get-Changes` (`1131f6aa-9c07-11d1-f79f-00c04fc2dcd2`) or `DS-Replication-Get-Changes-All` (`1131f6ad-9c07-11d1-f79f-00c04fc2dcd2`) extended rights are added to non-DC accounts:

```
SecurityEvent
| where EventID == 5136
| where ObjectDN =~ "DC=corp,DC=local"
| where AttributeLDAPDisplayName == "nTSecurityDescriptor"
| where AttributeValue has "1131f6aa-9c07-11d1-f79f-00c04fc2dcd2" or AttributeValue has "1131f6ad-9c07-11d1-
f79f-00c04fc2dcd2"
| project TimeGenerated, SubjectUserName, ObjectDN, AttributeValue
```

7.3 Enterprise Hardening Blueprint

Securing Active Directory against ACL-based attacks requires continuous hygiene and strict separation of administrative boundaries:

1. **Audit and Eliminate Dangerous ACE Paths:**
   - Regularly run graph-based security assessments to identify transitive control paths leading to Tier-0 assets.
   - Remove historical delegations that grant `GenericAll`, `WriteDacl`, or `WriteOwner` to non-administrative accounts on OUs hosting sensitive users or servers.
2. **Monitor the `AdminSDHolder` Template Object:**
   - Place an explicit SACL on `CN=AdminSDHolder,CN=System,DC=corp,DC=local` to generate high-priority SIEM alerts on any write operation.
3. **Remediate "Orphaned" Protected Accounts:**
   - When an administrator is removed from Domain Admins, Active Directory **does not automatically reset `adminCount` to 0**, nor does it re-enable inheritance. These accounts remain disconnected from OU policies. Identify and re-enable inheritance on former administrative accounts.
4. **Enforce Tiered Administration (Tier Model / Enterprise Access Model):**
   - Separate administrative accounts strictly: Tier-0 (Domain Admins), Tier-1 (Server Admins), Tier-2 (Workstation Admins).
   - Ensure that lower-tier administrators never possess write permissions, ownership, or password-reset authority over higher-tier accounts or containers.

8. Authoritative Specifications, RFCs & Cross-Vault Links

8.1 Authoritative Specifications

- **[MS-DTYP]:** *Windows Data Types* — Section 2.4.6 (`SECURITY_DESCRIPTOR`) and Section 2.5.1 (`Security Descriptor Definition Language - SDDL`). The definitive mathematical specification for Windows access control.
- **[MS-ADTS]:** *Active Directory Technical Specification* — Details directory object security descriptors, object ACE inheritance mechanics, and the SDProp background task.
- **[MS-PAC]:** *Privilege Attribute Certificate Data Structure* — Documents how group SIDs and user tokens are serialized into Kerberos tickets to form access tokens.

Group Policy Objects (GPO) — The Central Policy Engine in AD

> **Architectural Mission:** Master the internal mechanics of the Group Policy infrastructure in Active Directory Domain Services (AD DS). Understand why centralized policy orchestration replaces manual endpoint registry edits, analyze the split architecture between the Group Policy Container (GPC) in LDAP and the Group Policy Template (GPT) in SYSVOL, trace version synchronization mathematics via `GPT.INI`, deconstruct the LSDOU processing hierarchy, and examine how client-side extensions (CSEs) pull and apply declarative policies into operating system kernel states.

1. Architectural Foundations & Purpose

1.1 The Operational Necessity: Configuration Management at Scale

In a modern enterprise with 10,000 workstations and 1,000 servers, an administrator cannot manually log into endpoints to configure audit logging, enforce firewall rules, deploy software, or harden authentication protocols.

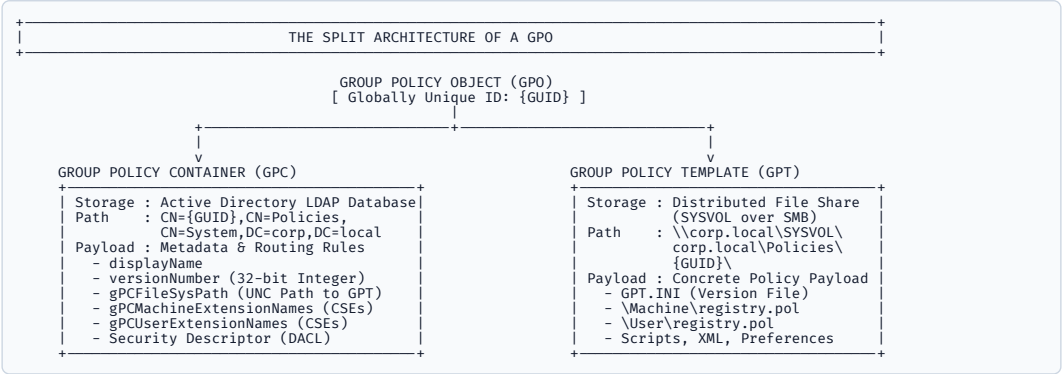
Group Policy is Active Directory's native, asynchronous, pull-based configuration management engine. It allows administrators to define a desired state once, attach that policy to a directory container, and have thousands of domain-joined Windows endpoints automatically converge toward that configuration without rebooting.

Group Policy operates on three core principles:

1. **Declarative State Enforcement:** Administrators define *what* settings must exist (e.g., "Disable NTLM fallback"), not the step-by-step procedural code to enforce it.
2. **Container-Based Targeting:** Policies are linked to logical directory containers (Sites, Domains, or Organizational Units), automatically applying to any computer or user object placed inside those containers.
3. **Pull-Based Autonomous Client Execution:** Domain Controllers do not "push" policies to endpoints. Instead, the local operating system on each client periodically reaches out to Domain Controllers over the network, detects configuration updates, downloads the policy files, and applies them locally.

1.2 The Two Halves of a Policy: The Split Architecture

A Group Policy Object (GPO) is not a single file or a single directory record. Instead, every GPO is **physically and logically split across two disparate storage mechanisms**:



Why Did Microsoft Split GPO Storage?

This split design was a deliberate engineering optimization:

- **LDAP (The GPC)** is an indexed database optimized for sub-millisecond query evaluation, security descriptor checks, and container linking (`gPLink`). However, database performance degrades severely if large binary files, installer scripts, or raw registry images are stored directly inside database columns.
- **The File System (The GPT)** is optimized for storing large text files, XML configurations, and binary payloads. However, a file system cannot natively participate in Active Directory OU hierarchy evaluations or access control queries.
- Therefore, Microsoft stores the **policy metadata, targeting rules, and security descriptors inside LDAP (GPC)**, while storing the **actual registry settings, scripts, and file payloads inside the SYSVOL file share (GPT)**.

1.3 Terminology & Core Primitives (Zero-Acronym Onboarding)

- **GPO (Group Policy Object):** The unified logical entity representing a collection of configuration policies.
- **GPC (Group Policy Container):** The Active Directory LDAP directory object under `CN=Policies,CN=System` containing GPO metadata and versioning.
- **GPT (Group Policy Template):** The file-system directory structure located inside the domain's `SYSVOL` file share containing the actual policy files (`registry.pol`, scripts, preferences).
- **SYSVOL (System Volume):** A special, replicated NTFS folder structure shared over SMB on every Domain Controller (at `\\<domain>\SYSVOL`), used to distribute Group Policy files and logon scripts to domain clients.

- **DFS-R (Distributed File System Replication):** The modern state-machine replication engine used by Domain Controllers to replicate the contents of `SYSVOL` across all DCs in a domain.
- **CSE (Client-Side Extension):** Dynamic link libraries (`.dll` files) registered within the client Windows operating system that parse specific policy formats (e.g., the Registry CSE, the Scripts CSE, the Folder Redirection CSE) and execute the configuration.
- **LSDOU (Local, Site, Domain, Organizational Unit):** The strict order of precedence governing which Group Policy Object takes priority when multiple policies conflict.
- **WMI Filter (Windows Management Instrumentation Filter):** A query attached to a GPO that evaluates operating system attributes on the client (such as OS version, RAM, or CPU architecture) before permitting the GPO to execute.
- **LAPS (Local Administrator Password Solution):** A Microsoft Group Policy extension that automatically generates, rotates, and stores unique random passwords for local administrator accounts on domain machines.

2. Scenario Walkthrough: Deploying Enterprise Hardening

2.1 The Enterprise Scenario Setup

To observe how Group Policy operates across the network, trace this real-world administrative workflow:

- **The Goal:** An Active Directory administrator wants to deploy **Windows LAPS** and enforce a registry policy that **Disables NTLM Fallback** across all engineering workstations.
- **The Target:** Workstations located in `OU=Workstations,OU=Engineering,DC=corp,DC=local`.
- **The Client:** Workstation `WKSTN01.corp.local` residing inside that OU.
- **The Tool:** The administrator opens the Group Policy Management Console (GPMC) on `DC01` and creates a new GPO named `SEC-Enforce-LAPS-NoNTLM`.

2.2 The Administrator's Write Operation (GPMC Workflow)

When the administrator clicks "Save" inside GPMC, the management tool executes an atomic two-phase write:

1. **Step 1: Minting the GPC in Active Directory (LDAP Write):** GPMC generates a random 128-bit GUID: `{31B2F340-016D-11D2-945F-00C04FB984F9}`. It transmits an LDAP `addRequest` to `DC01` to create the container: `CN={31B2F340-016D-11D2-945F-00C04FB984F9},CN=Policies,CN=System,DC=corp,DC=local`
   - It sets `displayName` to `SEC-Enforce-LAPS-NoNTLM`.
   - It sets `gPCFileSysPath` to `\\corp.local\SYSVOL\corp.local\Policies\{31B2F340 ... }`.

- It populates `gPCMachineExtensionNames` with the GUID of the Registry CSE.
- It sets the initial `versionNumber` to `65536` (Machine Version 1).

2. **Step 2: Provisioning the GPT in SYSVOL (SMB Write):** GPMC establishes an SMB connection to `\\DC01\SYSVOL\corp.local\Policies\` and creates a new folder named `{31B2F340-016D-11D2-945F-00C04FB984F9}`.
   - Inside this folder, it creates a text file named `GPT.INI`.
   - It writes the registry policy into a binary file: `\Machine\registry.pol` (containing instructions to set `RestrictNTLM_Incoming = 2`).
3. **Step 3: Linking the GPO to the Container (`gPLink`):** The administrator links the GPO to the Engineering OU. GPMC sends an LDAP `modifyRequest` to update the `gPLink` attribute on `OU=Workstations,OU=Engineering,DC=corp,DC=local`:

$$\text{gPLink} = [\mathbf{LDAP://CN=\{31B2F340...\},CN=Policies,CN=System,DC=corp,DC=local;0}]$$

The `;0` at the end indicates the link is enabled and not enforced.

3. The Split Architecture & Version Synchronization

3.1 GPC Metadata Attributes Dissected

When queried over LDAP, the Group Policy Container holds critical attributes:

Attribute Name	Syntax	Operational Purpose
`displayName`	String	Human-readable name displayed in GPMC (`SEC-Enforce-LAPS-NoNTLM`).
`versionNumber`	32-bit Integer	Tracks the synchronization version of the policy (encoded bitmask).
`flags`	Integer	Controls policy processing: • `0`: Both Machine and User settings enabled. • `1`: User configuration disabled. • `2`: Machine configuration disabled. • `3`: Entire GPO disabled.
`gPCFileSysPath`	UNC Path String	The network location of the physical GPT files in SYSVOL.
`gPCMachine ExtensionNames`	Multi-valued String	List of Client-Side Extension GUIDs required to execute the machine policy.
`gPCUserExtension Names`	Multi-valued String	List of Client-Side Extension GUIDs required to execute the user policy.
`nTSecurity Descriptor`	Binary SD	Access Control List determining who can read or edit the GPO.

3.2 The `versionNumber` Mathematics: Tracking User vs. Computer State

A single GPO contains two independent configuration sections: **Computer Configuration** and **User Configuration**. A change to a machine setting must not force user policies to re-evaluate, and vice versa.

To track both independent states inside a single 32-bit integer, Active Directory splits `versionNumber` into two 16-bit halves:

$$\textbf{versionNumber} = (\text{Machine Version} \ll 16) \mid \text{User Version}$$

```
32-bit versionNumber Layout:
+-------------------------------+-------------------------------+
|    HIGH 16 BITS (Bits 16-31)  |     LOW 16 BITS (Bits 0-15)   |
|     Machine Policy Version    |       User Policy Version     |
+-------------------------------+-------------------------------+
```

Concrete Bitmask Calculation:

Suppose an administrator edits `SEC-Enforce-LAPS-NoNTLM` three times for Machine settings, and zero times for User settings:

- Machine Version $= 3$ (`0x0003`)
- User Version $= 0$ (`0x0000`)
- versionNumber $= (3 \times 65536) + 0 = \mathbf{196608}$ (`0x00030000`).

Now, suppose an administrator edits a User setting twice:

- User Version $= 2$ (`0x0002`)
- versionNumber $= (3 \times 65536) + 2 = \mathbf{196610}$ (`0x00030002`).

3.3 The `GPT.INI` Version File

Inside the file system at `\\corp.local\SYSVOL\corp.local\Policies\{GUID}\GPT.INI`, Windows stores an identical integer value:

```
[General]
Version=196610
displayName=SEC-Enforce-LAPS-NoNTLM
```

How Endpoints Detect Policy Inconsistency:

When client endpoint `WKSTN01` evaluates this GPO:

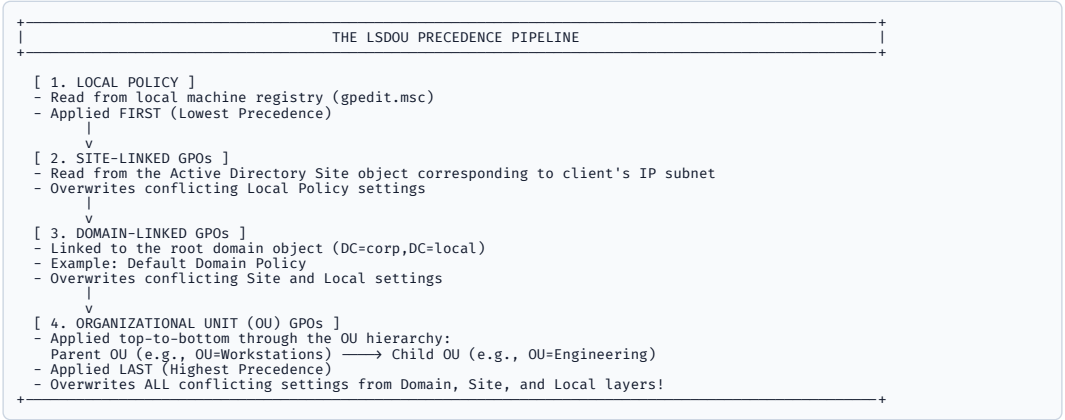
1. It reads `versionNumber` from the GPC via LDAP.
2. It reads `Version` from `GPT.INI` in SYSVOL via SMB.
3. **The Synchronization Check:** If the version in Active Directory and the version in `GPT.INI` do **not** match, Active Directory replication has fallen out of sync (e.g., directory replication finished before DFS-R completed file synchronization). The client detects a **Version Mismatch** and temporarily halts policy processing for that specific GPO to prevent applying corrupt or partially written configurations.

4. The Processing Hierarchy: The LSDOU Precedence Engine

When an endpoint evaluates policies, multiple GPOs inevitably contain conflicting instructions. One GPO might disable NTLM, while a legacy domain-wide policy might enable it. To resolve conflicts, the Windows Group Policy engine enforces a deterministic rule of precedence known as **LSDOU**.

4.1 The LSDOU Order of Precedence

Policies are processed strictly in the following sequential order:



> **The Architectural Rule of Precedence: "Last Applied Wins."** Because OU-linked GPOs execute last, settings configured on an Organizational Unit override conflicting settings inherited from the Domain, the Site, or the Local system.

4.2 Exceptions: Block Inheritance & Enforced (No Override)

Administrators can modify the standard LSDOU pipeline using two explicit architectural override mechanisms:

1. Block Inheritance:

- **Location:** Configured directly on an Organizational Unit container via the `gPOptions` attribute (`gPOptions = 1`).
- **Effect:** Prevents all policies linked to higher levels of the tree (Sites, Domains, and parent OUs) from flowing down into this OU and its child objects.
- **Exception:** It does **not** block policies marked as **Enforced**.

2. Enforced (Formerly "No Override"):

- **Location:** Configured on the specific GPO Link (`gPLink` option bit `0×00000002`).
- **Effect:** Two powerful architectural overrides occur:
  1. The Enforced GPO takes **absolute priority** over all lower-level GPOs, reversing the "last applied wins" rule.

2. The Enforced GPO **bypasses and punches through "Block Inheritance"**. Even if a child OU blocks inheritance, an Enforced domain-level GPO applies to all objects inside that OU without exception.

4.3 Policy Filtering: Security Filtering & WMI Filters

Not every object inside an OU must execute every GPO linked to that OU. Windows provides two granular filtering layers:

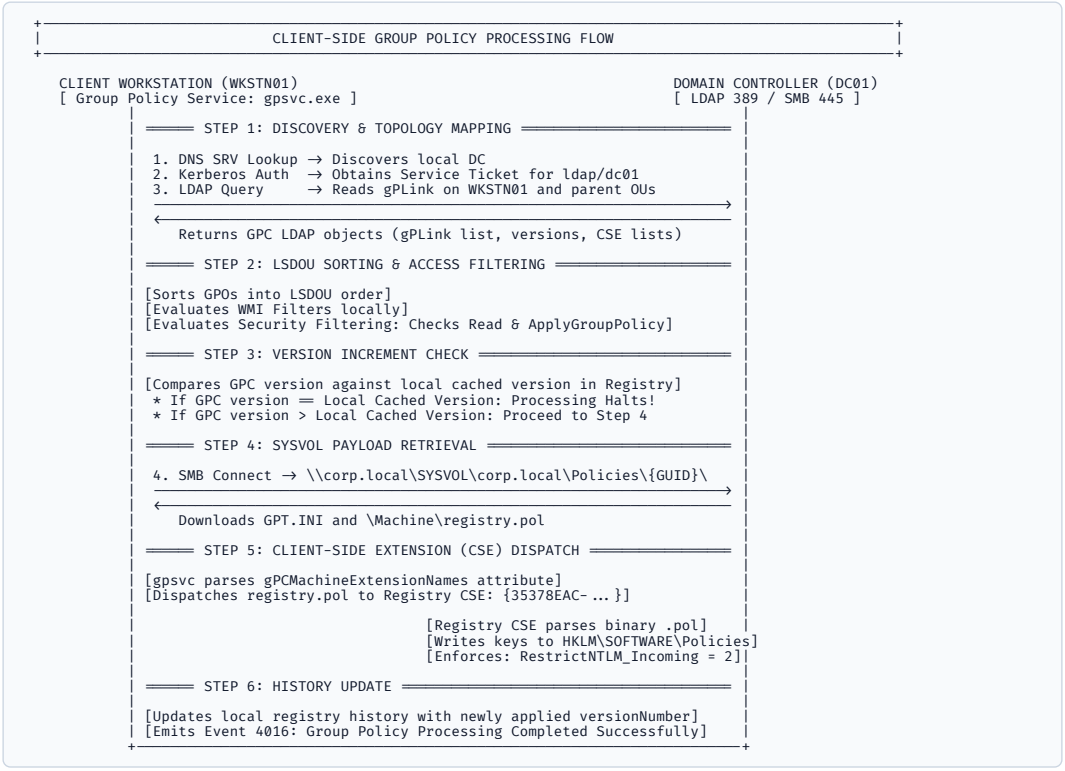
1. **Security Filtering (DACL Evaluation):** A GPO is applied to an endpoint only if the endpoint possesses two explicit permissions on the GPC's Access Control List:
   - **`Read`** (`READ_CONTROL`)
   - **`Apply Group Policy`** (Extended Right `0×00010000`)

   By default, GPOs grant both permissions to the special identity `Authenticated Users`. If an administrator removes `Authenticated Users` and adds only `Engineering-Workstations-Group`, only members of that security group execute the policy.
2. **WMI Filtering:** Administrators can attach a WMI query to a GPO (e.g., `SELECT * FROM Win32_OperatingSystem WHERE Version LIKE "10.0.22%" AND ProductType = "1"`). The client executes this query locally. If the query returns false, the GPO is ignored.

5. Client-Side Execution: The Pull-Based Architecture

How does an endpoint actually transform a file in SYSVOL into an active registry key or system security enforcement?

5.1 Diagram: The Client Policy Processing Flow



5.2 The Client-Side Extensions (CSEs) Demystified

The core Group Policy engine (`gpsvc.exe`) does not understand how to configure printers, install software, or modify the registry. Its only responsibility is to parse directory links, download files, and dispatch work to specialized DLLs called **Client-Side Extensions (CSEs)**.

When `gpsvc` reads a GPC, it inspects the **`gPCMachineExtensionNames`** attribute. This attribute contains bracketed pairs of GUIDs representing the CSE required to process that section: `[{35378EAC-683A-11D2-A89A-00C04FBBCFA2}{D0281610-C809-11D1-98B3-00C04FB1620D}]`

- The first GUID is the **Client-Side Extension GUID** registered in the local machine's registry under: `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\GPExtensions\`
- The registry key points directly to the DLL implementing the CSE.

Common Client-Side Extensions:

Extension Name	CSE GUID	Implementing DLL	Functionality
Registry	`{35378EAC-683A-11D2-A89A-00C04FBBCFA2}`	`gpapi.dll`	Parses `registry.pol` and writes keys into `HKLM\SOFTWARE\`

Extension Name	CSE GUID	Implementing DLL	Functionality
			`Policies` and `HKCU\SOFTWARE\Policies`.
Security	`{827D031D-0A42-11D0-86DA-00A024CAFE64}`	`scecli.dll`	Enforces local user rights assignments, account lockout thresholds, and audit policies via `GptTmpl.inf`.
Scripts	`{40B6664F-4972-11D1-A7CA-00A0C90826E0}`	`gpscript.dll`	Executes Startup, Shutdown, Logon, and Logoff scripts (`scripts.ini`).
Preferences	`{CDE6528A-EB96-450D-B129-E6433E428B34}`	`gppref.dll`	Deploys Scheduled Tasks, Local Users, Drive Maps, and Environment Variables via XML files.

5.3 Background Refresh Intervals

Endpoints do not wait for a user reboot or logon to refresh Group Policy:

- **Workstations and Member Servers:** Execute a background refresh every **90 minutes**, with a randomized **0 to 30 minute offset (skew)** to prevent thousands of machines from hitting Domain Controllers simultaneously (effective window: 90 to 120 minutes).
- **Domain Controllers:** Refresh policies every **5 minutes** to ensure security policies converge rapidly across the directory infrastructure.

6. Architectural Vulnerability Primitives (Zero Tool Dumping)

Security vulnerabilities involving Group Policy are not memory-corruption bugs. They are the inevitable result of **delegation misconfigurations, architectural trust properties, and unhardened storage models**.

6.1 The Sovereign Execution Privilege of Group Policy

- **The Design Principle:** The Group Policy client engine (`gpsvc.exe`) executes as **`NT AUTHORITY\SYSTEM`** on every domain machine.
- **The Friction Point:** When a machine processes a GPO, it inherently and unconditionally trusts the instructions written inside that GPO. The endpoint assumes that any instruction placed within SYSVOL was authored by an authorized enterprise administrator.
- **The Architectural Consequence:** If an adversary compromises write access to a GPO linked to an OU containing 5,000 workstations, the adversary writes an instruction to deploy a local administrator or execute a binary. Every single machine in that OU pulls the payload over SMB and executes it as `SYSTEM`. **Group Policy is the ultimate native mass-execution architecture.**

6.2 Structural Vulnerabilities in GPO Storage

1. **`WriteDacl` or `GenericAll` over the GPC (LDAP):** If a non-administrative account is granted write permissions over a GPO's directory object under `CN=Policies,CN=System`, the account can modify the GPO's security filtering, link malicious CSEs, or grant itself write access to the GPT.
2. **Write Access to the GPT in SYSVOL (SMB):** If an administrator incorrectly permissions the file system folder at `\\corp.local\SYSVOL\corp.local\Policies\{GUID}\`, an adversary can bypass Active Directory permissions entirely. The attacker edits `\Machine\registry.pol` or writes an immediate Scheduled Task directly into SYSVOL via SMB. The client downloads and executes the payload on its next 90-minute refresh cycle.
3. **Write Access to Container Links (`Write gPLink`):** If an adversary cannot modify existing GPOs, but possesses `WriteProperty` over the `gPLink` attribute on an Organizational Unit, the adversary creates a rogue GPO elsewhere in the domain (where they do have rights) and links it directly to that target OU, immediately seizing administrative control over all child machines.

6.3 The Historical GPP `cpassword` Disaster (MS14-025)

- **The Feature:** In Windows Server 2008, Microsoft introduced Group Policy Preferences (GPP), allowing administrators to deploy local users and services with pre-configured passwords stored in XML files in SYSVOL (e.g., `Groups.xml`).
- **The Flaw:** To secure the passwords, Microsoft encrypted them using AES-256 and stored the ciphertext in an attribute named `cpassword`. However, Microsoft **published the static 32-byte AES private key publicly on MSDN** to allow developers to build compatible tools.
- **The Disaster:** Because `SYSVOL` is globally readable by *any* authenticated user in the domain (`Authenticated Users` has Read permissions), any low-privileged domain user could search SYSVOL for `cpassword` XML files, decrypt the AES ciphertext using the static public key, and instantly recover plaintext local administrator passwords across the entire enterprise.

7. Defensive Telemetry, Detection Engineering & Hardening

7.1 Security Event Telemetry

Group Policy modifications and execution emit distinct telemetry across Domain Controllers and client endpoints:

Event ID	Log Name	Operational Meaning	Strategic Significance
5136	Security (DC)	A directory service object was modified.	**High-Value:** Traces modifications to `gPLink` on OUs, changes to GPC

Event ID	Log Name	Operational Meaning	Strategic Significance
			attributes, or edits to GPO permissions in Active Directory.
5137	Security (DC)	A directory service object was created.	Tracks newly created GPOs under `CN=Policies,CN=System`.
5145	Security (DC)	A network share object was accessed (Detailed File Share).	Monitors write operations targeting `\\SYSVOL\Policies\` folder structures.
4016	GroupPolicy/Operational (Client)	Starting processing of extension.	Client-side telemetry tracking when a specific CSE begins execution.
5016	GroupPolicy/Operational (Client)	Completed processing of extension.	Shows elapsed execution time and return codes for client-side policy applications.

7.2 Detection Heuristics & Hunting Logic

1. Unauthorized GPO Linking to High-Value Containers (KQL):

Detects modifications to the `gPLink` attribute on sensitive containers (the Domain Root or Domain Controllers OU):

```
SecurityEvent
| where EventID == 5136
| where AttributeLDAPDisplayName == "gPLink"
| where ObjectDN in ("DC=corp,DC=local", "OU=Domain Controllers,DC=corp,DC=local")
| project TimeGenerated, SubjectUserName, SubjectDomainName, ObjectDN, AttributeValue, OperationType
```

2. Detecting Direct Write Operations to SYSVOL Policies (KQL):

Monitors SMB write access to `.pol`, `.xml`, or script files within SYSVOL by non-administrative users:

```
SecurityEvent
| where EventID == 5145
| where ShareName endswith "\\SYSVOL"
| where RelativeTargetName has "Policies"
| where AccessMask has "0×2" // WriteData / AddFile
| where SubjectUserName !endswith "$" // Exclude machine sync operations
| project TimeGenerated, SubjectUserName, Computer, RelativeTargetName, AccessMask
```

7.3 Enterprise Hardening Blueprint

Securing the Group Policy infrastructure requires strict delegation hygiene and file-system auditing:

1. **Audit GPO Delegations Regularly:**
   - Ensure that write permissions (`GenericAll`, `WriteDacl`, `WriteProperty`, `CreateChild`) on objects under `CN=Policies,CN=System` are restricted strictly to **Domain Admins** and dedicated Tier-0 administrative roles.
   - Remove historical delegations that grant junior helpdesk staff write access to production GPOs.
2. **Restrict Container Linking Permissions:**

- Audit who possesses `WriteProperty` over `gPLink` and `gPOptions` across all OUs. Delegating OU administration should **never** include permission to link arbitrary GPOs unless explicitly architected.

3. **Purge Historical `cpassword` References:**
   - Scan all `SYSVOL` shares for legacy `.xml` files containing `cpassword` attributes and permanently delete them.
   - Enforce **Windows LAPS** domain-wide to manage local administrator credentials securely via Active Directory attributes protected by DACLs.
4. **Audit and Lock Down SYSVOL NTFS Permissions:**
   - Verify that standard users (`Authenticated Users`) retain **Read & Execute only** permissions across the entire `SYSVOL` folder hierarchy. Never grant write or modify permissions to non-administrators.

8. Authoritative Specifications, RFCs & Cross-Vault Links

8.1 Authoritative Specifications

- **[MS-GPOL]:** *Group Policy: Core Protocol* — The definitive technical specification defining GPC LDAP attributes, `GPT.INI` structures, version calculation bitmasks, and client-side processing algorithms.
- **[MS-GPREG]:** *Group Policy: Registry Extension Encoding* — Documents the binary format and PDU structures of `registry.pol` files.
- **[MS-DFSNM] / [MS-FRS2]:** *Distributed File System / Distributed File System Replication (DFS-R)* — Specifications governing `SYSVOL` replication across Domain Controllers.

Privileged Groups & Access Tokens — Runtime Kernel Authorization

> **Architectural Mission:** Master the internal mechanics of Windows runtime authorization and identity materialization. Understand how static LDAP directory objects in Active Directory transform into volatile in-memory Access Tokens within the Windows kernel, dissect Primary versus Impersonation token architectures, analyze Mandatory Integrity Control (MIC) and User Account Control (UAC) linked tokens, deconstruct the Win32 mechanics of high-risk kernel privileges, and evaluate the structural delegation hazards of Active Directory group scopes and legacy operator groups.

1. Architectural Foundations: Static Objects vs. Runtime Tokens

1.1 The Fundamental Boundary: Directory Records vs. Kernel Subjects

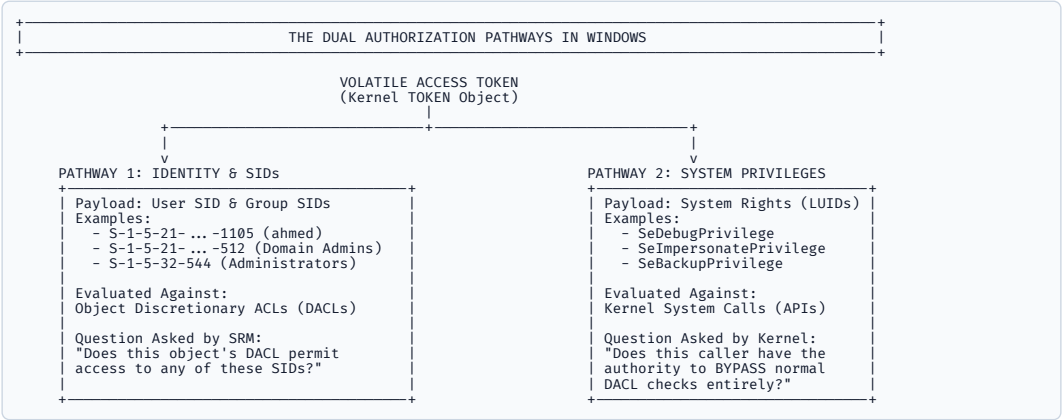
In Active Directory engineering, it is common to conflate an account stored in the directory with an account executing actions on an operating system. Architecturally, these two concepts belong to completely different layers of the computing stack:

- **The Directory Object (Static Storage):** A user account (such as `CN=Ahmed,OU=Users,DC=corp,DC=local`) is merely a static collection of rows and columns inside the `ntds.dit` database on a Domain Controller. The Windows executive kernel (`ntoskrnl.exe`) cannot evaluate LDAP objects directly. The kernel does not execute network directory queries when a thread attempts to open a file handle, read memory, or bind to a network socket.
- **The Access Token (Dynamic Runtime Subject):** When a user logs into a Windows system, the Local Security Authority Subsystem Service (LSASS) interrogates the incoming authentication proof (the Kerberos PAC or NTLM validation buffer) and requests that the kernel instantiate an in-memory **Access Token** (`TOKEN` structure).

> **Architectural Law:** The Windows Security Reference Monitor (SRM) never evaluates who a user is based on their Active Directory object. Authorization is performed **strictly and exclusively against the volatile Access Token attached to the calling process or thread**.

1.2 The Dual Authorization Architecture

An Access Token enforces authorization through **two parallel, independent pathways**:



1. **Pathway 1 (Group Memberships & SIDs):** Defines **who the caller is**. SIDs inside the token are matched against the Discretionary Access Control Lists (DACLs) of securable objects (files, directory objects, registry keys). Access is granted or denied based on standard discretionary permissions.
2. **Pathway 2 (Token Privileges / User Rights):** Defines **what operating system superpowers the caller wields**, regardless of group memberships or object DACLs. A single privilege (such as `SeBackupPrivilege`) allows a thread to bypass the DACLs of every file on the system, granting access to data that the user's SIDs are explicitly denied!

1.3 Terminology & Core Primitives (Zero-Acronym Onboarding)

- **Access Token:** An opaque Windows executive kernel object representing the security context of a process or thread, containing SIDs, privileges, and integrity levels.
- **Primary Token:** The base access token assigned to a process upon creation, inherited by all child processes by default.
- **Impersonation Token:** A specialized token assigned to an individual thread, allowing that thread to temporarily execute in the security context of a different client.
- **LUID (Locally Unique Identifier):** A 64-bit integer guaranteed to be unique on the local operating system until reboot, used by the kernel to track individual privileges (e.g., `SeDebugPrivilege`) and logon sessions.
- **MIC (Mandatory Integrity Control):** A core Windows security mechanism that assigns Integrity Levels (ILs) to securable objects and tokens to enforce the "No-Write-Up" principle.
- **UAC (User Account Control):** A Windows containment architecture that splits administrative logons into two linked tokens: a restricted Medium-

Integrity token for routine execution and an elevated High-Integrity token for administrative tasks.

- **LSASS (Local Security Authority Subsystem Service):** The protected user-mode process (`lsass.exe`) responsible for enforcing local security policy, managing authentications, and minting Access Tokens.
- **AGDLP (Accounts → Global → Domain Local → Permissions):** Microsoft's best-practice identity architecture for nesting user accounts into Global groups, nesting Global groups into Domain Local groups, and assigning permissions to Domain Local groups.

2. Walkthrough: From Logon Proof to Process Execution

2.1 The Scenario Setup

To observe how an Access Token is materialized and utilized, trace this enterprise sequence:

- **User:** `ahmed` (SID: `S-1-5-21-394 ... -1105`).
- **Endpoint:** Workstation `WKSTN01.corp.local`.
- **Target Action:** Ahmed types his password at the Windows logon screen, logs in interactively, and launches a command prompt (`cmd.exe`).

2.2 The Token Materialization Sequence

1. **Interactive Logon Submission:** The Windows Logon process (`winlogon.exe`) captures Ahmed's credentials via the Credential Provider interface and passes them securely to `lsass.exe`.
2. **Authentication & PAC Parsing:** The Kerberos package inside LSASS initiates an `AS-REQ` to the Domain Controller, receives an `AS-REP`, and unseals the Privilege Attribute Certificate (PAC).
   - LSASS extracts Ahmed's user SID (`S-1-5-21- ... -1105`).
   - LSASS extracts Ahmed's primary group RID (`513` - Domain Users) and domain security groups (e.g., `Engineering-Users` RID `1602`).
   - LSASS queries the local Security Account Manager (SAM) on `WKSTN01` to resolve any local groups Ahmed belongs to (e.g., the local machine's `Administrators` or `Remote Desktop Users` group).
3. **Kernel Token Minting (`NtCreateToken`):** LSASS switches into kernel mode via executive system call `NtCreateToken` (or internal kernel function `SepCreateToken`).
   - The kernel allocates a `TOKEN` object within non-paged kernel pool memory.
   - The kernel populates the token with Ahmed's user SID, resolved group SIDs, assigned system privileges, and default DACL.
   - The kernel stamps the token with a unique 64-bit Logon Session LUID (e.g., `0×14a89b`).

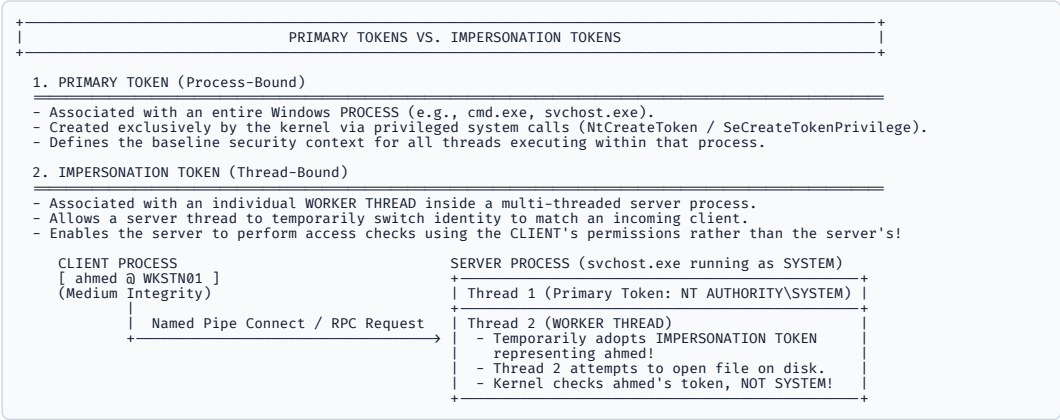
4. **Shell Instantiation:** `winlogon.exe` passes this token to the Userinit process (`userinit.exe`), which invokes the Win32 API `CreateProcessAsUser` to launch the Windows graphical shell: `explorer.exe` Ahmed's Primary Token is permanently attached to `explorer.exe`.

The Process Inheritance Chain:
When Ahmed opens a terminal:

1. Ahmed clicks the shortcut for `cmd.exe`.
2. The parent process (`explorer.exe`) calls Win32 API `CreateProcess`.
3. Under the Windows process creation architecture, **child processes inherit a duplicate copy of their parent's Primary Token by default**.
4. The kernel instantiates a new process object for `cmd.exe` and attaches a duplicate of Ahmed's token.
5. Every system call executed by `cmd.exe` is now evaluated against Ahmed's security context.

3. Token Anatomy: Primary vs. Impersonation Tokens

Within the Windows kernel, all tokens share the identical underlying data structure (`_TOKEN`), but serve two distinct operational models:



3.1 The Four Impersonation Levels

When a client connects to a server over a local named pipe, RPC, or ALPC (Advanced Local Procedure Call), the client dictates how far the server can go in adopting its identity. Windows defines **four hierarchical Impersonation Levels** (`SECURITY_IMPERSONATION_LEVEL`):

Level	Enumeration	Kernel Capability & Boundaries
Anonymous	`SecurityAnonymous` (`0`)	The server cannot obtain any identity information about the client. The client is completely anonymous (`NTAUTHORITY\ANONYMOUSLOGON`).

Level	Enumeration	Kernel Capability & Boundaries
Identification	`Security Identification` (`1`)	**Read-Only Identity:** The server can read the client's SIDs and privileges to perform access checks locally, but the server **cannot execute system calls under the client's identity**.
Impersonation	`Security Impersonation` (`2`)	**Local Impersonation:** The server can fully impersonate the client on the local machine. The server thread can open local files, write local registry keys, and execute local processes in the client's security context. **It cannot impersonate the client across the network to remote systems.**
Delegation	`Security Delegation` (`3`)	**Full Transitive Impersonation:** The server can impersonate the client locally **and** forward the client's credentials over the network to access remote servers across the Active Directory forest (requires Kerberos Delegation).

4. Mandatory Integrity Control (MIC) & UAC Linked Tokens

4.1 Mandatory Integrity Control (MIC)

In legacy Windows operating systems, authorization was governed entirely by Discretionary ACLs (DACLists). This created a severe architectural flaw: if Ahmed was a local administrator, *every single application* Ahmed executed (including web browsers downloading untrusted scripts) executed with full administrative authority. A compromised browser could immediately overwrite operating system binaries because Ahmed's DACL permitted it.

To solve this, Windows Vista introduced **Mandatory Integrity Control (MIC)**:

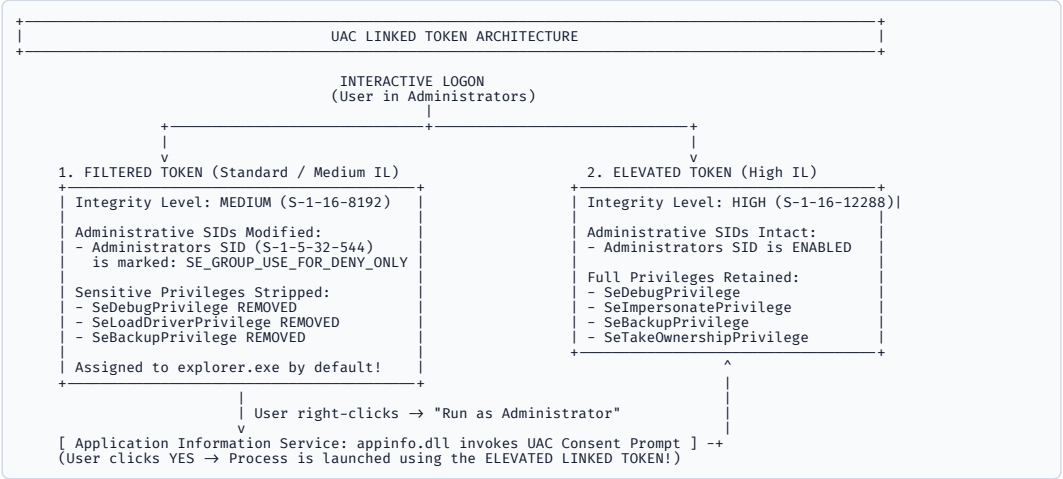
- Every securable object (files, registry keys, processes) is stamped with an **Integrity Level (IL)**.
- Every Access Token is stamped with an **Integrity Level SID**.
- **The Core Rule ("No-Write-Up / No-Read-Up"):** A process at a lower integrity level is mathematically prohibited from writing to, injecting threads into, or modifying an object at a higher integrity level, **even if the object's DACL grants the user full write access (`GenericAll`)**.

```
INTEGRITY LEVEL HIERARCHY:
+-------------------+-----------------------+-----------------------------------------------------+
| Integrity Level   | Well-Known SID        | Operating System Usage                              |
+-------------------+-----------------------+-----------------------------------------------------+
| SYSTEM            | S-1-16-16384 (0x4000) | Operating system core services (lsass.exe, drivers). |
| HIGH              | S-1-16-12288 (0x3000) | Elevated administrative processes (UAC approved).    |
| MEDIUM            | S-1-16-8192  (0x2000) | Standard interactive user desktop applications.      |
| LOW               | S-1-16-4096  (0x1000) | Sandboxed applications (Protected Mode Edge/Chrome). |
| UNTRUSTED         | S-1-16-0     (0x0000) | Anonymous processes and restricted app containers.   |
+-------------------+-----------------------+-----------------------------------------------------+
```

4.2 User Account Control (UAC) & The Linked Token Architecture

How does Windows allow an administrator to browse the web safely as a standard user while retaining the ability to elevate permissions?

The Mechanism of Linked Tokens: When a user who belongs to the local `Administrators` group logs in, LSASS does **not** create a single administrative token. Instead, LSASS mints **two linked tokens simultaneously**:



- **The `SE_GROUP_USE_FOR_DENY_ONLY` Attribute:** In the Filtered Token, the local `Administrators` SID is not deleted; it is flagged as **Deny-Only**. This means the SID can still be used by the SRM to *deny* access if an explicit deny rule exists, but it can never be used to *grant* administrative access.
- **The UAC Elevation:** When a user selects "Run as Administrator", the shell communicates via RPC with the **Application Information Service (`appinfo.dll`)**. If approved, `appinfo.dll` queries the Filtered Token for its `LinkedToken` handle, retrieves the **Elevated High-Integrity Token**, and launches the requested binary in that elevated context.

5. Dangerous Privileges: The Win32 Kernel Superpowers

Privileges are defined in the Windows kernel as 64-bit Locally Unique Identifiers (LUIDs). Unlike group memberships, which are bound by DACLs, **privileges override object security checks completely**.

Below is the definitive anatomical dissection of the five most dangerous kernel privileges:

5.1 `SeImpersonatePrivilege` & `SeAssignPrimaryTokenPrivilege`

- **Constant:** `SE_IMPERSONATE_NAME` / `SE_ASSIGNPRIMARYTOKEN_NAME`

- **Default Holders:** `NT AUTHORITY\SYSTEM`, `NT AUTHORITY\LOCAL SERVICE`, `NT AUTHORITY\NETWORK SERVICE`, and IIS Application Pools (`IIS_IUSRS`).
- **The Kernel Capability:** Allows a thread to impersonate any client token that connects to it via a local transport (named pipe, RPC, or COM), or assign a primary token to a newly spawned process.
- **The Architectural Hazard:** Any service account holding `SeImpersonatePrivilege` can trigger an internal system component running as `SYSTEM` to connect to a local named pipe (e.g., via local RPC or COM marshalling). When the `SYSTEM` account connects, the service thread captures the `SYSTEM` token, calls `DuplicateTokenEx`, and launches a full administrative process. **Holding `SeImpersonatePrivilege` on a machine is mathematically equivalent to holding full `NT AUTHORITY\SYSTEM` access.**

5.2 `SeDebugPrivilege`

- **Constant:** `SE_DEBUG_NAME`
- **Default Holders:** Local `Administrators` group (Elevated High-Integrity Token).
- **The Kernel Capability:** Originally engineered for software developers to debug failing applications. When enabled via Win32 API `AdjustTokenPrivileges`, **the kernel bypasses the discretionary ACL checks of every process in the operating system**.
- **The Architectural Hazard:** A process with `SeDebugPrivilege` can call `OpenProcess` with access mask `PROCESS_ALL_ACCESS` targeting *any* process running on the system—including protected core operating system binaries. The process can open `lsass.exe`, dump its virtual memory to disk, and extract all cached Kerberos tickets, session keys, and NTLM password hashes.

5.3 `SeBackupPrivilege` & `SeRestorePrivilege`

- **Constant:** `SE_BACKUP_NAME` / `SE_RESTORE_NAME`
- **Default Holders:** Built-in `Backup Operators` group and local `Administrators`.
- **The Kernel Capability:** Engineered to allow backup software to archive files without requiring explicit read access to every user's private folders. When a caller passes the flag `FILE_FLAG_BACKUP_SEMANTICS` to `CreateFile`, **the Windows kernel bypasses the object's DACL entirely**.
- **The Architectural Hazard:** Any user possessing `SeBackupPrivilege` can open and read any file on the operating system—including locked, in-use system files. On a Domain Controller, an account with `SeBackupPrivilege` can read `%SystemRoot%\NTDS\ntds.dit` and the registry hive `%SystemRoot%\System32\Config\SYSTEM`, completely compromising every credential in the Active Directory forest.

5.4 `SeTakeOwnershipPrivilege`

- **Constant:** `SE_TAKE_OWNERSHIP_NAME`
- **Default Holders:** Local `Administrators` group.

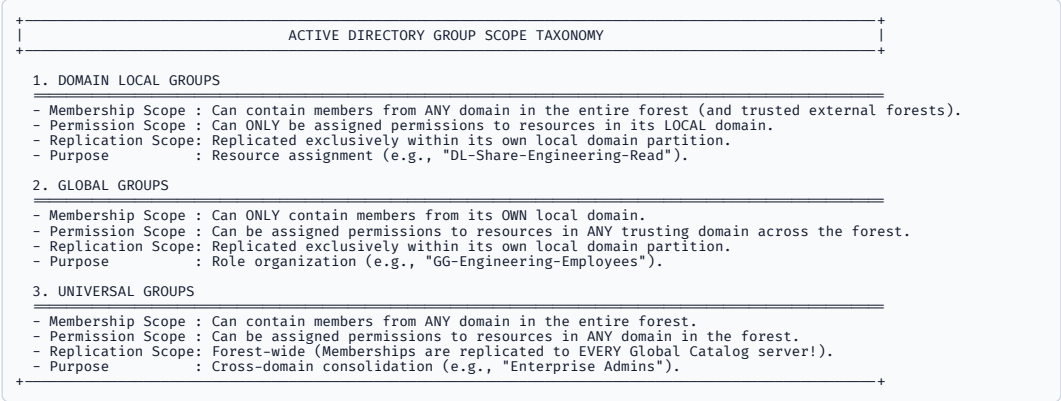
- **The Kernel Capability:** Allows a caller to overwrite the **Owner SID** field of any securable object's security descriptor with their own SID, regardless of what permissions the object's DACL specifies.
- **The Architectural Hazard:** As established in the SRM Access Check algorithm, the object Owner retains an immutable mathematical right to modify the DACL (`WRITE_DAC`). A user with `SeTakeOwnershipPrivilege` seizes ownership of a file, registry key, or directory object, grants themselves `GenericAll`, and takes unconstrained control.

5.5 `SeLoadDriverPrivilege`

- **Constant:** `SE_LOAD_DRIVER_NAME`
- **Default Holders:** Local `Administrators` group.
- **The Kernel Capability:** Allows a caller to register and load a kernel-mode device driver into memory via the native API `NtLoadDriver`.
- **The Architectural Hazard:** Allows immediate transition from user-mode execution to kernel-mode (Ring-0) execution. Once executing in Ring-0, the caller controls the processor hardware, can disable Kernel Patch Protection (PatchGuard), patch kernel access check routines, and hide processes from endpoint detection systems.

6. Built-in Groups & Scopes in Active Directory

Active Directory structures security groups into three distinct **Group Scopes** to balance replication performance across multiple domains against cross-domain resource access:



6.1 The AGDLP Architecture

Microsoft formalizes group nesting using the **AGDLP** design pattern:

$$\textbf{A}\text{ccounts} \longrightarrow \textbf{G}\text{lobal Groups} \longrightarrow \textbf{D}\text{omain } \textbf{L}\text{ocal Groups} \longrightarrow \textbf{P}\text{ermissions}$$

1. User **Accounts** (`ahmed`) are placed into a **Global Group** representing their job function (`GG-Engineering`).
2. The Global Group is nested inside a **Domain Local Group** representing a resource permission (`DL-FileShare-Read`).

3. The **Permission** (DACL) on the file share is granted directly to the Domain Local Group.

6.2 The Architectural Risk of Legacy Operator Groups

Active Directory contains several built-in administrative groups created in the Windows NT era. In modern Active Directory environments, **these groups represent catastrophic delegation risks**:

Legacy Group Name	RID	Inherent Privilege / Architectural Risk	Escalation Path to Domain Admin
Account Operators	`548`	Can create, modify, and delete user accounts, computers, and groups across the domain (except protected groups).	Modifies a service account or machine account, resets passwords, or assigns Kerberos SPNs to achieve domain compromise.
Server Operators	`549`	Can log on locally to Domain Controllers, create/stop services on DCs, and format system drives.	Logs into a DC, creates a service executing as `SYSTEM`, or replaces a service binary to achieve immediate Tier-0 control.
Backup Operators	`551`	Members are granted `SeBackupPrivilege` and `SeRestorePrivilege` on all Domain Controllers.	Logs into a DC, opens `%SystemRoot%\NTDS\ntds.dit` and the `SYSTEM` hive via backup semantics, and extracts the entire domain password database.
Print Operators	`550`	Can log on locally to Domain Controllers and manage printers and drivers.	Exploits local print spooler interfaces on a DC to achieve code execution as `SYSTEM`.

> **Architectural Law:** Membership in **Account Operators, Server Operators, or Backup Operators is functionally equivalent to Domain Admins.** These legacy groups must remain completely empty in modern enterprise environments.

7. Defensive Telemetry, Detection Engineering & Hardening

7.1 Security Event Telemetry

Privilege assignments and token elevations emit distinct telemetry in Windows Security logs:

Event ID	Log Name	Operational Meaning	Strategic Significance
4672	Security	Special privileges assigned to new logon.	**High-Value:** Emitted during logon if the user's token contains sensitive privileges (`SeDebugPrivilege`, `SeBackupPrivilege`,

Event ID	Log Name	Operational Meaning	Strategic Significance
			`SeImpersonate Privilege`).
4673	Security	A privileged service was called.	Logged when a process attempts to exercise a sensitive privilege (e.g., calling `AdjustToken Privileges` to enable `SeDebugPrivilege`).
4627	Security	Group membership information.	Emitted alongside logon events (Event 4624), listing all expanded group SIDs embedded within the user's token.
4728 / 4732	Security	A member was added to a security group.	Critical for monitoring additions to built-in operator groups or Domain Admins.

7.2 Detection Heuristics & Hunting Logic

1. Suspicious Special Privilege Assignment (KQL):

Detects logon events granting Tier-0 privileges (`SeDebugPrivilege` or `SeBackupPrivilege`) to non-administrative accounts:

```
SecurityEvent
| where EventID == 4672
| where PrivilegeList has "SeDebugPrivilege" or PrivilegeList has "SeBackupPrivilege"
| where TargetUserName !in ("Administrator", "SYSTEM", "LOCAL SERVICE", "NETWORK SERVICE")
| where TargetUserName !endswith "$" // Exclude machine accounts
| project TimeGenerated, TargetUserName, Computer, PrivilegeList
```

2. Monitoring Legacy Operator Group Additions (KQL):

Alerts immediately whenever an account is added to high-risk legacy operator groups:

```
SecurityEvent
| where EventID in (4728, 4732, 4756) // Group Member Added events
| where TargetUserName in ("Account Operators", "Server Operators", "Backup Operators", "Print Operators")
| project TimeGenerated, SubjectUserName, TargetUserName, MemberName
```

7.3 Enterprise Hardening Blueprint

Securing runtime authorization requires stripping unnecessary privileges and enforcing credential hygiene:

1. **Empty Legacy Operator Groups Permanently:**
   - Ensure that `Account Operators`, `Server Operators`, `Backup Operators`, and `Print Operators` have **zero members**.
   - Delegate specific, granular permissions (e.g., password reset over specific OUs) via standard Active Directory DACLs rather than assigning accounts to built-in operator groups.
2. **Restrict Sensitive Privileges via User Rights Assignment:**
   - Configure GPO: `Computer Configuration → Windows Settings → Security Settings → Local Policies → User Rights Assignment`:

    - `Debug programs` (`SeDebugPrivilege`) → Restrict strictly to `Administrators` (or remove entirely on production endpoints).
    - `Back up files and directories` (`SeBackupPrivilege`) → Restrict strictly to dedicated backup service accounts.
    - `Impersonate a client after authentication` (`SeImpersonatePrivilege`) → Restrict strictly to service accounts requiring it.
3. **Enforce the "Protected Users" Security Group:**
    - Add all domain administrative accounts to the **Protected Users** group. This prevents credentials from being cached in plaintext in LSASS memory, eliminating `SeDebugPrivilege` memory scraping vectors.
4. **Enforce UAC and Disable Token Filtering Exceptions:**
    - Ensure User Account Control is enabled across all endpoints.
    - Restrict local administrative accounts from using network logons via registry key `LocalAccountTokenFilterPolicy = 0`, forcing local administrator authentications over the network to strip administrative privileges.

8. Authoritative Specifications, RFCs & Cross-Vault Links

8.1 Authoritative Specifications

- **[MS-DTYP]:** *Windows Data Types* — Section 2.4.5 (`TOKEN`) and Section 2.4.4 (`SECURITY_IDENTIFIER`). The definitive specification for access tokens, SID structures, and privilege LUIDs.
- **[MS-KILE]:** *Kerberos Protocol Extensions* — Details how the PAC is parsed by LSASS to construct the user's Access Token.
- **[MS-PAC]:** *Privilege Attribute Certificate Data Structure* — Documents the group and identity buffers used in token generation.

AD-Integrated DNS Architecture — The Identity Resolution Backbone

> **Architectural Mission:** Master the internal mechanics of Active Directory-Integrated Domain Name System (ADIDNS). Understand why Active Directory is architecturally non-viable without DNS, analyze how directory partitions host DNS zones as LDAP objects within `DomainDnsZones` and `ForestDnsZones`, dissect the Domain Controller (DC) Locator process and SRV record resolution, deconstruct Secure Dynamic Updates via GSS-TSIG, examine the scavenging and tombstoning lifecycle, and evaluate how directory access control lists (DACLs) on DNS nodes create structural network interception surfaces.

1. Architectural Foundations: Why Active Directory Cannot Function Without DNS

1.1 From NetBIOS Flat Broadcasts to Hierarchical Service Discovery

In the pre-Active Directory Windows NT 4.0 era, name resolution relied on flat, broadcast-heavy mechanisms: **NetBIOS over TCP/IP Name Service (NBT-NS)** and the **Windows Internet Name Service (WINS)**. These legacy protocols were fundamentally unsuited for modern distributed enterprise computing:

- They possessed no concept of organizational hierarchy or domain trees.
- They generated severe broadcast traffic storms across subnets.
- Most critically, they could not answer the core service discovery question demanded by modern distributed directories:

> *"Which specific server in this enterprise hosts the Kerberos Key Distribution Center (KDC) and LDAP directory services for my domain, and which of those servers is located closest to my physical network subnet?"*

When Microsoft engineered Active Directory Domain Services (AD DS), it made the **Domain Name System (DNS)** the mandatory, foundational backbone of the entire identity plane. In Active Directory, DNS is not merely a utility for translating hostnames to IP addresses; **it is the service discovery, topology mapping, and authentication routing engine for the entire forest**.

1.2 Standalone BIND vs. Active Directory-Integrated DNS (ADIDNS)

Traditional DNS implementations (such as standalone Berkeley Internet Name Domain [BIND] or standard file-backed DNS servers) manage zones via static text files stored on local disks. In an enterprise environment, this traditional model introduces critical single points of failure:

Architectural Dimension	Standalone File-Backed DNS (BIND / File-Based)	Active Directory-Integrated DNS (ADIDNS)
Storage Engine	Flat text files (`named.conf`, zone text files) stored on local disk.	**Directory Objects inside `ntds.dit`** (Application Naming Contexts).
Replication Topology	Single-master zone transfers (AXFR / IXFR) from a single Primary server.	**Multi-Master Replication** via the Directory Replication Service Remote Protocol (DRSUAPI).
Availability / Fault Tolerance	If the Primary DNS server goes offline, zone updates fail globally.	**Zero Single Point of Failure.** Every Domain Controller running DNS is a writable primary.
Record Security	Coarse zone-level permissions or IP-based access control.	**Granular Per-Record DACLs** via Active Directory Security Descriptors (`nTSecurityDescriptor`).
Update Authentication	Shared static TSIG keys or unauthenticated updates.	**GSS-TSIG (RFC 3645)** authenticated dynamically via **Kerberos tickets**.

By storing DNS zones directly inside Active Directory database partitions, Microsoft eliminated the single-master bottleneck of DNS, bound name resolution directly into the Kerberos authentication boundary, and granted every individual DNS record an enterprise-grade security descriptor.

1.3 Terminology & Core Primitives (Zero-Acronym Onboarding)

- **ADIDNS (Active Directory-Integrated DNS):** A DNS zone whose database records are stored directly within the Active Directory database (`ntds.dit`) and replicated via multi-master directory replication.
- **SRV Record (Service Location Record):** A standardized DNS record type (RFC 2782) that specifies the hostname, port number, priority, and weight of servers providing specific network services (such as LDAP or Kerberos).

- **DC Locator:** The programmatic algorithm executing inside the Windows Netlogon service (`netlogon.dll` / `DsGetDcName`) that discovers, pings, and selects the optimal Domain Controller for an endpoint.
- **Dynamic DNS (DDNS):** An extension to DNS (RFC 2136) allowing client endpoints to automatically register and update their own A (IPv4) and AAAA (IPv6) address records with the DNS server.
- **GSS-TSIG (Generic Security Services Algorithm for Secret Key Transaction Authentication for DNS):** An RFC 3645 standard that uses the Generic Security Services Application Program Interface (GSS-API) and Kerberos tickets to cryptographically sign and authenticate Dynamic DNS updates.
- **Application Naming Context:** A partition within Active Directory (`DomainDnsZones` or `ForestDnsZones`) dedicated to replicating non-security application data strictly among specified Domain Controllers running DNS.
- **Scavenging:** The automated maintenance process running on Windows DNS servers that periodically purges obsolete, stale dynamic DNS records whose timestamps have expired.
- **Tombstoning:** The transitional lifecycle state where a deleted or scavenged DNS record is flagged as inactive (`dNSTombstoned = TRUE`) across directory replication before physical garbage collection.
- **GQBL (Global Query Block List):** A security feature in Windows DNS Server that refuses by default to resolve queries for high-risk protocol names such as `wpad` (Web Proxy Auto-Discovery) and `isatap` (Intra-Site Automatic Tunnel Addressing Protocol).

2. Walkthrough: The DC Locator Process in Action

2.1 The Enterprise Scenario Setup

To understand how an endpoint establishes identity trust upon booting up, trace this concrete sequence:

- **Client Workstation:** `WKSTN01.corp.local` (IP: `10.10.1.50`, physical location: Cairo Office).
- **Local Active Directory Site:** `HeadQuarters-Cairo` (Subnet: `10.10.0.0/16`).
- **Remote Active Directory Site:** `Europe-London` (Subnet: `10.20.0.0/16`).
- **Domain Controllers:**
  - `DC01.corp.local` (IP: `10.10.1.10`, located in `HeadQuarters-Cairo`).
  - `DC02.corp.local` (IP: `10.20.1.10`, located in `Europe-London`).

When `WKSTN01` boots up and an employee prepares to log in, **`WKSTN01` does not have any hardcoded IP address for a Domain Controller**. It relies entirely on the **DC Locator** algorithm executed by `netlogon.dll`.

2.2 Diagram: The DC Locator Resolution Sequence



2.3 The Critical SRV Record Hierarchy

When Active Directory initializes, the Netlogon service on every Domain Controller automatically compiles and registers a structured hierarchy of SRV records. These records are published under the **`_msdcs`** subdomain (e.g., `_msdcs.corp.local`):

1. **Generic Domain Controller Locator:** `_ldap._tcp.dc._msdcs.corp.local`
   - Maps to port `389`. Used by any domain client to locate any available DC in the domain.
2. **Site-Specific Domain Controller Locator:** `_ldap._tcp.<SiteName>._sites.dc._msdcs.corp.local`
   - Example: `_ldap._tcp.HeadQuarters-Cairo._sites.dc._msdcs.corp.local`.
   - Restricts resolution strictly to Domain Controllers physically located within the caller's geographic site, eliminating cross-WAN latency and bandwidth bottlenecks.
3. **Kerberos Key Distribution Center Locator:** `_kerberos._tcp.dc._msdcs.corp.local`
   - Maps to port `88`. Used by Kerberos clients to locate the KDC for ticket granting.
4. **Primary Domain Controller (PDC) Emulator Locator:** `_ldap._tcp.pdc._msdcs.corp.local`

- Locates the single DC holding the PDC Emulator Flexible Single Master Operation (FSMO) role. Essential for urgent password change verification, GPO edits, and domain time synchronization.

5. **Global Catalog Locator:** `_gc._tcp.<SiteName>._sites.corp.local`
   - Maps to port `3268`. Used to locate Global Catalog servers for forest-wide identity searches and universal group membership expansion during logon.

3. The Physical Reality: How DNS Records Exist in LDAP

3.1 The Storage Partitions: `DomainDnsZones` and `ForestDnsZones`

In Windows 2000, AD-Integrated DNS zones were stored inside the standard Domain Naming Context (`DC=corp,DC=local`). This created an operational flaw: DNS records replicated to *every* Domain Controller in the domain, even if that DC was not configured as a DNS server.

Starting in Windows Server 2003, Microsoft transitioned ADIDNS into **Application Naming Contexts**:

```
+-------------------------------------------------------------------------------------------------+
|                          ADIDNS DIRECTORY PARTITION ARCHITECTURE                                 |
+-------------------------------------------------------------------------------------------------+

  1. ForestDnsZones PARTITION
  ═══════════════════════════════════════════════════════════════════════════════════════════════
  - Directory Path   : DC=ForestDnsZones,DC=corp,DC=local
  - Replication Scope: Replicates to EVERY Domain Controller running DNS across the ENTIRE forest.
  - Primary Contents : The forest-wide root discovery zone:
                       _msdcs.corp.local

  2. DomainDnsZones PARTITION
  ═══════════════════════════════════════════════════════════════════════════════════════════════
  - Directory Path   : DC=DomainDnsZones,DC=corp,DC=local
  - Replication Scope: Replicates to all Domain Controllers running DNS within THIS specific domain.
  - Primary Contents : The domain's forward and reverse lookup zones:
                       corp.local
                       10.10.in-addr.arpa
+-------------------------------------------------------------------------------------------------+
```

3.2 The `dnsNode` Object Class

Every DNS record stored inside an AD-Integrated zone is an actual, physical object in the directory database (`ntds.dit`).

- **Object Class:** `dnsNode`
- **Container Path:** `DC=<RecordName>,DC=corp.local,CN=MicrosoftDNS,DC=DomainDnsZones,DC=corp,DC=local`

For example, workstation `WKSTN01`'s address record is stored at: `DC=WKSTN01,DC=corp.local,CN=MicrosoftDNS,DC=DomainDnsZones,DC=corp,DC=local`

Critical Attributes of a `dnsNode` Object:

1. `dnsRecord` **(Octet String / Binary BLOB):** Stores the raw binary representation of the DNS resource record per **[MS-DNSP] §2.2.2.2.4**:
   - Record Type (e.g., `0x0001` for Type A, `0x0021` for Type SRV, `0x0005` for Type CNAME).
   - Serial Number and Versioning.
   - Time-to-Live (TTL).

    - IP Address or Target FQDN string.
2. `dNSTombstoned` **(Boolean):** Set to `TRUE` when a record has expired or been deleted, marking it for scavenging.
3. `nTSecurityDescriptor` **(Binary Security Descriptor): Every single DNS record in ADIDNS possesses its own Discretionary Access Control List (DACL)!**
    - When `WKSTN01$` dynamically registers its DNS record, `WKSTN01$`'s computer SID is stamped as the **Owner** of the `dnsNode` object, granting it full write permissions (`GenericAll`) over its own record.
    - Standard domain users (`Authenticated Users`) do not have write access to `WKSTN01`'s record, preventing a malicious user from modifying another machine's IP address.

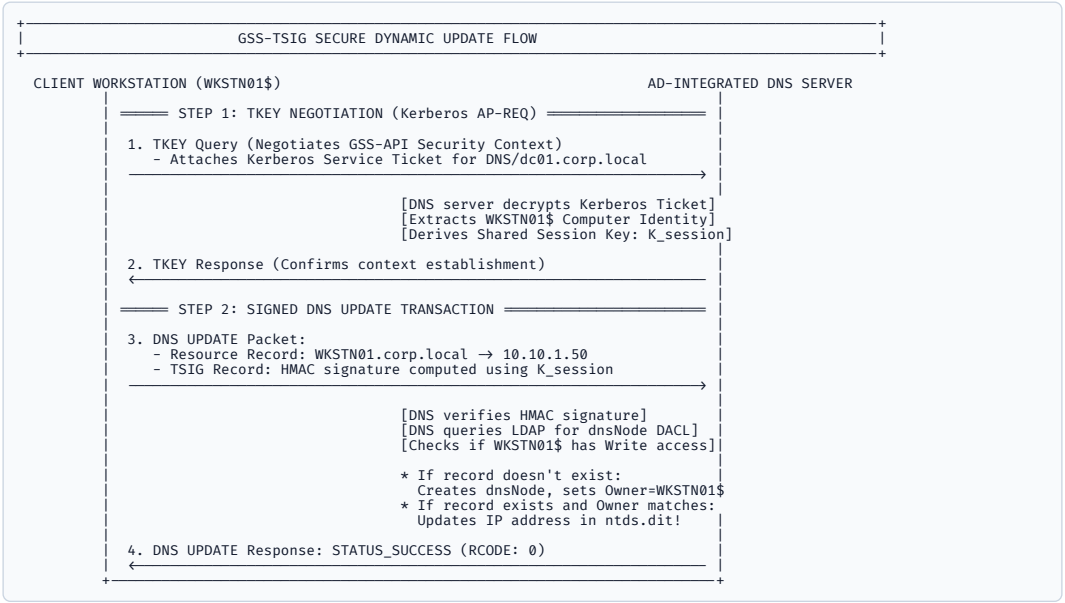
4. Dynamic Updates: Non-Secure vs. Secure (GSS-TSIG)

4.1 Non-Secure Dynamic Updates (RFC 2136)

- **The Mechanism:** An unauthenticated DNS client sends a raw UDP `DNS UPDATE` packet (Opcode 5) to the DNS server instructing it to add or modify an A record: `"Map server01.corp.local to 10.10.1.99"`
- **The Failure Mode:** Non-secure updates perform **zero authentication**. The DNS server does not verify who sent the packet. Any host on the network can claim any hostname, overwrite records belonging to Domain Controllers, and hijack corporate traffic. Non-secure updates should **never** be permitted in an enterprise environment.

4.2 Secure Dynamic Updates via GSS-TSIG (RFC 3645)

Windows Active Directory environments enforce **Secure Dynamic Updates Only**. Under this policy, dynamic DNS updates are cryptographically bound to Kerberos:



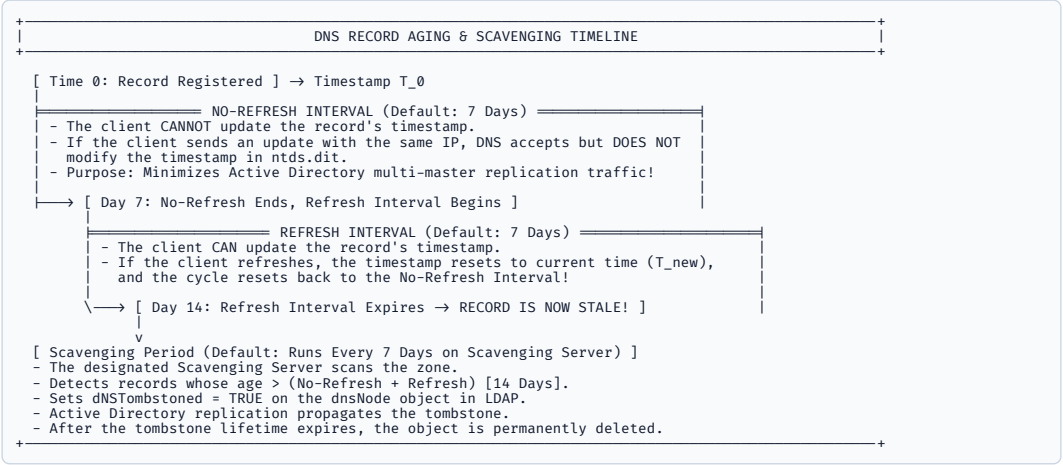
1. **TKEY Exchange:** The client contacts the DNS server and initiates a Transaction Key (`TKEY`) exchange using Kerberos (`GSS-SPNEGO`), presenting a Service Ticket for the SPN `DNS/dc01.corp.local`.
2. **Session Key Derivation:** Both parties derive an ephemeral symmetric session key (K_{session}).
3. **Signed Transaction (`TSIG`):** The client transmits the `DNS UPDATE` packet with a Transaction Signature (`TSIG`) record containing an HMAC calculated using the session key.
4. **Directory ACL Enforcement:** The DNS server verifies the HMAC signature, confirms the caller is genuinely `WKSTN01$`, and evaluates the DACL on the `dnsNode` object in LDAP. If the caller has permission, the record is committed to `ntds.dit`.

5. Zone Scavenging and the Tombstoning Lifecycle

In a dynamic enterprise network where laptops change IP addresses across Wi-Fi networks and virtual machines are frequently decommissioned, DNS zones would rapidly accumulate thousands of stale, orphaned records without automated cleanup.

5.1 The Scavenging Time Windows

Windows DNS Server manages dynamic record cleanup through **Aging and Scavenging**, governed by two sequential intervals:



- **Static Records (Timestamp = 0):** Records created manually by administrators have a timestamp of `0`. **Static records are never scavenged**, ensuring critical servers, routers, and infrastructure records persist indefinitely.

6. Architectural Vulnerability Primitives (Zero Tool Dumping)

Security vulnerabilities in Active Directory-Integrated DNS stem directly from **default directory permissions, unhardened fallbacks, and zone configuration oversights**.

6.1 The Open Creation Boundary: `CreateChild` for `Authenticated Users`

- **The Design Principle:** To allow any domain-joined laptop or workstation to dynamically register its own hostname, Microsoft configured the default DACL on ADIDNS zone containers (`DC=corp.local,CN=MicrosoftDNS, ...`) with: `Grant Authenticated Users: CreateChild (dnsNode)`
- **The Friction Point:** The permission is **not restricted to computer accounts**. Any standard domain user account (`CORP\ahmed`) possesses the exact same directory right to create new `dnsNode` objects inside the corporate zone!
- **The Architectural Consequence:** An adversary with standard user credentials can query the zone for non-existent records and register arbitrary A records (e.g., creating `legit-portal.corp.local` pointing to an attacker-controlled IP). When internal corporate users attempt to navigate to that portal, DNS resolves it directly to the attacker's server, enabling credential harvesting and man-in-the-middle attacks.

6.2 Wildcard Injection (`*` Records)

- **The Mechanics:** In DNS, an asterisk (`*`) represents a wildcard record. If a client queries DNS for a name that does not exist (e.g., `internal-reporting-app.corp.local`), the DNS server checks if a wildcard record exists

(`*.corp.local`). If present, the DNS server returns the IP address specified in the wildcard record.

- **The Architectural Consequence:** Because standard users can create records in ADIDNS, an attacker creates an object named `*` in the zone pointing to their own machine. Any mistyped URL, failed internal service connection, or unresolved internal hostname across the entire enterprise resolves directly to the attacker.

6.3 The WPAD Hijack Surface

- **The Protocol:** Windows systems utilize **Web Proxy Auto-Discovery (WPAD)** to automatically discover corporate web proxies by querying DNS for the hostname: `wpad.corp.local`
- **The Friction Point:** If an organization does not utilize a proxy server, the record `wpad.corp.local` does not exist in the zone.
- **The Defense & Bypass:** Microsoft introduced the **Global Query Block List (GQBL)**, instructing Windows DNS Server to refuse resolution for `wpad` by default. However, if an administrator disables the GQBL, or if an attacker registers the record directly as a `dnsNode` object in LDAP, endpoints automatically route corporate HTTP/HTTPS traffic through the attacker's listener, capturing Net-NTLM challenge-response authentications.

7. Defensive Telemetry, Detection Engineering & Hardening

7.1 DNS Security Event Telemetry

Monitoring DNS activity requires collecting both Directory Service audit events and Windows DNS Server audit logs:

Event ID	Log Name	Operational Meaning	Strategic Significance
5136	Security (DC)	A directory service object was modified.	**High-Value:** Generated when a `dnsNode` object's IP address (`dnsRecord`) or permissions (`nTSecurityDescriptor`) are modified in LDAP.
5137	Security (DC)	A directory service object was created.	Tracks the creation of new `dnsNode` objects (detects unauthorized record creation or wildcard injection).
541	DNS Server / Audit	A dynamic update was processed.	Logs client IP, updated record name, and update status.

7.2 Detection Heuristics & Hunting Logic

1. Detecting Wildcard or WPAD DNS Node Creation (KQL):

Detects the creation of high-risk DNS records within Active Directory Application Partitions:

```
SecurityEvent
| where EventID == 5137
| where ObjectDN has "CN=MicrosoftDNS"
| where ObjectDN has "DC=*" or ObjectDN has "DC=wpad"
| project TimeGenerated, SubjectUserName, SubjectDomainName, ObjectDN
```

2. Detecting Non-Machine Accounts Creating DNS Records (KQL):

Standard dynamic DNS updates are generated by computer accounts ending with `$`. Flags any record creation initiated by human user accounts:

```
SecurityEvent
| where EventID == 5137
| where ObjectDN has "CN=MicrosoftDNS"
| where SubjectUserName !endswith "$" // Flag human user accounts
| where SubjectUserName !in ("SYSTEM", "Administrator")
| project TimeGenerated, SubjectUserName, ObjectDN
```

7.3 Enterprise Hardening Blueprint

Securing AD-Integrated DNS requires enforcing cryptographic update policies and cleaning default permissions:

1. **Enforce “Secure Only” Dynamic Updates:**
   - On all AD-Integrated zones, set Dynamic Updates to **Secure only**. Never allow non-secure updates.
2. **Restrict Container Creation Rights:**
   - Modify the DACL on `CN=MicrosoftDNS,DC=DomainDnsZones,DC=corp,DC=local` and `CN=MicrosoftDNS,DC=ForestDnsZones,DC=corp,DC=local`.
   - Remove `CreateChild` rights from `Authenticated Users` or restrict them to a specific security group composed exclusively of domain computer accounts.
3. **Verify Global Query Block List (GQBL) Enforcement:**
   - Ensure the Global Query Block List is active on all DNS servers:

```
    Get-DnsServerGlobalQueryBlockList
    # Verify that 'wpad' and 'isatap' are listed and enabled
```

1. **Pre-Create a Static WPAD Record:**
   - If your organization does not use WPAD, create an empty static `wpad.corp.local` A record pointing to a non-routable loopback address (`127.0.0.1`) with a static timestamp of `0`. This permanently prevents attackers from creating the record in the zone.

8. Authoritative Specifications, RFCs & Cross-Vault Links

8.1 Authoritative Specifications

- **RFC 2782:** *A DNS RR for specifying the location of services (DNS SRV)* — The foundation for Active Directory service discovery.
- **RFC 2136:** *Dynamic Updates in the Domain Name System (DNS UPDATE)*.
- **RFC 3645:** *Generic Security Service Algorithm for Secret Key Transaction Authentication for DNS (GSS-TSIG)* — Formal specification for Kerberos-signed DNS updates.
- **[MS-DNSP]:** *Domain Name System (DNS) Server Management Protocol* — Microsoft specification detailing the binary structure of `dnsRecord` attributes and `dnsNode` object classes in LDAP.

Certificate Services Architecture (AD CS) — Foundations

> **Architectural Mission:** Master the internal mechanics of Active Directory Certificate Services (AD CS) and enterprise Public Key Infrastructure (PKI). Understand why asymmetric cryptography was introduced to augment symmetric Kerberos and NTLM authentications, dissect the Active Directory PKI directory containers in the Configuration partition, deconstruct Certificate Templates and Extended Key Usage (EKU) object identifiers, analyze enrollment protocols from MS-ICPR to Web Enrollment, and evaluate why misconfigured Subject Alternative Name (SAN) flags create mathematical identity impersonation.

1. Architectural Foundations: Why Kerberos Needed an Asymmetric Companion

1.1 The Operational Limits of Symmetric Credentials

For the first decade of Active Directory, enterprise authentication relied almost exclusively on symmetric secrets: human passwords, NTLM password hashes, and symmetric Kerberos keys. While computationally efficient, pure symmetric identity architecture suffers from three fundamental enterprise limits:

1. **Symmetric Secret Duplication:** In symmetric authentication, the authenticating authority (the KDC) must store the identical secret—or a mathematical equivalent—as the user. If an adversary compromises the database (`ntds.dit`), every credential in the forest is exposed simultaneously.
2. **Lack of Hardware Cryptographic Binding:** Symmetric passwords cannot be securely locked inside hardware cryptographic chips (such as Smart Cards, YubiKeys, or Trusted Platform Modules [TPMs]) while retaining flexibility for multi-system Single Sign-On (SSO).
3. **Third-Party & Cross-Boundary Identity Proof:** A user cannot present a Kerberos symmetric ticket to an external non-domain VPN gateway, an 802.1X wireless access point, or a code-signing repository without establishing complex cross-forest trusts or exposing long-term domain keys.

1.2 The Asymmetric Solution: Public Key Infrastructure (PKI)

Public-key cryptography resolves these limitations through mathematically coupled keypairs:

- **The Private Key:** Retained exclusively by the principal (user, computer, or service), sealed inside hardware storage (TPM) or local operating system protected memory. The private key is **never transmitted across the network wire**.
- **The Public Key:** Freely published to the world, used to encrypt data for the owner or verify the owner's digital signatures.

A **Digital Certificate (X.509 v3)** is an immutable digital credential that cryptographically binds an identity (e.g., `ahmed@corp.local`) to a specific Public Key, authenticated by the digital signature of a trusted **Certificate Authority (CA)**.

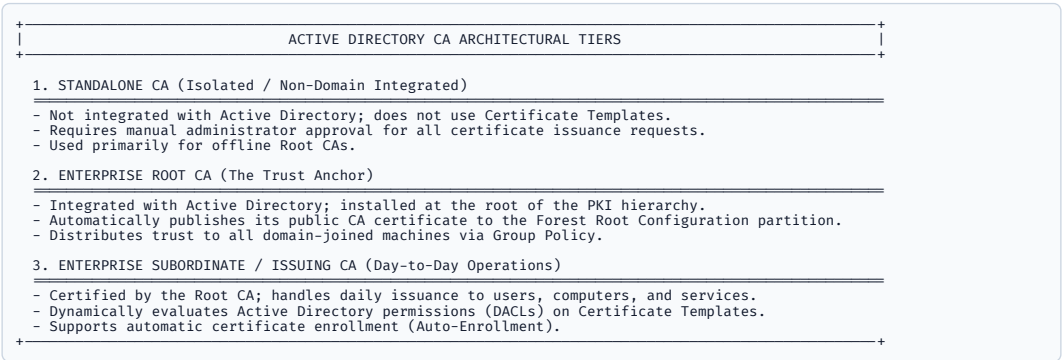
1.3 What is Active Directory Certificate Services (AD CS)?

Active Directory Certificate Services (AD CS) is Microsoft's enterprise implementation of PKI, natively embedded within the Active Directory directory service. Rather than operating as an isolated security appliance, AD CS deeply integrates with:

- **The Active Directory Database:** Stores CA policies, revocation points, and certificate templates directly inside the forest Configuration partition.
- **Kerberos (PKINIT):** Bridges asymmetric certificates into the Kerberos protocol, allowing users to exchange certificate proofs for standard Kerberos Ticket Granting Tickets (TGTs).
- **Group Policy Objects (GPOs):** Automates certificate issuance, renewal, and trust distribution across tens of thousands of endpoints via **Auto-Enrollment**.

1.4 Certificate Authority Roles

AD CS defines three architectural tiers of Certificate Authorities:



1.5 Terminology & Core Primitives (Zero-Acronym Onboarding)

- **CA (Certificate Authority):** The trusted server responsible for issuing, signing, managing, and revoking X.509 digital certificates.
- **CSR (Certificate Signing Request):** A standardized, cryptographically signed data block (PKCS#10) sent by a client to a CA containing the client's public key and requested identity attributes.

- **Certificate Template:** A directory object stored in Active Directory that defines the enrollment rules, cryptographic constraints, validity lifetimes, and permitted uses of certificates issued from it.
- **EKU (Extended Key Usage):** An X.509 certificate extension containing Object Identifiers (OIDs) that define what actions the certificate is authorized to perform (such as Client Authentication, Code Signing, or Server Authentication).
- **SAN (Subject Alternative Name):** An X.509 extension that specifies additional identities bound to the public key (such as User Principal Names [UPNs], DNS hostnames, IP addresses, or email addresses).
- **PKINIT (Public Key Cryptography for Initial Authentication in Kerberos):** An RFC 4556 extension allowing a client to authenticate to the Kerberos KDC using an X.509 certificate instead of a password.
- **NTAuthStore (`NTAuthCertificates`):** A critical container object in Active Directory (`CN=NTAuthCertificates,CN=Public Key Services, ...`) containing the certificates of CAs explicitly authorized to issue certificates for domain authentication (PKINIT and smart card logon).
- **CRL (Certificate Revocation List):** A cryptographically signed list published periodically by a CA listing certificates that have been revoked prior to their scheduled expiration.
- **OCSP (Online Certificate Status Protocol):** A lightweight HTTP-based protocol (RFC 6960) allowing clients to query the revocation status of a specific certificate in real time without downloading full CRLs.
- **AIA (Authority Information Access):** Metadata embedded within a certificate pointing to URLs where the issuing CA's certificate can be downloaded to build certificate validation chains.
- **CDP (CRL Distribution Point):** Metadata embedded within a certificate specifying the LDAP or HTTP network paths where the CA's current CRL can be retrieved.

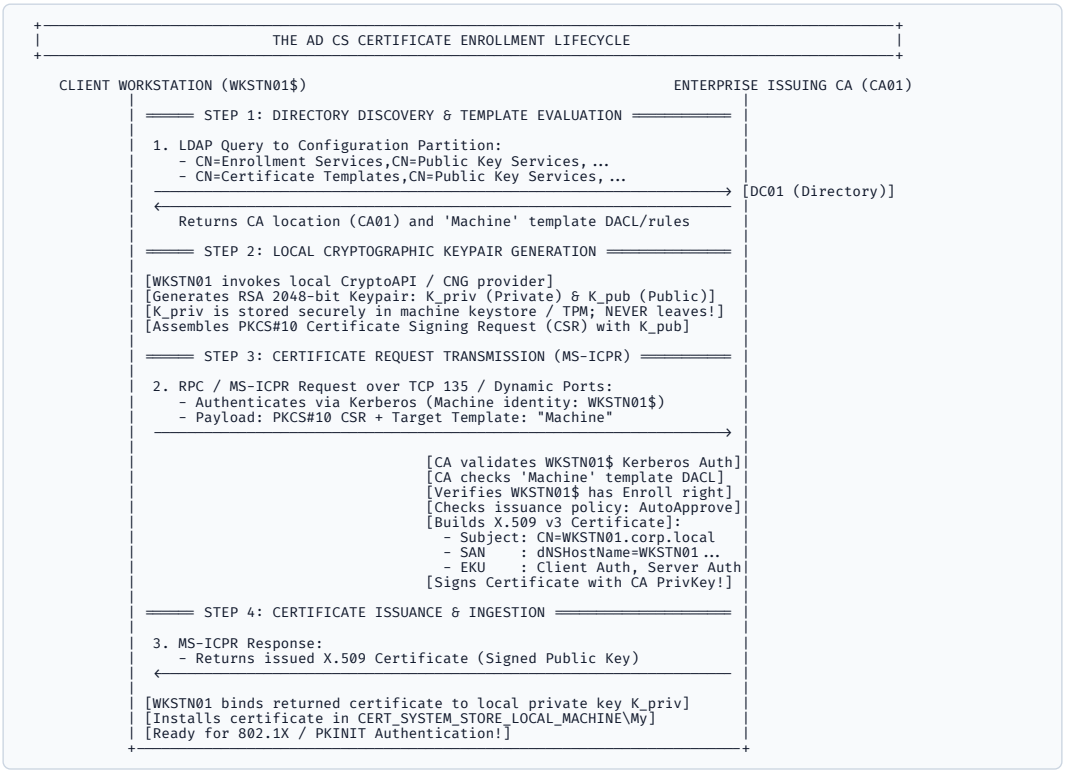
2. Walkthrough: Workstation Certificate Enrollment

2.1 The Scenario Setup

To understand how PKI operates on the wire, trace this enterprise enrollment:

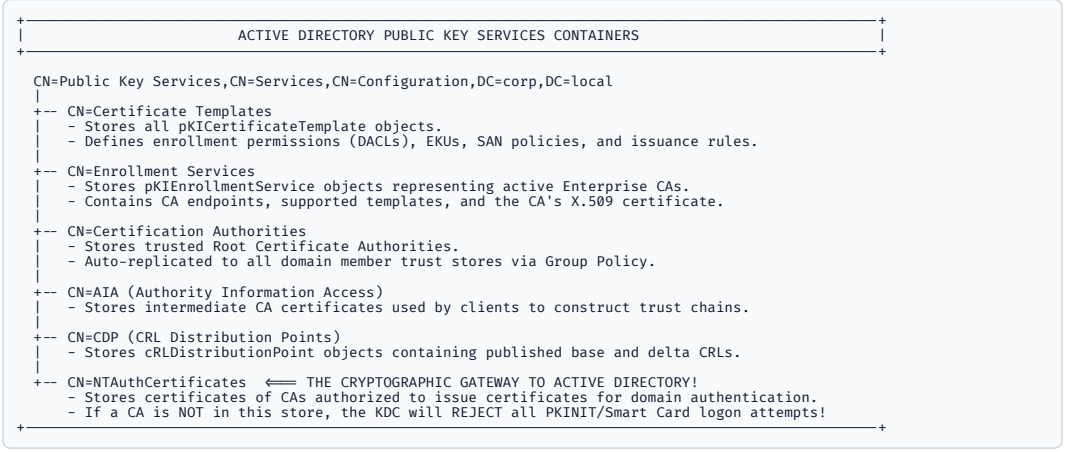
- **Requesting Endpoint:** Computer account `WKSTN01$` on workstation `WKSTN01.corp.local`.
- **Issuing Certificate Authority:** Enterprise CA `CORP-CA01` running on `CA01.corp.local`.
- **Target Objective:** `WKSTN01` needs an X.509 computer certificate based on the enterprise template `Machine` to authenticate against corporate 802.1X Wi-Fi networks and secure internal communication.

2.2 Diagram: The Certificate Enrollment Lifecycle



3. The Active Directory PKI Containers in the Configuration Partition

Enterprise PKI does not store its configuration in local registry files; it stores it in the **Active Directory Configuration Partition** (`CN=Configuration,DC=corp,DC=local`), replicating forest-wide to every Domain Controller under: `CN=Public Key Services,CN=Services,CN=Configuration,DC=corp,DC=local`



The Paramount Significance of the `NTAuthStore`

The `NTAuthCertificates` object (`CN=NTAuthCertificates,CN=Public Key Services,...`) is the absolute root of authentication trust in Active Directory:

- When a user or computer attempts a certificate-based logon (PKINIT or Smart Card), the Domain Controller receives the client's certificate.
- The DC checks whether the certificate chains back to a CA whose public certificate is present in the **`cACertificate` attribute of the `NTAuthCertificates` object**.
- If the issuing CA is absent from `NTAuthCertificates`, the DC aborts the logon with error `KDC_ERR_PADATA_TYPE_NOSUPP` or `STATUS_PKINIT_FAILURE`, **even if the CA is fully trusted for standard SSL/TLS**.

4. Certificate Templates: Structure, EKUs & SAN Flags

A Certificate Template is a blueprint for certificate issuance stored as an Active Directory object of class **`pKICertificateTemplate`**.

4.1 Critical Template Attributes

- **`msPKI-Cert-Template-OID`:** A unique Object Identifier that identifies the template.
- **`pKIExtendedKeyUsage` (EKU OIDs):** Defines the cryptographic purposes allowed for certificates minted from this template.
- **`msPKI-Certificate-Name-Flag` (32-bit Bitmask):** Controls how the CA constructs the Subject and Subject Alternative Name (SAN) of the certificate.
- **`msPKI-Enrollment-Flag`:** Controls auto-enrollment, key archival, and directory publication.
- **`nTSecurityDescriptor`:** The Active Directory DACL controlling which users, computers, or groups have permission to enroll (`Enroll` / `AutoEnroll`).

4.2 Extended Key Usage (EKU) Object Identifiers Demystified

An EKU specifies what application protocols will accept the certificate:

Common EKU Name	Object Identifier (OID)	Operational Significance
Client Authentication	`1.3.6.1.5.5.7.3.2`	**High-Value:** Permits using the certificate to authenticate as an Active Directory principal via **Kerberos PKINIT** or mutual TLS.
Smart Card Logon	`1.3.6.1.4.1.311.20.2.2`	**High-Value:** Permits interactive Windows desktop logon via PKINIT.
Any Purpose	`2.5.29.37.0`	**Unrestricted Super-EKU:** Valid for all cryptographic operations, including Client Auth, Server Auth, and Code Signing.

Common EKU Name	Object Identifier (OID)	Operational Significance
Certificate Request Agent	`1.3.6.1.4.1.311.20.2.1`	**Enrollment Agent:** Authorizes the holder to act as an enrollment proxy, requesting certificates on behalf of *other* users.
Server Authentication	`1.3.6.1.5.5.7.3.1`	Used by web and LDAP servers to secure incoming TLS connections (HTTPS/LDAPS).
Code Signing	`1.3.6.1.5.5.7.3.3`	Used to digitally sign executable binaries and PowerShell scripts.

4.3 Subject Alternative Name (SAN) Generation: The Core Divide

When a certificate is minted, who determines the identity written inside it? This is governed by the `msPKI-Certificate-Name-Flag` attribute:

Mode 1: Active Directory Supplies the Subject (Secure Default)

- **Flag Value:** `CT_FLAG_SUBJECT_REQUIRE_DIRECTORY_PATH` / `CT_FLAG_SUBJECT_REQUIRE_UPN` (`0×00400000` / `0×00800000`).
- **Mechanism:** The CA ignores any identity requested by the client. The CA inspects the client's authenticated Active Directory identity, queries `ntds.dit`, and builds the SAN directly from the client's verified directory attributes (`sAMAccountName`, `userPrincipalName`, `dNSHostName`). A standard user cannot request a certificate identifying them as someone else.

Mode 2: Enrollee Supplies the Subject (The Architectural Hazard)

- **Flag Value:** `CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT` (`0×00000001`).
- **Mechanism:** The CA explicitly instructs itself:

> *"Allow the requesting client to specify whatever Subject and Subject Alternative Name (SAN) they want inside their Certificate Signing Request (CSR)."*

5. Enrollment Protocols Demystified

Active Directory Certificate Services supports multiple transport protocols to accept certificate requests:

1. **MS-ICPR / DCOM (Remote Procedure Call):**
   - The native Windows enterprise enrollment protocol operating over RPC (TCP port 135 and dynamic ephemeral RPC ports). Uses `ICertRequestD2` interfaces authenticated via Kerberos.
2. **Web Enrollment (`/certsrv`):**
   - A legacy ASP web application hosted inside Internet Information Services (IIS) on the CA (accessible at `http://<CA>/certsrv/`). It allows users to submit CSRs through a web browser. Authenticates using HTTP Basic, NTLM, or Kerberos.
3. **CEP / CES (Certificate Enrollment Policy & Web Services):**

- Modern SOAP web services operating over HTTPS (RFC 5246) allowing endpoints outside the corporate network (or across untrusted forests) to query policies (CEP) and enroll certificates (CES).

4. **NDES / SCEP (Network Device Enrollment Service):**
   - Implements the Simple Certificate Enrollment Protocol (SCEP) to allow network routers, switches, and mobile devices that do not support Active Directory domain accounts to enroll certificates using one-time challenge passwords.

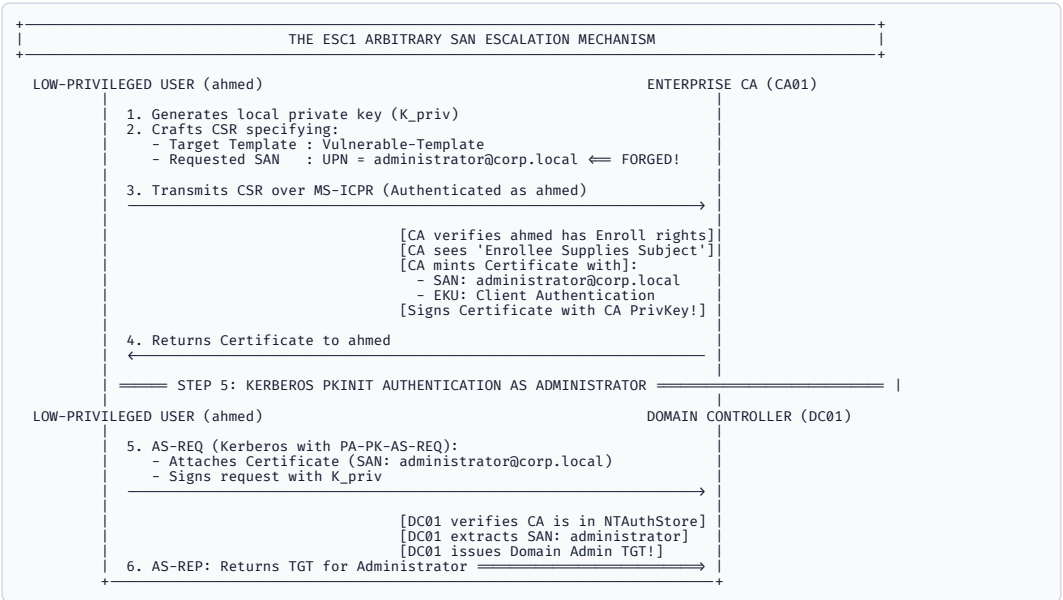
6. Architectural Implications: The Genesis of AD CS Vulnerabilities

Security vulnerabilities in AD CS are rarely software bugs in the Windows crypto engine. They are the mathematically inevitable consequence of **conflicting policy settings between certificate templates, directory ACLs, and transport protocols**.

6.1 Arbitrary SAN Injection (The Concept Behind ESC1)

Consider an Enterprise CA that publishes a Certificate Template with the following three simultaneous properties:

1. **Enrollment Permissions:** The template DACL grants `Enroll` to a broad group (e.g., `Domain Users`).
2. **Client Authentication EKU:** The template includes `Client Authentication` (`1.3.6.1.5.5.7.3.2`) or `Any Purpose` (`2.5.29.37.0`).
3. **Enrollee Supplies Subject:** The template has the `CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT` flag set (`0×00000001`).



Because the CA trusted the client to declare its own identity, and because Active Directory's Kerberos engine maps identities via the SAN, **a low-privileged user obtains an administrative Kerberos TGT without ever knowing the administrator's password.**

6.2 The Enrollment Agent Proxy Hazard (The Concept Behind ESC3)

If a template is configured with the **Certificate Request Agent** EKU (`1.3.6.1.4.1.311.20.2.1`), any user who enrolls in it receives an "Enrollment Agent" certificate. If a second template permits enrollment on behalf of others and requires an Enrollment Agent signature, the holder of the agent certificate signs a CSR for `administrator@corp.local` and submits it to the CA. The CA accepts the proxy request, minting a Client Authentication certificate for the administrator.

6.3 HTTP Web Enrollment NTLM Relay (The Concept Behind ESC8)

The AD CS Web Enrollment interface (`/certsrv`) runs inside IIS and historically supported NTLM authentication while having **Extended Protection for Authentication (EPA) disabled** and **HTTP (cleartext) enabled**.

- **The Attack Flow:** An adversary coerces an NTLM authentication from a Domain Controller (via the PrinterBug `MS-RPRN` or PetitPotam `MS-EFSR`).
- **The Relay:** The adversary relays the incoming NTLM authentication to the CA's `/certsrv/certfnsh.asp` endpoint over HTTP.
- **The Consequence:** The CA authenticates the caller as the Domain Controller machine account (`DC01$`) and issues a computer certificate identifying `DC01$`. The attacker uses this certificate via PKINIT to request a TGT as `DC01$` and executes a DCSync attack, completely compromising the domain.

7. Defensive Telemetry, Detection Engineering & Hardening

7.1 AD CS Event Telemetry

Monitoring certificate issuance operations requires enabling auditing on the Certificate Authority:

Event ID	Log Name	Operational Meaning	Strategic Significance
4886	Security (CA)	Certificate Services approved a certificate request and issued a certificate.	**Primary Issuance Telemetry:** Captures Requester Name, Certificate Template, and Subject Alternative Name (SAN).
4887	Security (CA)	Certificate Services approved a certificate request and issued a certificate.	Detailed record showing the exact attributes committed to the certificate.

Event ID	Log Name	Operational Meaning	Strategic Significance
4888	Security (CA)	Certificate Services denied a certificate request.	Identifies unauthorized enrollment attempts.
4898	Security (CA)	A Certificate Template was loaded.	Emitted when templates are updated or loaded into CA memory.
5136	Security (DC)	A directory service object was modified.	**High-Value:** Traces modifications to template objects under `CN=Certificate Templates` (e.g., adding `ENROLLEE_SUPPLIES_SUBJECT` or granting `Enroll` permissions).

7.2 Detection Heuristics & Hunting Logic

1. Detecting Mismatched SAN Requests (KQL):

Flags certificates issued where the Requester identity does not match the Subject Alternative Name (SAN) embedded inside the certificate:

```
SecurityEvent
| where EventID == 4887
| extend Requester = tostring(parse_json(EventData).Requester)
| extend SubjectAltName = tostring(parse_json(EventData).SubjectAlternativeName)
| where isnotempty(SubjectAltName)
| where Requester !has SubjectAltName // Flag identity divergence!
| project TimeGenerated, Requester, SubjectAltName, CertificateTemplate, Computer
```

7.3 Enterprise Hardening Blueprint

Securing AD CS requires eliminating hazardous template configurations and securing network endpoints:

1. **Eradicate `ENROLLEE_SUPPLIES_SUBJECT` on Authentication Templates:**
   - Audit all templates with Client Authentication or Smart Card Logon EKUs. Ensure the Subject Name is configured strictly to **Build from this Active Directory information** (`CT_FLAG_SUBJECT_REQUIRE_DIRECTORY_PATH`).
2. **Harden or Disable HTTP Web Enrollment (`/certsrv`):**
   - If `/certsrv` is not strictly required, uninstall the **Certification Authority Web Enrollment** role service entirely.
   - If required:
     - Enforce **HTTPS** universally.
     - Enable **Extended Protection for Authentication (EPA)** in IIS.
     - Disable NTLM authentication, forcing **Kerberos (Negotiate)** only.
3. **Audit and Restrict Template DACLs:**
   - Ensure standard users (`Domain Users`, `Authenticated Users`) do not possess `Enroll` or `AutoEnroll` permissions on powerful administrative templates.
   - Restrict write permissions (`GenericAll`, `WriteDacl`, `WriteProperty`) on `CN=Certificate Templates` strictly to **Domain Admins** and **Enterprise Admins**.

4. **Deploy Strong Certificate Mapping (KB5014754):**
    - Ensure all Domain Controllers enforce strong certificate mapping for PKINIT, verifying the presence of the non-malleable `szOID_NTDS_CA_SECURITY_EXT` (`1.3.6.1.4.1.311.25.2`) extension containing the caller's permanent Active Directory SID.

8. Authoritative Specifications, RFCs & Cross-Vault Links

8.1 Authoritative Specifications

- **[MS-WCEP] / [MS-ICPR]:** *Windows Client Certificate Enrollment Protocol & ICertPassage Remote Protocol* — Documents the native RPC enrollment interfaces.
- **[MS-CRTD]:** *Certificate Templates Structure* — Defines the Active Directory schema attributes, bitmasks, and OID structures of `pKICertificateTemplate` objects.
- **RFC 4556:** *Public Key Cryptography for Initial Authentication in Kerberos (PKINIT)* — The formal specification bridging X.509 certificates into Kerberos ticket granting.
- **RFC 5280:** *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*.

Trust Architecture — Relationships Between Domains

> **Architectural Mission:** Master the internal mechanics of Active Directory Trust relationships and cross-realm Kerberos authentication. Understand how distinct domains and forests federate identity without replicating directory databases, dissect the cryptographic anchor of the shared Trust Key and the Trusted Domain Object (TDO), trace the multi-hop Inter-Realm Referral TGT ticket exchange, deconstruct SID Filtering, Name Suffix Routing, and Quarantine boundaries, and analyze why intra-forest trusts fail to provide security isolation while cross-forest trusts enforce hard cryptographic perimeters.

1. Architectural Foundations: Identity Federation Without Database Replication

1.1 The Enterprise Federation Dilemma

In a growing enterprise or multinational corporation, organizations frequently operate multiple distinct Active Directory domains or separate forests (e.g., resulting from corporate mergers, regulatory geographical partitioning, or supply chain partnerships).

An architectural question arises:

> *"How can an employee in the European child domain (`eu.corp.local`) securely access an internal resource hosted on a server in the corporate headquarters domain (`dc01.corp.local`) without replicating the entire European user database into headquarters?"*

Duplicating directory databases across organizational boundaries is an engineering failure:

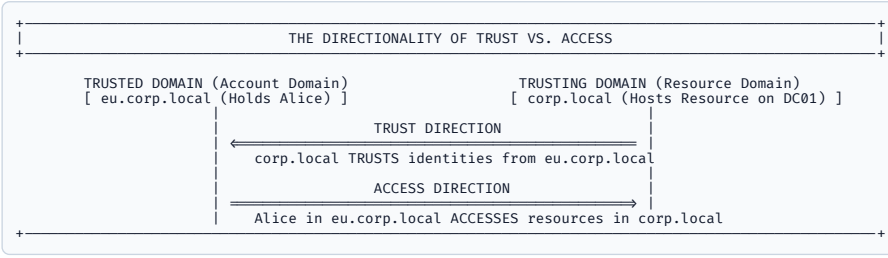
- It creates massive network bandwidth overhead across WAN links.
- It introduces directory naming collisions and schema conflicts.
- It destroys decentralized administrative autonomy and violates regional data sovereignty laws.

1.2 The Trust Concept: Cryptographic Authentication Agreements

To solve this dilemma, Active Directory implements **Trust Relationships**.

A Trust is an established cryptographic agreement between two independent Kerberos realms where the **Trusting Domain** agrees to honor authentication proofs minted by the **Trusted Domain**:

- **Trusting Domain (Resource Domain):** The domain hosting the resource (e.g., the file share, server, or application) that trusts another domain.
- **Trusted Domain (Account Domain):** The domain holding the user accounts whose identities are verified and vouched for.



> **The Directionality Rule: Access flows in the opposite direction of the trust arrow.** If Domain B trusts Domain A (Trust arrow: $B \longrightarrow A$), users in Domain A can access resources in Domain B.

1.3 The Trust Taxonomy

Active Directory supports several distinct categories of trust relationships:

Trust Category	Transitivity	Boundary Scope	Typical Operational Use Case
Parent-Child Trust	**Transitive**	Intra-Forest	Established automatically between parent and child domains (e.g., `corp.local` and `eu.corp.local`). Two-way transitive by default.
Tree-Root Trust	**Transitive**	Intra-Forest	Established automatically between distinct domain trees within the same forest sharing a common schema and configuration (e.g., `corp.local` and `acquiredcompany.com`).
Forest Trust	**Transitive**	Cross-Forest	Links two separate forest root domains. Transitive across all domains within both forests. Supports Kerberos AES encryption and requires Forest Functional Level 2003+.
External Trust	**Non-Transitive**	Cross-Forest / Legacy	Connects two specific domains across different forests, or links to a legacy NT4 domain. Does not flow transitively to child domains.

Trust Category	Transitivity	Boundary Scope	Typical Operational Use Case
Realm Trust	Transitive / Non-Transitive	Non-Windows	Connects an Active Directory domain to an external non-Windows Kerberos realm (such as an MIT Kerberos realm running on Linux/UNIX).
Shortcut Trust	**Transitive**	Intra-Forest	An explicit trust link created between two distant child domains in the same forest to optimize Kerberos referral hop latency across large domain trees.

1.4 Terminology & Core Primitives (Zero-Acronym Onboarding)

- **TDO (Trusted Domain Object):** An Active Directory object of class `trustedDomain` stored under `CN=System,DC=corp,DC=local` that records the operational attributes, direction, partner name, and security flags of a trust relationship.
- **Trust Key:** A shared, symmetric cryptographic password negotiated between the Domain Controllers of two trusting domains, used to seal and unseal Inter-Realm Kerberos referral tickets.
- **Inter-Realm TGT (Referral TGT):** A specialized Kerberos Ticket Granting Ticket issued by a client's local KDC, encrypted with the shared Trust Key, addressed to the Ticket Granting Service (TGS) of the remote trusting domain.
- **SID Filtering (Quarantine):** A security boundary enforcement mechanism where the KDC of a trusting forest inspects the Privilege Attribute Certificate (PAC) of an incoming cross-forest referral ticket and strips out any SIDs that do not belong to the trusted domain's legitimate SID namespace.
- **SID History (`sIDHistory`):** An Active Directory attribute storing previous Security Identifiers assigned to a principal during historical domain migrations, carried inside the PAC to maintain legacy resource access.
- **Name Suffix Routing:** The directory mechanism that routes authentication requests across a Forest Trust based on DNS domain namespaces and NetBIOS names associated with each forest.
- **Selective Authentication:** A trust setting that enforces strict access control by requiring foreign users to be explicitly granted the `Allowed to Authenticate` permission on specific target computer objects before the KDC will issue service tickets.
- **PAM Trust (Privileged Access Management Trust):** A specialized, one-way shadow forest trust (introduced in Windows Server 2016) used to isolate Tier-0 administrative identities inside a dedicated "Bastion" forest.

2. The Cryptographic Anchor: The Trust Key & The TDO

2.1 The Trusted Domain Object (TDO)

Every trust relationship is represented within the Active Directory database by a **Trusted Domain Object (TDO)** residing in the Domain Naming Context under: `CN=<DomainNetBIOSName>,CN=System,DC=corp,DC=local`

Critical TDO Attributes:

- **`trustPartner`:** The Fully Qualified Domain Name (FQDN) of the partner domain (e.g., `eu.corp.local`).
- **`trustDirection`:** Integer bitmask defining the direction of trust:
  - `1`: Inbound Trust (Trusting Domain).
  - `2`: Outbound Trust (Trusted Domain).
  - `3`: Bidirectional / Two-Way Trust.
- **`trustType`:** Integer indicating partner protocol (e.g., `2` for an Active Directory Windows domain, `3` for an MIT Kerberos realm).
- **`trustAttributes`:** 32-bit bitmask governing security boundary controls:
  - `0×00000004` (`TRUST_ATTRIBUTE_QUARANTINED_DOMAINS`): **SID Filtering is enforced.**
  - `0×00000008` (`TRUST_ATTRIBUTE_FOREST_TRANSITIVE`): Trust is forest-wide transitive.
  - `0×00000040` (`TRUST_ATTRIBUTE_CROSS_ORGANIZATION`): Selective Authentication is enabled.

2.2 The Trust Key Architecture

How do two Domain Controllers in completely separate domains establish mutual cryptographic trust without sharing a master administrative password?

The Shared Symmetric Secret: When a trust is established, the Domain Controllers of both domains negotiate a cryptographically random, 30-character shared secret password known as the **Trust Key**.

- In the local domain, the Trust Key is represented as a specialized service principal:

krbtgt/TRUSTED_REALM_NAME " ∧ "quad krbtgt/TRUSTING_REALM_NAME

- The Trust Key password is encrypted and stored in two locations:
  1. Inside `ntds.dit` within the TDO object's secure properties.
  2. Inside the local Domain Controller's **LSA Secrets** registry store (under keys such as `$MACHINE.ACC` and `G$BCKUPKEY`).
- **Automatic Rotation:** Active Directory Domain Controllers automatically rotate this shared Trust Key password **every 30 days**. Like the domain's root `krbtgt` account, Active Directory maintains both the current and previous Trust Key in memory to ensure ongoing authentications are not abruptly severed during rotation.

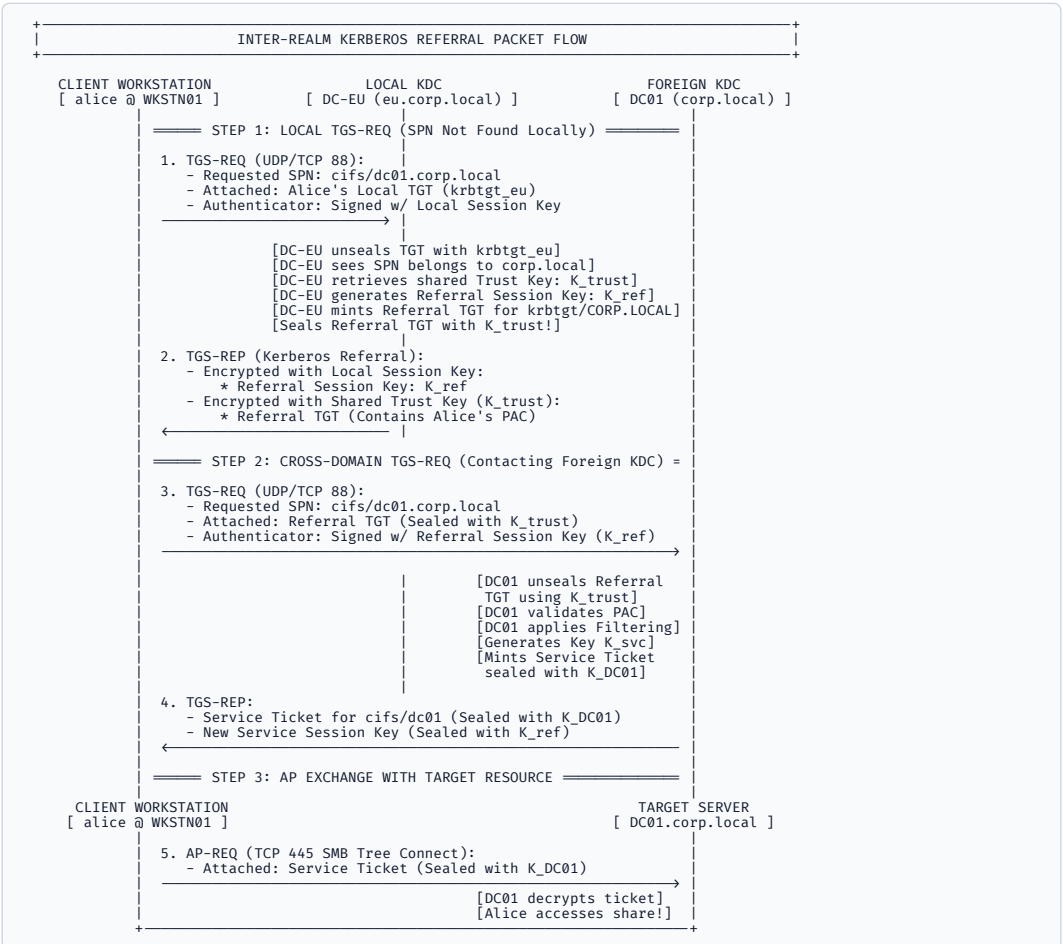
3. Walkthrough: Cross-Domain Kerberos Referral Flow

3.1 The Scenario Setup

To observe how an Inter-Realm Kerberos handshake operates on the wire, trace this enterprise access:

- **User:** `alice@eu.corp.local` (Resides in child domain `eu.corp.local`).
- **Client Machine:** `WKSTN01.eu.corp.local`.
- **Target Resource:** A shared resource hosted on `dc01.corp.local` (Resides in parent domain `corp.local`): `\\dc01.corp.local\shared` (SPN: `cifs/dc01.corp.local`).
- **Local Domain Controller / KDC:** `DC-EU.eu.corp.local`.
- **Foreign Domain Controller / KDC:** `DC01.corp.local`.
- **The Shared Key:** Both DCs possess the inter-domain Trust Key: K_{trust}.

3.2 Diagram: The Inter-Realm Kerberos Referral Exchange



3.3 The Referral Hop Step-by-Step

Step 1: Querying the Local KDC

Alice's workstation possesses a local Ticket Granting Ticket issued by `eu.corp.local`. Alice attempts to connect to `\\dc01.corp.local\shared`. `WKSTN01` sends a `TGS-REQ` packet to `DC-EU` specifying SPN `cifs/dc01.corp.local`.

- `DC-EU` inspects its local directory database. The computer account `DC01$` does not exist in `eu.corp.local`.
- `DC-EU` analyzes the domain suffix (`corp.local`). Because `corp.local` is the direct parent domain, an active trust relationship exists.

Step 2: Minting the Inter-Realm Referral TGT

Instead of returning an error, `DC-EU` generates a **Kerberos Referral**:

- It retrieves the shared **Trust Key** (K_{trust}) corresponding to partner domain `corp.local`.
- It generates an ephemeral **Referral Session Key** (K_{ref}).
- It mints a **Referral TGT** addressed to the principal `krbtgt/CORP.LOCAL`.
  - Inside this ticket, it embeds Alice's identity, her group SIDs, her PAC, and a copy of K_{ref}.
  - **It encrypts this entire ticket using the shared Trust Key (K_{trust}).**
- It sends the Referral TGT back to Alice in a `TGS-REP` message.

Step 3: Presenting the Referral TGT to the Foreign KDC

Alice's workstation receives the referral. Alice cannot decrypt the ticket, but she unseals the response wrapper to obtain the Referral Session Key (K_{ref}).

- Alice constructs a new `TGS-REQ` packet addressed directly to the foreign Domain Controller (`DC01.corp.local`), attaching the Referral TGT and an Authenticator signed with K_{ref}.

Step 4: Foreign KDC Verification & Service Ticket Issuance

`DC01` receives Alice's request:

- `DC01` inspects the ticket header and retrieves its own copy of the shared **Trust Key** (K_{trust}).
- It decrypts the incoming Referral TGT. Because decryption succeeds, `DC01` has mathematical proof that this ticket was minted by an authorized partner Domain Controller (`DC-EU`).
- `DC01` extracts Alice's PAC, executes **SID Filtering** checks (detailed in Section 4), locates the local computer account `DC01$`, retrieves `DC01$`'s machine key (K_{DC01}), and mints a standard Service Ticket for `cifs/dc01.corp.local`.
- Alice receives the final Service Ticket and connects to `DC01` over SMB port 445 without ever transmitting a password across the network.

4. Boundary Enforcement: SID Filtering vs. Intra-Forest Trust Mechanics

Understanding trust architecture requires distinguishing between trusts that enforce hard cryptographic isolation and trusts that serve merely as administrative routing agreements.

4.1 Why Intra-Forest Trust is NOT a Security Boundary

Within a single Active Directory forest (between Parent-Child domains or Tree-Root domains):

- All domains share a single, unified Configuration partition and a single Schema.
- All domains are linked by automatic, two-way transitive Kerberos trusts.
- **SID Filtering is DISABLED by default.**

The Mechanism of SID History Escalation:

When an Active Directory user migrates from Domain A to Domain B, Active Directory populates their legacy SID into their `sIDHistory` attribute. When the user logs in, the KDC includes these historical SIDs in the `ExtraSids` buffer of the PAC so the user retains access to files in their old domain.

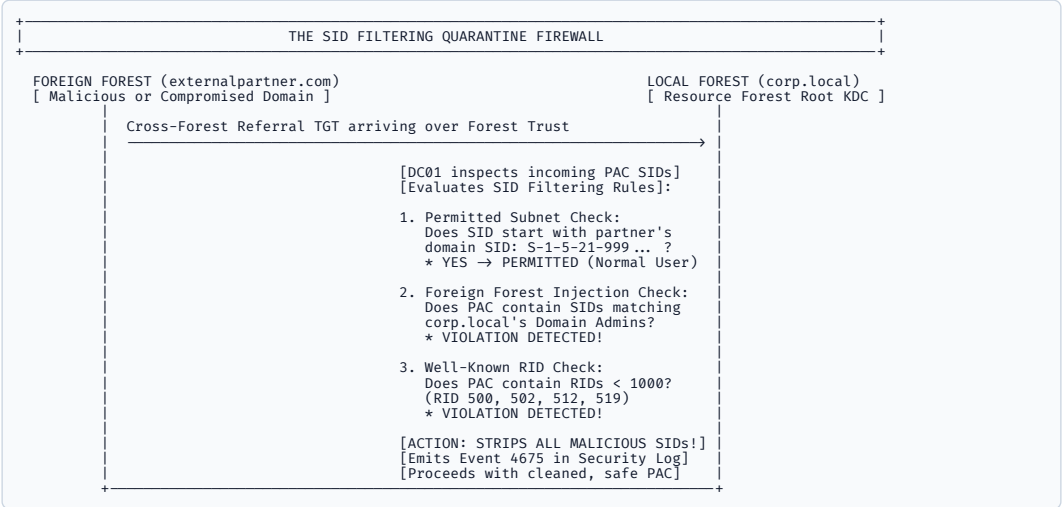
Because child domains inside a forest are assumed to be cooperative:

- If an adversary achieves local administrative compromise of a child domain controller (`DC-EU`), the adversary extracts the shared **Trust Key** (K_{trust}).
- The adversary can forge an Inter-Realm Referral TGT locally.
- **The Exploit Mechanism:** The adversary injects the **Enterprise Admins SID** (`S-1-5-21-<root>-519`) directly into the `ExtraSids` buffer of the forged ticket.
- When presented to the root Domain Controller (`DC01`), the root DC decrypts the ticket with K_{trust}. Because this is an intra-forest trust, **the root DC does not filter out foreign administrative SIDs**. The root DC accepts the Enterprise Admins SID and issues a ticket granting full forest compromise.

> **Architectural Law:** A child domain within a forest cannot defend itself against a compromised parent, and **a parent domain cannot defend itself against a compromised child**. The Forest remains the only true security boundary.

4.2 Why Forest Trusts WITH SID Filtering ARE a Security Boundary

When establishing a **Forest Trust** between two completely separate Active Directory forests (e.g., `corp.local` and `externalpartner.com`), Microsoft enforces **SID Filtering (Quarantine)** by default:



The Rules of SID Filtering (MS-KILE / MS-PAC):

1. **Domain Namespace Verification:** The receiving KDC ensures that all SIDs in the PAC originate exclusively from the domain SID namespace assigned to the trusted partner forest.
2. **Stripping Well-Known Privileged RIDs:** Any SID possessing a Relative Identifier below `1000` (such as `500` for Administrator, `512` for Domain Admins, or `519` for Enterprise Admins) is **unconditionally stripped from the PAC**.
3. **Quarantine Enforcement:** Even if an attacker in the partner forest compromises their own forest root and injects administrative SIDs into a referral ticket, **the receiving forest's KDC removes those SIDs at the trust boundary**. The attacker cannot escalate privileges into the trusting forest.

4.3 Selective Authentication

By default, when a Forest Trust is established, **Forest-Wide Authentication** is active: any user in Forest A can authenticate to any computer in Forest B (though resource DACLs still govern file access).

Administrators can harden cross-forest relationships by enabling **Selective Authentication**:

- **Mechanism:** The KDC in Forest B will **refuse to issue a Service Ticket** for any computer in Forest B unless the incoming user explicitly possesses the **`Allowed to Authenticate`** permission (`ADS_RIGHT_DS_CONTROL_ACCESS` / `0×00000100`) on the target computer's Active Directory object!
- **Impact:** Even if a user from Forest A has valid credentials, they cannot complete an authentication handshake against servers in Forest B unless pre-approved by an administrator.

5. Defensive Telemetry, Detection Engineering & Hardening

5.1 Trust Security Event Telemetry

Cross-domain authentications and trust modifications emit high-priority audit events:

Event ID	Log Name	Operational Meaning	Strategic Significance
4769	Security (DC)	A Kerberos service ticket was requested.	**Inter-Realm Telemetry:** Look for `ServiceName` starting with `krbtgt/` pointing to a foreign realm (e.g., `krbtgt/CORP.LOCAL`).
4675	Security (DC)	SIDs were filtered out of an incoming identity proof.	**High-Value Attack Marker:** Generated whenever the SID Filtering firewall detects and strips unauthorized or forged SIDs from a cross-forest referral PAC!
4820	Security (DC)	A Kerberos TGT request was denied due to Selective Authentication.	Indicates that a foreign user attempted to authenticate to a computer where they lack the `Allowedto Authenticate` permission.
4706 / 4707	Security (DC)	A new trust relationship was created or deleted.	Emitted when a Trusted Domain Object (TDO) is provisioned or removed.

5.2 Detection Heuristics & Hunting Logic

1. Detecting SID Filtering Violations (KQL):

Flags attempts to inject unauthorized cross-forest SIDs through trust boundaries:

```
SecurityEvent
| where EventID == 4675
| project TimeGenerated, SubjectUserName, SubjectDomainName, TargetUserName, Computer, EventData
```

2. Auditing Inter-Realm Ticket Requests (KQL):

Tracks cross-domain activity by monitoring TGS requests addressed to foreign `krbtgt` accounts:

```
SecurityEvent
| where EventID == 4769
| where ServiceName startswith "krbtgt/" and ServiceName !endswith "corp.local"
| project TimeGenerated, TargetUserName, ClientAddress, ServiceName, TicketOptions
```

5.3 Enterprise Hardening Blueprint

Securing Active Directory trust relationships requires strict perimeter filtering:

1. **Enforce SID Filtering on All External Trusts:**
   - Verify that SID Filtering (Quarantine) is enabled across all non-intra-forest trusts using `netdom`:

```
netdom trust corp.local /Domain:externalpartner.com /FilterSIDs:Yes
```

1. **Disable SID History on Forest Trusts:**
   - Unless actively executing a cross-forest domain migration, explicitly disable SID History across forest trusts:

```
netdom trust corp.local /Domain:externalpartner.com /EnableSIDHistory:No
```

1. **Deploy Selective Authentication for Sensitive Resource Domains:**
   - Configure external and forest trusts to use **Selective Authentication** rather than Forest-Wide Authentication, ensuring only authorized users can authenticate to specific server objects.
2. **Isolate Administrative Forests with PAM Trusts:**
   - When deploying Red Forest / Bastion Forest architectures, use one-way incoming PAM trusts with **Shadow Principals** and strictly audited SID mapping, ensuring administrative identities never cross into untrusted user tiers.

6. Authoritative Specifications, RFCs & Cross-Vault Links

6.1 Authoritative Specifications

- **[MS-KILE]:** *Kerberos Protocol Extensions* — Section 3.2.5.4 details cross-realm referral ticket issuance, routing, and trust key decryption.
- **[MS-PAC]:** *Privilege Attribute Certificate Data Structure* — Section 4.1.2 defines the mathematical rules of the SID Filtering algorithm and prohibited RIDs.
- **[MS-ADTS]:** *Active Directory Technical Specification* — Section 6.1.6 defines the schema attributes and flags of `trustedDomain` objects (TDOs).
- **RFC 4120:** *The Kerberos Network Authentication Service (V5)* — Section 3.3 documents Inter-Realm authentication and referral path traversal.

About the Author

Ahmed Sophy Elzeky (0xZiki)

Cybersecurity Software Engineer & Offensive Tool Architect
ahmedelzeky2446@gmail.com | Mansoura, Egypt

Ahmed Sophy Elzeky, known in professional security circles as **0xZiki**, is an offensive security engineer and systems architect with deep specialization in enterprise identity protocol internals, raw network protocol engineering, and production-grade security tooling. He is pursuing his Bachelor of Science in Computer Science at Mansoura University, Egypt.

Ahmed's engineering work is characterized by a commitment to zero-dependency, systems-level implementation. His **Nemesis Core Framework** is a high-concurrency web security platform capable of processing over one thousand requests per second through dual-engine asynchronous architectures and circuit-breaker resilience patterns. **GhostSniffer** is a stealth network discovery engine built entirely on raw sockets with manual RFC 1071 checksum implementations and half-open SYN scanning—designed for environments where third-party dependencies are operationally inadmissible. **UltronNet** provides encrypted custom-protocol communication with automated TLS negotiation, bidirectional port forwarding, and full PTY allocation for persistent access scenarios.

His research agenda centers on the deep mechanics of enterprise identity architectures: Kerberos state machine behavior under adversarial conditions, NTLM relay chain construction, DPAPI key material extraction, Active Directory Certificate Services misconfigurations across the ESC class of vulnerabilities, and the access token inheritance model across Windows privilege boundaries. This research has been validated across extensive enterprise simulation ranges, informing both the theoretical framework and the practical attack chains documented throughout this volume.

Ahmed authors the **Active Directory Arsenal** project to bridge the gap between OS internals scholarship and production-grade offensive engineering—making protocol-level knowledge accessible to the next generation of security practitioners.